**ZEROSHEILD PRIVATE LIMITED**
C3iHub, 2nd Floor, New Technopark Building (In front of NCC Office),
IIT Kanpur, Kalyanpur, Kanpur Nagar, Uttar Pradesh – 208016
**Ph – +91 – 8088054916**
**www.zeroshield.ai**

EXPRESSION OF INTEREST (EoI) RESPONSE

# Comprehensive Vulnerability Assessment Framework for Large Language Models

## A Sovereign, Air-Gapped, Automated Security Assessment and Certification Capability for Defence-Grade Large Language Models

| | |
|---|---|
| **Submitted to** | Director, Directorate of Technology Development Fund (DTDF), Defence Research and Development Organisation (DRDO), DRDO Bhawan, Rajaji Marg, New Delhi – 110011 |
| **EoI Reference No.** | DGTM/01/100010/M/48/VAF/PC1 |
| **Scheme** | Technology Development Fund (TDF), DRDO – Grant-in-Aid for indigenous defence technology (MSME / Start-up category) |
| **Submitted by** | ZeroShield Private Limited |
| **CIN** | U62020UT2025PTC020093 |
| **Category** | Private Limited Company – Micro Enterprise (Udyam MSME) – Indian-owned and Indian-controlled |
| **Document Reference** | ZS/EoI/DRDO-TDF/VAF-LLM/2026/01 |
| **Classification** | RESTRICTED – Commercial-in-Confidence (for evaluation use only) |
| **Date of Submission** | ____ / ____ / 2026 |
| **Language of Submission** | English |

*This document constitutes the response of ZeroShield Private Limited to the Expression of Interest cited above. It is submitted solely for the purpose of technical evaluation and short-listing under the TDF Scheme and does not, by itself, constitute a bid, tender, or commercial offer. In conformance with clause 4.2.1 of the EoI, no cost, price, quotation, or financial figure relating to the proposed project is stated anywhere in this response. Statutory share-capital figures, where reproduced, are matters of public record maintained by the Ministry of Corporate Affairs and are disclosed only for the purpose of establishing the legal and financial status of the applicant, and are not to be construed as any indication of project cost.*

ZEROSHEILD PRIVATE LIMITED
C3iHub, 2nd Floor, New Technopark Building (In front of NCC Office),
IIT Kanpur, Kalyanpur, Kanpur Nagar, Uttar Pradesh – 208016
Ph – +91 – 8088054916
www.zeroshield.ai

# Covering Letter

Ref: ZS/EoI/DRDO-TDF/VAF-LLM/2026/01
Date: ____ / ____ / 2026

To,

The Director,
Directorate of Technology Development Fund (DTDF),
Defence Research and Development Organisation (DRDO),
DRDO Bhawan, Rajaji Marg, New Delhi – 110011.

**Subject:** Submission of Expression of Interest in response to EoI No. DGTM/01/100010/M/48/VAF/PC1 for the "Comprehensive Vulnerability Assessment Framework for Large Language Models".

**Reference:** Request for Expression of Interest issued by DTDF, DRDO under the Technology Development Fund Scheme (MSME / Start-up category).

Respected Sir,

We, ZeroShield Private Limited, an Indian private limited company incorporated under the Companies Act, 2013 and registered as a Micro Enterprise under the Udyam (MSME) framework, are pleased to submit our Expression of Interest in response to the subject EoI. Having carefully examined the EoI document and all the terms, conditions, instructions, and eligibility criteria contained therein, we hereby confirm our interest in, and our technical capability for, the indigenous design, development, and delivery of the proposed Comprehensive Vulnerability Assessment Framework for Large Language Models.

Our firm specialises in the security of Artificial Intelligence and Large Language Model systems, encompassing adversarial testing, red-teaming, model-artefact assurance, privacy auditing, and compliance certification. The proposed framework is squarely within our core domain of competence. We have organised our response to address, in full and clause-by-clause, the problem definition, scope of work, detailed specifications, and the essential qualification criteria set out in Part-II of the EoI, and we have provided supporting technical depth in the form of system architecture, design artefacts, security architecture, deployment topology, a work breakdown structure, a delivery schedule completed within twenty-one (21) months — inside the stipulated twenty-four (24) month ceiling — a risk-management plan, and a compliance traceability matrix.

We confirm that the firm is owned and controlled by resident Indian citizens, that it carries no foreign shareholding whatsoever, that it has no shareholder, direct or indirect, from any country sharing a land border with India, and that it has never been debarred, banned, blacklisted, or placed under suspension or hold by the Ministry of Defence or any other arm of the Government of India. The firm is not under liquidation, court receivership, or any similar proceeding, and there is no litigation or arbitration history against the firm.

We further undertake, as required by the EoI, that all model weights, prompt corpora, vulnerability findings, and reports arising from this engagement shall remain within the secure Indian perimeter throughout development, testing, and deployment, and that no discovered vulnerability or zero-day shall be sold, transferred, or disclosed to any third party without the explicit written authorisation of the End-User. We are willing to execute a Non-Disclosure Understanding (NDU) with TDF as and when required.

In strict compliance with clause 4.2.1 of the EoI, we confirm that no cost, price, or financial figure relating to the proposed project has been mentioned anywhere in this response. All information and documents submitted herewith are true, correct, and self-certified by the authorised signatory of the firm, and we accept that DTDF reserves the right to verify the originals.

We look forward to participating in the subsequent stages of the process and to contributing to the establishment of a sovereign, defence-grade LLM security assessment capability for the nation.

Thanking you,

Yours faithfully,

For and on behalf of **ZeroShield Private Limited**

(Vartul Goyal)

Director & Chief Technology Officer (Authorised Signatory)
DIN: 11337955 | Email: vartul@zeroshield.ai | Mobile: 8088054916

# Document Control and Distribution

This section records the control metadata, revision history, and distribution restrictions applicable to this Expression of Interest response. The document is a controlled deliverable and is to be handled in accordance with its security classification.

**Table 1. Document Control Record**

| Attribute | Detail |
|---|---|
| Document title | Expression of Interest Response – Comprehensive Vulnerability Assessment Framework for Large Language Models |
| Document reference | ZS/EoI/DRDO-TDF/VAF-LLM/2026/01 |
| EoI reference | DGTM/01/100010/M/48/VAF/PC1 |
| Version | 1.0 (Final for submission) |
| Status | Released for EoI submission |
| Classification | RESTRICTED – Commercial-in-Confidence |
| Prepared by | Technical Proposals Group, ZeroShield Private Limited |
| Reviewed by | Chief Technology Officer |
| Approved by | Board of Directors, ZeroShield Private Limited |
| Authorised signatory | Vartul Goyal, Director & CTO (DIN 11337955) |
| Language | English |
| Total figures | Thirty-one (31) |
| Total tables | Thirty-plus (numbered sequentially) |
| Annexures | A through F |

Source: ZeroShield Private Limited document management register.

*Interpretation: The document is a single controlled release (v1.0) prepared specifically for this EoI and supersedes any prior draft.*

**Table 2. Revision History**

| Ver. | Date | Description of change | Author |
|---|---|---|---|
| 0.1 | 2026 | Initial structure and requirement mapping | Tech. Proposals Group |
| 0.5 | 2026 | Technical content, architecture, and design artefacts incorporated | CTO / Tech. Proposals Group |
| 0.9 | 2026 | Eligibility response, annexures, and compliance matrix added; internal review | CTO |
| 1.0 | 2026 | Final review, approval, and release for submission | Board / CTO |

Source: Internal version-control log.

*Interpretation: Revisions reflect progressive consolidation; only v1.0 is submitted.*

**Table 3. Distribution List**

| Recipient / Holder | Purpose | Copy |
|---|---|---|
| Director TDF, DTDF, DRDO | EoI evaluation and short-listing | Controlled e-copy via DefProc portal |
| ZeroShield Private Limited (master) | Record and audit | Controlled master copy |

Source: ZeroShield Private Limited distribution register.

*Interpretation: Circulation is restricted to the evaluating authority and the applicant's own controlled records.*

**Handling notice.** This document contains information proprietary to ZeroShield Private Limited and is submitted in confidence. It shall not be reproduced, disclosed, or used, in whole or in part, for any purpose other than the evaluation of this EoI, save with the written consent of the firm or as required by the Government of India in connection with the TDF process. The classification marking applies to the document as a whole.

# Table of Contents

**Covering Letter** . . . 2

**Document Control and Distribution** . . . 4

**List of Abbreviations and Acronyms** . . . 23

**1. Name of the Firm** . . . 24

**2. Contact Details** . . . 24

**3. Title of the Project** . . . 24

**4. Project Description** . . . 24

4.1 Introduction and Background . . . 26

4.1.1 Purpose of This Document . . . 26

4.1.2 Strategic Context . . . 26

4.1.3 The Sovereignty Imperative . . . 27

4.1.4 Objectives of the Proposed Capability . . . 27

4.1.5 Scope and Structure of This Response . . . 28

4.1.6 About ZeroShield Private Limited . . . 28

4.2 Problem Definition and Threat Landscape . . . 29

4.2.1 Restatement of the Problem . . . 29

4.2.2 A Taxonomy of LLM Vulnerabilities . . . 29

4.2.3 India-Specific Threat Surfaces . . . 31

4.2.4 Consequences of Inaction . . . 32

4.3 Limitations of Existing Approaches . . . 33

4.3.1 Categories of Existing Tooling . . . 33

4.3.2 Gap Analysis . . . 33

4.3.3 Specific Weaknesses Addressed . . . 34

4.3.4 Positioning Statement . . . 34

4.4 Solution Overview and Design Principles . . . 35
4.4.1 Solution Concept . . . 35
4.4.2 Design Principles . . . 35
4.4.3 Conformance to the EoI Scope . . . 35
4.5 System Architecture . . . 37
4.5.1 Architectural Overview . . . 37
4.5.2 Layered Responsibilities . . . 38
4.5.3 End-to-End Assessment Pipeline . . . 39
4.5.4 Cross-Cutting Services . . . 39
4.6 The Six Assessment Modules . . . 40
4.6.1 Module M1 — Model Ingestion and Secure Setup . . . 40
4.6.2 Module M2 — Static Analysis and Artefact Scanning . . . 41
4.6.3 Module M3 — Adversarial Attack Generation . . . 42
4.6.4 Module M4 — Agentic Red-Teaming . . . 43
4.6.5 Module M5 — Privacy, API, and Embedding Audit . . . 44
4.6.6 Module M6 — Certification and Reporting . . . 45
4.6.7 Inter-Module Data Model . . . 46
4.7 Functional and Performance Specifications . . . 47
4.7.1 Approach to Specification . . . 47
4.7.2 Functional Specifications . . . 47
4.7.3 Performance Specifications and Figures of Merit . . . 47
4.7.4 OWASP, ATLAS, and NIST Coverage . . . 49
4.7.5 Multilingual Specification . . . 49
4.8 Technology Stack . . . 51
4.8.1 Selection Principles . . . 51
4.8.2 Stack by Layer . . . 51
4.8.3 Offline Dependency Management . . . 52
4.9 Data Architecture and Data-Flow . . . 53
4.9.1 Data Domains . . . 53
4.9.2 Data Life-Cycle . . . 53
4.9.3 Context-Level Data-Flow (Level 0) . . . 53

4.9.4 Detailed Data-Flow (Level 1) . . . 54

4.10 Detailed Design . . . 56

4.10.1 Component Model . . . 56

4.10.2 Domain Model . . . 56

4.10.3 Behavioural Design — Assessment Run . . . 57

4.10.4 Behavioural Design — Agentic Red-Team Loop . . . 58

4.10.5 Assessment State Model . . . 59

4.11 Security Architecture . . . 61

4.11.1 Security Objectives . . . 61

4.11.2 Trust Zones and Isolation . . . 61

4.11.3 Controls . . . 61

4.11.4 Threats to the Framework and Mitigations . . . 62

4.12 Deployment and Network Topology . . . 63

4.12.1 Deployment Model . . . 63

4.12.2 Network Topology and Segmentation . . . 63

4.12.3 Air-Gap Conformance . . . 64

4.12.4 Scalability and Portability . . . 64

4.13 Work Breakdown Structure . . . 65

4.13.1 Decomposition Approach . . . 65

4.13.2 Work-Package Register . . . 65

4.13.3 Definition of Done and Acceptance . . . 66

4.14 Detailed OWASP LLM Top 10 (2025) Coverage and Test Approach . . . 67

4.14.1 Purpose . . . 67

4.14.2 Risk-to-Module-to-Metric Mapping . . . 67

4.14.3 Per-Risk Test Approach . . . 67

4.14.4 ATLAS and NIST Alignment . . . 69

4.15 Evaluation Methodology, Metrics and Statistical Validation . . . 70

4.15.1 Principles of Measurement . . . 70

4.15.2 Metric Definitions . . . 70

4.15.3 Statistical Validation . . . 71

4.15.4 Acceptance Methodology per Figure of Merit . . . 71

4.15.5 Measurement Integrity . . . 72

4.16 Detailed Module Specifications . . . 73

4.16.1 Specification Format . . . 73

4.16.2 M1 — Model Ingestion and Secure Setup . . . 73

4.16.3 M2 — Static Analysis and Artefact Scanning . . . 73

4.16.4 M3 — Adversarial Attack Generation . . . 73

4.16.5 M4 — Agentic Red-Teaming Engine . . . 74

4.16.6 M5 — Privacy, API and Embedding Audit . . . 74

4.16.7 M6 — Scoring, Certification and Reporting . . . 74

4.16.8 Inter-Module Contract . . . 75

4.17 Security Controls Catalogue and Standards Cross-Reference . . . 76

4.17.1 Control Framework Approach . . . 76

4.17.2 Controls Catalogue . . . 76

4.17.3 ISO/IEC 27001:2022 Theme Cross-Reference . . . 76

4.17.4 NIST AI RMF Function Mapping . . . 77

4.18 Assumptions, Constraints, Dependencies and Interfaces . . . 78

4.18.1 Assumptions . . . 78

4.18.2 Constraints . . . 78

4.18.3 External Dependencies . . . 78

4.18.4 Interface Contracts . . . 79

4.18.5 Consolidated Exit Criteria . . . 80

4.19 Comparative Analysis of Existing Approaches and Differentiators . . . 81

4.19.1 Purpose and Method of Comparison . . . 81

4.19.2 Limitations of Existing Approaches . . . 81

4.19.3 Comparative Capability Matrix . . . 82

4.19.4 Differentiators of the Proposed Framework . . . 82

4.20 Proof-of-Concept Definition and Test-Case Taxonomy . . . 84

4.20.1 Scope of the Proof-of-Concept . . . 84

4.20.2 Test-Case Taxonomy and Indicative Distribution . . . 84

4.20.3 Test-Case Structure . . . 84

4.20.4 Proof-of-Concept Success Definition . . . 85

4.21 Deployment, Operations, Training and Handover . . . 86
4.21.1 Deployment into an Accredited Environment . . . 86
4.21.2 Operations and Maintenance . . . 86
4.21.3 Training and Knowledge Transfer . . . 86
4.21.4 Documentation and Handover Artefacts . . . 86
4.21.5 Post-Handover Support and Sustainment . . . 87
4.22 AI Capability Overview and Project Description . . . 88
4.22.1 Restatement of the Problem in AI Terms . . . 88
4.22.2 The Proposed Solution in Brief . . . 88
4.22.3 Relationship to the Wider Framework . . . 88
4.22.4 Mapping of EoI Deliverables to Sections . . . 88
4.23 Work Breakdown Structure of the AI Capability . . . 90
4.23.1 Decomposition into Modules and Subsystems . . . 90
4.23.2 Hardware Elements . . . 90
4.24 Solution Architecture and End-to-End Block Diagram . . . 91
4.24.1 End-to-End Flow . . . 91
4.24.2 Architectural Properties . . . 92
4.25 Technology Stack . . . 93
4.25.1 Component Selection and Rationale . . . 93
4.26 Transformer-Based Language-Model Approach . . . 95
4.26.1 Base-Model Selection . . . 95
4.26.2 Domain-Adaptive Pre-training . . . 95
4.26.3 Parameter-Efficient Fine-Tuning . . . 95
4.26.4 Demonstrated Methodology . . . 96
4.27 Supervised Fine-Tuning and Instruction Tuning . . . 97
4.27.1 Training Tasks and Data . . . 97
4.27.2 Evaluation and Model Card . . . 97
4.28 Cybersecurity Corpus Curation . . . 98
4.28.1 Sources . . . 98
4.28.2 Cleaning, Labelling and Governance . . . 98

4.28.3 Sample Record Schema . . . 99

4.29 Retrieval-Augmented Generation (RAG) Design . . . 100

4.29.1 Indexing and Retrieval . . . 100

4.29.2 Correlation and Grounding . . . 100

4.29.3 Knowledge-Base Maintenance . . . 101

4.30 Agentic AI Framework . . . 102

4.30.1 Planner, Executor and Tools . . . 102

4.30.2 Proof-of-Concept Chain Reasoning . . . 102

4.30.3 Controls on Agency . . . 103

4.31 Air-Gapped Deployment Strategy . . . 104

4.31.1 Principles . . . 104

4.31.2 Air-Gap Controls for the AI Capability . . . 104

4.32 Vulnerability Detection and Evaluation Plan . . . 105

4.32.1 Detection Approach . . . 105

4.32.2 Evaluation Datasets . . . 105

4.32.3 Metrics and Targets . . . 106

4.33 Malware Analysis and Reverse-Engineering Capability . . . 107

4.33.1 Tooling . . . 107

4.33.2 Analysis Workflow . . . 108

4.33.3 Safety and Containment . . . 108

4.34 Secure Code Patching Approach . . . 109

4.34.1 Patch Generation . . . 109

4.34.2 Patch Validation . . . 109

4.34.3 Fix Rationale and Human Sign-off . . . 109

4.35 Proof-of-Concept Plan for the AI Capability . . . 110

4.35.1 Type-I Demonstration (Source Code) . . . 110

4.35.2 Type-II Demonstration (Binary / Malware) . . . 110

4.35.3 Proof-of-Concept Scope and Success . . . 110

4.36 Testing and Qualification Plan . . . 112

4.36.1 Levels of Testing . . . 112

4.36.2 Analyst Validation . . . 112
4.36.3 Qualification Readiness . . . 112

**5. Development Estimates (with respect to Time) . . . 113**

5.1 Project Schedule, Milestones and Roadmap . . . 113
5.1.1 Scheduling Philosophy . . . 113
5.1.2 Phases and Milestones . . . 114
5.1.3 Deliverables Schedule . . . 115
5.1.4 Suo-Moto Design and Development Declaration . . . 116

5.2 Development Timeline, Milestone Deliverables and Suo-moto Capability . . . 116
5.2.1 Development Timeline (≤ 21 Months) . . . 116
5.2.2 Milestone-wise Deliverables . . . 116
5.2.3 Suo-moto Similar Product and Technology . . . 117

**6. Likely Academic Partners / R&D Institutions Required . . . 118**

**7. Existing Academia / Research Institute Support . . . 118**

**8. Likely Indigenous Content in the Proposed Solution . . . 118**

8.1 Indigenous Content and Capability Roadmap . . . 118
8.1.1 Indigenous-Content Position . . . 118
8.1.2 Composition of Indigenous Content . . . 119
8.1.3 Capability-Maturity Roadmap . . . 120
8.1.4 Strategic Value of Indigenous Capability . . . 121

8.2 Indigenous Content Declaration . . . 122
8.2.1 Declaration . . . 122
8.2.2 Indicative Breakdown . . . 122

**9. Risk Identification and Mitigation Plan . . . 124**

9.1 Risk Identification and Mitigation . . . 124
9.1.1 Risk-Management Method . . . 124
9.1.2 Risk Register . . . 125

9.1.3 Risk Governance . . . 126

**10. Additional Infrastructure Requiring Funding Support . . . 126**

**11. Experience in Design & Development including Associated Tools . . . 126**

11.1 AI / ML / LLM Project and Capability Evidence . . . 127

11.1.1 Demonstrated Capability Areas . . . 127

11.1.2 Evidence to Be Furnished . . . 128

11.1.3 Representative Project Evidence . . . 128

11.2 Cybersecurity Product and Domain Evidence . . . 129

11.2.1 Product Portfolio (Self-Declared) . . . 129

11.2.2 Architectural Competence . . . 129

11.3 Open-Source Engagement, Hackathons and Publications . . . 130

11.3.1 Engagement with the Open-Source Ecosystem . . . 130

11.3.2 Acceptable Evidence and What Will Be Furnished . . . 130

**12. Essential Test Infrastructure Required for Solution . . . 131**

12.1 Isolated / Air-Gappable Test Environment . . . 131

12.1.1 Isolation Policy . . . 132

12.1.2 Host Inventory . . . 132

12.2 Inference Serving Setup . . . 134

12.2.1 Serving Options . . . 134

12.2.2 Serving Configuration . . . 134

12.2.3 Representative Serving Evidence . . . 134

12.3 Source Control and CI/CD Evidence . . . 136

12.3.1 Practices . . . 136

12.3.2 Representative Repository Structure . . . 137

12.3.3 Representative CI Evidence . . . 137

12.4 Secure Data Ingestion Practice . . . 139

12.4.1 Procedure . . . 139

12.4.2 Representative Ingestion Evidence . . . 140

12.5 Code-Analysis Sandbox . . . 141
12.5.1 Sandbox Controls . . . 141
12.5.2 Representative Sandbox Evidence . . . 142

**13. Availability of Experienced Manpower . . . 143**

**14. Management Capabilities . . . 143**

14.1 Quality, Security and Engineering Discipline . . . 144
14.1.1 Management-Capability Posture . . . 144
14.1.2 Information-Security Discipline (ISO/IEC 27001 Alignment) . . . 144
14.1.3 AI Risk Management (NIST AI RMF and ISO/IEC 42001 Awareness) . . . 144
14.1.4 Secure Software Development and DevSecOps . . . 145
14.1.5 Independent Verification and Validation . . . 145
14.1.6 Quality Records and Configuration Management . . . 145

14.2 Secure SDLC and DevSecOps Process . . . 146
14.2.1 Phase Activities . . . 146

14.3 AI Risk-Management Awareness (NIST AI RMF / ISO/IEC 42001) . . . 147
14.3.1 Mapping to NIST AI RMF Functions . . . 147
14.3.2 Relationship to ISO/IEC 42001 . . . 147

**15. Undertaking by the Firm . . . 148**

**16. Organization Structure, Legal Status and Current Contract Commitments / Works in Progress . . . 148**

**17. Financial Capabilities . . . 150**

**18. Litigation / Arbitration History . . . 150**

**Statement of Compliance, Authenticity and Self-Certification . . . 150**

**Annexure A — Letter of Willingness and Undertaking . . . 152**

**Annexure B — Compliance Traceability Matrix . . . 154**

**Annexure C — Key-Personnel Profile and Role Specifications . . . . . 155**

E.1 Technical Authority — Director and Chief Technology Officer . . . . . 155

E.2 Role Specifications for Positions to be Staffed . . . . . 155

**Annexure D — References, Standards and Bibliography . . . . . 157**

F.1 Standards and Frameworks . . . . . 157

F.2 Selected Research and Technical Literature . . . . . 157

**Annexure E — Glossary of Key Terms . . . . . 158**

**Annexure F — Defence-Specific Threat Considerations and Assurance Rationale . . . . . 158**

H.1 Defence-Specific Threat Scenarios . . . . . 159

H.2 Defence-Concern-to-Capability Mapping . . . . . 159

H.3 Assurance Rationale . . . . . 160

H.4 Boundary of Assurance . . . . . 160

# List of Figures

*Figures are numbered sequentially in order of appearance.*

Figure 1. Operational and threat context for defence-grade LLM assurance . . . 26

Figure 2. Consolidated threat taxonomy for Large Language Models . . . 29

Figure 3. Logical system architecture within the air-gapped perimeter . . . 37

Figure 4. End-to-end assessment pipeline (process flow of a single assessment) . . . 39

Figure 5. Module M1 — model ingestion and secure setup (stage view) . . . 40

Figure 6. Module M2 — static analysis and artefact scanning (stage view) . . . 41

Figure 7. Module M3 — adversarial attack generation (stage view) . . . 42

Figure 8. Module M4 — agentic multi-turn red-teaming (stage view) . . . 43

Figure 9. Module M5 — privacy, API, and embedding audit (stage view) . . . 44

Figure 10. Module M6 — certification scoring and reporting (stage view) . . . 45

Figure 11. Target capability profile versus typical existing-tool coverage . . . 48

Figure 12. Framework coverage map (OWASP LLM Top 10, MITRE ATLAS, NIST AI RMF) . . . 49

Figure 13. Multilingual attack-surface coverage and transformation pipeline . . . 49

Figure 14. Indicative technology stack across the architectural layers . . . 51

Figure 15. Data life-cycle from model intake to certified disposal . . . 53

Figure 16. Level-0 (context) data-flow diagram . . . 53

Figure 17. Level-1 data-flow diagram (decomposition into module-processes) . . . 54

Figure 18. UML component diagram of the assessment framework . . . 56

Figure 19. UML class (domain) diagram of core entities . . . 56

Figure 20. Sequence diagram — end-to-end assessment run . . . 57

Figure 21. Sequence diagram — multi-turn agentic red-team loop . . . 58

Figure 22. Assessment life-cycle state machine . . . . . 59

Figure 23. Security architecture and trust zones . . . . . 61

Figure 24. Deployment topology on air-gapped GPU infrastructure . . . . . 63

Figure 25. Network topology and segmentation within the air-gapped perimeter . . . . . 63

Figure 26. Work breakdown structure for the proof-of-concept programme . . . . . 65

Figure 27. End-to-end block diagram of the AI vulnerability detection and remediation capability . . . . . 91

Figure 28. Layered technology stack of the AI capability . . . . . 93

Figure 29. Model design and training / fine-tuning pipeline . . . . . 95

Figure 30. Cybersecurity corpus curation pipeline . . . . . 98

Figure 31. Illustrative curated-corpus record schema . . . . . 99

Figure 32. Retrieval-augmented generation architecture with vector database . . . . . 100

Figure 33. Agentic multi-step analysis and proof-of-concept chain reasoning . . . . . 102

Figure 34. Source-code vulnerability detection and evaluation pipeline . . . . . 105

Figure 35. Indicative detection-performance targets by dataset . . . . . 106

Figure 36. Malware analysis and reverse-engineering workflow . . . . . 107

Figure 37. Secure code patching workflow . . . . . 109

Figure 38. Proof-of-concept flows for Type-I source code and Type-II binary / malware . . . . . 110

Figure 39. Indicative twenty-one-month schedule by work package . . . . . 113

Figure 40. Indicative indigenous-content profile by solution dimension . . . . . 118

Figure 41. Indicative capability-maturity roadmap for the framework . . . . . 120

Figure 42. Indigenous content and sovereignty of the proposed capability . . . . . 122

Figure 43. Risk heat matrix positioning the ten principal programme risks . . . . . 124

Figure 44. Representative fine-tuning run log and held-out evaluation summary . . . . . 128

Figure 45. Isolated, air-gappable test-environment network . . . . . 131

Figure 46. Representative egress-verification output from a test-environment host . . . . . 133

Figure 47. Representative inference-serving start-up and smoke-test output . . . . . 134

Figure 48. Source control and CI/CD pipeline with embedded security gates . . . . . 136

Figure 49. Representative repository structure of the framework . . . . . 137

Figure 50. Representative CI run — signed commits, pipeline stages, SBOM and signing . . . . . 137

Figure 51. Secure data-ingestion standard operating procedure . . . . . 139

Figure 52. Representative data-ingestion batch manifest and log . . . . . 140

Figure 53. Code-analysis sandbox architecture . . . . . 141

Figure 54. Representative code-analysis sandbox run output . . . . . 142

Figure 55. Secure SDLC / DevSecOps process with security woven through every phase . . . . . 146

Figure 56. Project organisation and governance structure . . . . . 148

# List of Tables

*Tables are numbered sequentially in order of appearance.*

Table 1. Document Control Record . . . 4
Table 2. Revision History . . . 4
Table 3. Distribution List . . . 4
Table 4. Abbreviations and Acronyms Used in This Document . . . 23
Table 5. Identity of the applicant firm . . . 24
Table 6. Critical technologies . . . 25
Table 7. OWASP Top 10 for LLM Applications (2025) mapped to assessment modules . . . 31
Table 8. Comparative gap analysis of existing approaches versus the proposed framework . . . 33
Table 9. Traceability from EoI-stated system composition to proposed modules . . . 35
Table 10. Functional specification, module assignment, and verification approach . . . 47
Table 11. Performance parameters, targets, and figures of merit . . . 47
Table 12. Technology stack by architectural layer with rationale . . . 51
Table 13. Security controls, purpose, and placement . . . 61
Table 14. Work-package register and mapping to assessment modules . . . 65
Table 15. OWASP LLM Top 10 (2025) mapped to modules and figures of merit . . . 67
Table 16. Definitions of the principal evaluation metrics . . . 70
Table 17. Acceptance methodology binding figures of merit to metrics and procedures . . . 71
Table 18. Specification of module M1 . . . 73
Table 19. Specification of module M2 . . . 73
Table 20. Specification of module M3 . . . 73
Table 21. Specification of module M4 . . . 74

Table 22. Specification of module M5 . . . . . 74

Table 23. Specification of module M6 . . . . . 74

Table 24. Security controls catalogue for the assessment framework . . . . . 76

Table 25. Indicative alignment with ISO/IEC 27001:2022 control themes . . . . . 76

Table 26. Mapping of the framework to NIST AI RMF functions . . . . . 77

Table 27. External dependency categories and management basis . . . . . 78

Table 28. Finding schema — principal fields . . . . . 79

Table 29. Certification-report schema — principal sections . . . . . 79

Table 30. Comparative capability matrix — existing approaches versus the proposed framework . . . . . 82

Table 31. Indicative test-case taxonomy and distribution (~100 cases) . . . . . 84

Table 32. Structure of a validated test case . . . . . 84

Table 33. Training and knowledge-transfer plan . . . . . 86

Table 34. Mapping of EoI-specified deliverable items to the sections that address them . . . . . 88

Table 35. Work breakdown of the AI capability into modules, subsystems and principal elements . . . . . 90

Table 36. AI technology stack — representative components and selection rationale . . . . . 93

Table 37. Indicative model-card structure for a fine-tuned model . . . . . 97

Table 38. Principal corpus sources and their contribution . . . . . 98

Table 39. Air-gap controls specific to the AI capability . . . . . 104

Table 40. Evaluation datasets and their role . . . . . 105

Table 41. Reverse-engineering capabilities and their use . . . . . 107

Table 42. Proof-of-concept scope and success definition for the AI capability . . . . . 110

Table 43. Levels of testing for the AI capability . . . . . 112

Table 44. Programme milestones and exit criteria . . . . . 114

Table 45. Principal deliverables and availability by milestone . . . . . 115

Table 46. Phase-wise development timeline for the AI capability (within 21 months) . . . 116
Table 47. Milestone-wise deliverables for the AI capability . . . 116
Table 48. Indigenous and vendored elements by dimension . . . 119
Table 49. Indicative indigenous-content breakdown by element . . . 122
Table 50. Risk register with likelihood, impact, severity and mitigation . . . 125
Table 51. Experience in design and development (essential criteria A1–A9) . . . 126
Table 52. Demonstrated AI / ML / LLM capability areas . . . 127
Table 53. Representative AI/ML/LLM evidence artefacts and availability . . . 128
Table 54. Firm cybersecurity product-portfolio areas (self-declared) . . . 129
Table 55. Acceptable capability evidence and the firm's position . . . 130
Table 56. Essential test infrastructure (A1–A7) . . . 131
Table 57. Isolation policy for the test environment . . . 132
Table 58. Representative host inventory of the isolated enclave . . . 132
Table 59. Inference-serving options and their use . . . 134
Table 60. Representative inference-serving configuration . . . 135
Table 61. Source-control and CI/CD practices . . . 136
Table 62. Secure data-ingestion SOP . . . 139
Table 63. Code-analysis sandbox controls . . . 141
Table 64. Availability of experienced manpower (A1–A7) . . . 143
Table 65. Management capabilities available with the firm . . . 143
Table 66. Management-capability options and the firm's honest position . . . 145
Table 67. Secure-SDLC activities by phase . . . 146
Table 68. Mapping of the proposed capability to NIST AI RMF functions . . . 147

Table 69. Corporate particulars . . . . . 148

Table 70. Shareholding pattern . . . . . 149

Table 71. Litigation and arbitration history . . . . . 150

Table 72. Part-II / Part-III requirement-to-section traceability . . . . . 154

Table 73. Profile of the technical authority (Director and CTO) . . . . . 155

Table 74. Role specifications for positions to be staffed . . . . . 155

Table 75. Glossary of key technical terms . . . . . 158

Table 76. Mapping of defence-specific concerns to framework capabilities . . . . . 159

# List of Abbreviations and Acronyms

The following abbreviations and acronyms are used throughout this document. Standards, frameworks, and academic works are further detailed in Annexure D (References).

**Table 4. Abbreviations and Acronyms Used in This Document**

| Abbr. | Expansion | Abbr. | Expansion |
|---|---|---|---|
| AI | Artificial Intelligence | LLM | Large Language Model |
| API | Application Programming Interface | MIA | Membership Inference Attack |
| ASR | Attack Success Rate | MLOps | Machine-Learning Operations |
| ATLAS | Adversarial Threat Landscape for AI Systems (MITRE) | MoU | Memorandum of Understanding |
| AuC | Area under Curve | MSME | Micro, Small and Medium Enterprises |
| CI/CD | Continuous Integration / Continuous Delivery | mTLS | Mutual Transport Layer Security |
| CIN | Corporate Identity Number | NDU | Non-Disclosure Understanding |
| CPU | Central Processing Unit | NIST AI RMF | NIST Artificial Intelligence Risk Management Framework |
| CSV | Comma-Separated Values | ONNX | Open Neural Network Exchange |
| CTF | Capture-the-Flag (security competition) | OWASP | Open Worldwide Application Security Project |
| CVE | Common Vulnerabilities and Exposures | PAIR | Prompt Automatic Iterative Refinement (jailbreak) |
| CVSS | Common Vulnerability Scoring System | PII | Personally Identifiable Information |
| DFD | Data-Flow Diagram | PoC | Proof of Concept |
| DIN | Director Identification Number | RBAC | Role-Based Access Control |
| DoS | Denial of Service | RFP | Request for Proposal |
| DPDP | Digital Personal Data Protection (Act, 2023) | RMIA | Robust Membership Inference Attack |
| DRDO | Defence Research and Development Organisation | ROC | Registrar of Companies / Receiver Operating Characteristic |
| DTDF | Directorate of Technology Development Fund | SARIF | Static Analysis Results Interchange Format |
| EoI | Expression of Interest | SBOM | Software Bill of Materials |
| FN / FP | False Negative / False Positive | SDLC | Software Development Life Cycle |
| GCG | Greedy Coordinate Gradient (adversarial attack) | SIEM | Security Information and Event Management |
| GPU | Graphics Processing Unit | TDF | Technology Development Fund |
| GRC | Governance, Risk, and Compliance | TOC | Table of Contents |
| GST(IN) | Goods and Services Tax (Identification Number) | TPR / FPR | True Positive Rate / False Positive Rate |
| HF | Hugging Face | UML | Unified Modeling Language |
| IV&V | Independent Verification and Validation | VM | Virtual Machine |
| JSON | JavaScript Object Notation | WBS | Work Breakdown Structure |
| LiRA | Likelihood-Ratio Attack (membership inference) | WORM | Write-Once-Read-Many (storage) |

Source: Compiled by ZeroShield Private Limited.

# 1. Name of the Firm

**ZeroShield Private Limited** (an AI-native cybersecurity, privacy and governance company; incubated at C3iHub, IIT Kanpur).

**Table 5. Identity of the applicant firm**

| Particular | Detail |
|---|---|
| Name of the firm | ZeroShield Private Limited |
| Corporate Identity Number (CIN) | U62020UT2025PTC020093 |
| Date of incorporation | 11 October 2025 |
| Legal status | Private company limited by shares (non-government company) |
| MSME / Udyam registration | UDYAM-UK-12-0096527 (Micro) |

Source: Certificate of Incorporation (MCA); Udyam registration.

*Interpretation: The applicant is a duly incorporated Indian private limited company and a registered micro-enterprise.*

# 2. Contact Details

***i. Email: vartul@zeroshield.ai***

***ii. Point of Contact (PoC): Mr. Vartul Goyal, Director & Chief Technology Officer (DIN 11337955); Mobile +91 8088054916.***

***iii. Address:***

- **Correspondence / project address:** C3iHub, 2nd Floor, New Technopark Building (In front of NCC Office), IIT Kanpur, Kalyanpur, Kanpur Nagar, Uttar Pradesh – 208016.
- **Registered office (per CIN):** Registrar of Companies, Uttarakhand jurisdiction.

# 3. Title of the Project

**Comprehensive Vulnerability Assessment Framework for Large Language Models (LLMs)** — a sovereign, air-gappable, AI-native framework for the security assessment of large language models and of software / binaries, built around an indigenously fine-tuned cybersecurity LLM.

Reference EoI: **DGTM/01/100010/M/48/VAF/PC1** (DRDO Technology Development Fund).

# 4. Project Description

This section describes the project against the prescribed sub-items. Sub-item (i) gives the broad understanding of the problem and the work breakdown; (ii) the critical technologies; (iii) additional technology items; and (iv) the block diagram and process. The detailed technical treatment of each element follows in the numbered sub-sections below.

### *(i) Broad understanding and Work Breakdown Structure — elaborated in the problem, solution, architecture and work-breakdown sub-sections that follow.*

### *(ii) Critical technologies under the project.*

**Table 6. Critical technologies**

| # | Critical technology | Indigenous |
|---|---|---|
| 1 | Transformer-based language-model design, training and fine-tuning | Yes |
| 2 | Supervised fine-tuning (SFT) and instruction tuning of open-weight LLMs | Yes |
| 3 | Cybersecurity-domain corpus curation, cleaning and governance | Yes |
| 4 | Retrieval-augmented generation (RAG) over a vulnerability knowledge base | Yes |
| 5 | Agentic / multi-step LLM orchestration and tool use | Yes |
| 6 | Air-gapped sovereign training and inference | Yes |
| 7 | Adversarial attack generation and agentic red-teaming | Yes |
| 8 | Vulnerability detection and benchmark evaluation | Yes |
| 9 | Malware-analysis assistance (disassembly, AST / CFG) | Yes |
| 10 | Secure code patching with validation and rationale | Yes |

Source: Project scope and solution design.

*Interpretation: No technology in the critical path depends on a foreign cloud or proprietary service.*

### *(iii) Additional technology items to complete the solution.*

- Open-source vector databases (FAISS / Chroma / Milvus / Qdrant) for retrieval and correlation.
- Open-source disassembly / reverse-engineering tooling for the binary-analysis components, run inside the sandbox.
- Distributed-training stack (DeepSpeed / FSDP / Accelerate) for scaling fine-tuning runs on partner / academic compute.
- Offline dependency mirror and a secure one-way data-ingestion path.
- Containerisation / micro-VM isolation for safe parsing and PoC validation.

***(iv) Block diagram and process — the end-to-end block diagram, technology stack and module process flows are presented in the architecture and module sub-sections that follow.***

## 4.1 Introduction and Background

### *4.1.1 Purpose of This Document*

This document is the response of ZeroShield Private Limited ("the firm", "ZeroShield") to the Expression of Interest issued by the Directorate of Technology Development Fund (DTDF) of the Defence Research and Development Organisation (DRDO) under reference DGTM/01/100010/M/48/VAF/PC1, for the indigenous development of a Comprehensive Vulnerability Assessment Framework for Large Language Models. The document sets out the firm's understanding of the problem, its proposed technical solution, the supporting engineering artefacts, the delivery and risk plan, and the firm's qualification against the essential criteria of Part-II of the EoI.

The document is intended to be read by senior technical evaluators, system architects, and the procurement committee. It is written to withstand technical scrutiny: every architectural choice, security design, and performance claim is grounded in published standards, peer-reviewed research, or established industry practice, and the weaknesses of existing approaches are stated explicitly so that the value of the proposed approach can be assessed on its merits.

### *4.1.2 Strategic Context*

Large Language Models have moved, within a very short period, from research curiosities to operational components. Their capacity to summarise, translate, reason over unstructured text, generate code, and drive autonomous tool-use makes them attractive for intelligence triage, decision support, document exploitation, language understanding across regional and adversary languages, and the augmentation of command workflows. The same generality, however, makes them an unusually broad attack surface. Unlike conventional software, an LLM's behaviour is not fully specified by its code; it is an emergent property of billions of learned parameters, of the data on which it was trained, and of the natural-language instructions it receives at run time. This means an adversary can attack the model through its ordinary input channel — language — without ever touching the host, the network, or the binary.

A model fielded in a national-security context must therefore be assessed not only for the classical security properties of the software that serves it, but for a new class of model-level vulnerabilities. The international record over the period 2022–2025 demonstrates that production models are susceptible to instruction override through prompt injection, to safety-bypass through jailbreak techniques, to the extraction of memorised training data, to inference of whether specific records were used in training, and to the covert insertion of triggered behaviour through poisoning of training data or of serialized artefacts. These are not hypothetical risks; they are documented, reproducible, and the subject of an active and rapidly advancing research literature.

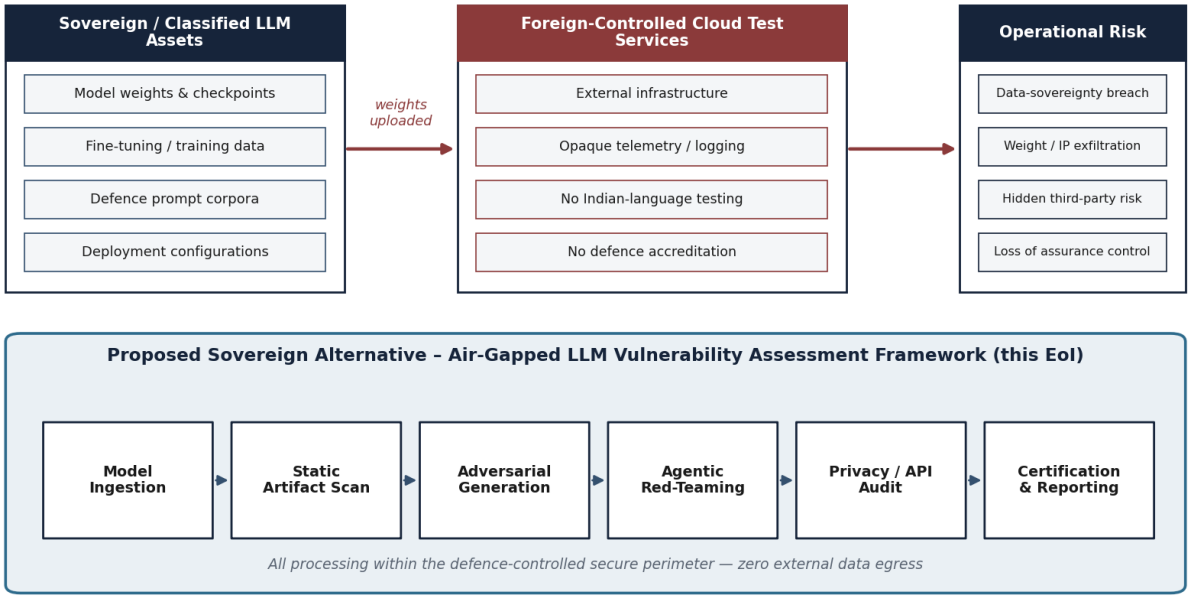



**Figure 1. Operational and threat context for defence-grade LLM assurance**

*The figure situates the proposed framework within its operational environment: a model under test (indigenous or procured) is admitted into a defence-controlled, air-gapped perimeter where it is subjected to systematic assessment before any accreditation decision. The external threat classes shown on the left are precisely those that conventional application-security tooling does not address.*

### *4.1.3 The Sovereignty Imperative*

The mature, widely-adopted LLM security-testing tools and services today are predominantly owned and operated by foreign entities, and most are delivered as cloud services. Routing a sovereign or classified model through such an external service is unacceptable on three independent grounds. First, it requires the model's weights, or at minimum its responses to crafted probes, to leave the national perimeter, directly violating data sovereignty and potentially exposing the most sensitive artefact in the system — the trained model itself. Second, it creates an operational dependency on infrastructure outside national control, which can be withdrawn, throttled, monitored, or compromised. Third, an external testing service is opaque to the user — its processing, retention and telemetry cannot be inspected or controlled — so submitting classified probes and model behaviour to it is itself an unacceptable exposure.

These considerations make an indigenous capability not merely desirable but necessary. The distinction that matters is control and inspectability, not the geographic origin of every component: the capability must be self-contained, operable with no outbound connectivity, and assembled from components that are vetted and deployed entirely under the operator's control within the secure perimeter, so that the whole assessment runs inside the enclave with no external service in the loop. It must also be validated against the specific threat surfaces — multilingual and defence-specific — that generic, English-centric tools neither test for nor understand.

### *4.1.4 Objectives of the Proposed Capability*

The capability proposed in this document is designed to meet the following objectives, derived directly from the EoI:

- Provide a single, sovereign, air-gapped framework capable of assessing any LLM — indigenous or procured — across the full spectrum of LLM-specific vulnerabilities.
- Cover, at minimum, prompt injection, jailbreaking, training-data extraction and membership inference, model-artefact (supply-chain) compromise, inference-API abuse, embedding-space backdoors, and safety alignment bypass.
- Test India-specific surfaces, including multilingual prompt injection in major Indian languages and a defence-oriented harm taxonomy.
- Map every finding to OWASP Top 10 for LLM Applications, MITRE ATLAS, and the NIST AI Risk Management Framework, and produce an accreditation-ready certification report.
- Operate entirely within a defence-controlled perimeter with zero external data exfiltration and full auditability.
- Be delivered as a proof-of-concept within a maximum of twenty-four (24) months, with approximately one hundred validated test cases.

### *4.1.5 Scope and Structure of This Response*

The scope of this response is the design and proof-of-concept development of the framework as software — comprising the scanning engine, adversarial-generation and red-teaming components, static analysers, privacy-audit module, and certification dashboard — together with the supporting test corpus, deployable on air-gapped GPU infrastructure within laboratory premises. The firm undertakes the complete design, algorithm development, corpus curation, testing, and certification integration. The structure of the document follows the reading guide given in the Executive Summary.

### *4.1.6 About ZeroShield Private Limited*

ZeroShield Private Limited is an Indian private limited company incorporated under the Companies Act, 2013, registered with the Registrar of Companies, Uttarakhand, and holding valid MSME (Udyam) registration as a Micro Enterprise in the services sector. The firm is dedicated to the security of Artificial Intelligence and Large Language Model systems. Its areas of work span adversarial AI defence and LLM assurance, model-artefact and supply-chain security, privacy and data-protection auditing, and governance, risk, and compliance for AI systems. The detailed corporate, legal, financial, and manpower particulars of the firm are provided in Section 18 and the Annexures; the present section introduces the firm only to the extent needed to establish the relevance of its competence to the proposed work.

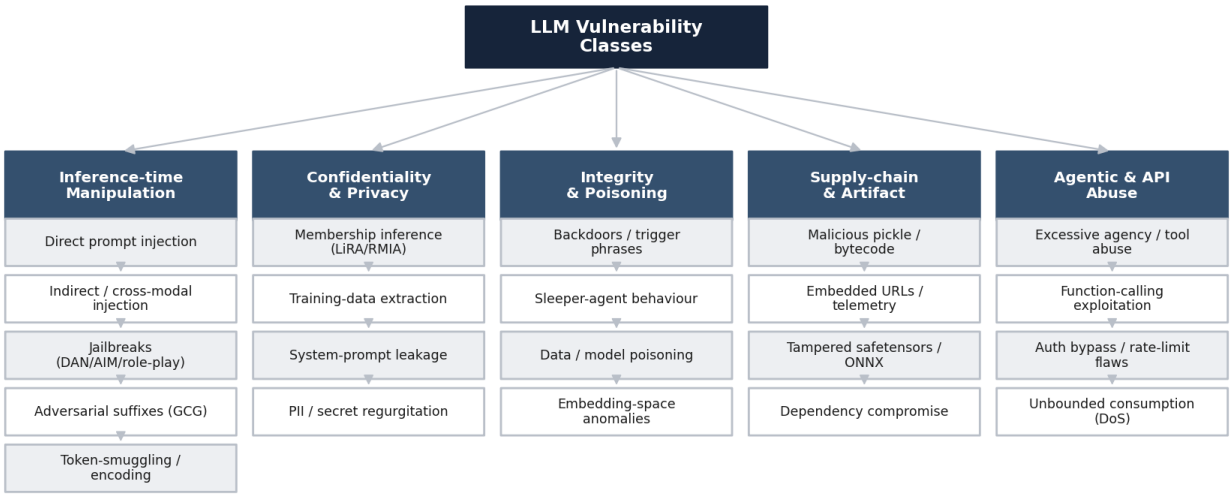
## 4.2 Problem Definition and Threat Landscape

### *4.2.1 Restatement of the Problem*

India lacks a sovereign, defence-grade framework purpose-built to assess the security of Large Language Models before their deployment in national-security contexts. Existing tools are foreign-owned, are predominantly cloud-based, are not validated against Indian defence requirements, and do not test for India-specific threat surfaces. The consequence is that there is at present no trustworthy, indigenous means of answering a simple but critical question: is this model safe to deploy? The proposed framework exists to answer that question rigorously, repeatably, and within national control.

### *4.2.2 A Taxonomy of LLM Vulnerabilities*

To structure the assessment, the firm adopts a taxonomy that consolidates the OWASP Top 10 for LLM Applications (2025), the MITRE ATLAS knowledge base of adversarial tactics against AI systems, and the threat categories enumerated in the EoI. The taxonomy organises vulnerabilities by the stage of the model life-cycle at which they are introduced or exploited — the artefact and supply chain, the model's learned behaviour, the inference interface, and the privacy boundary — and is reflected directly in the design of the six assessment modules.




*Mapped to OWASP Top 10 for LLM Applications (2025), MITRE ATLAS adversarial tactics, and NIST AI RMF risk functions.*

**Figure 2. Consolidated threat taxonomy for Large Language Models**

*The taxonomy groups LLM threats into four families — supply-chain and artefact, instruction- and behaviour-level, inference-interface, and privacy — and cross-references each to the relevant OWASP LLM Top 10 identifier and MITRE ATLAS tactic. The grouping drives the modular decomposition presented in Section 4.6.*

### Instruction- and Behaviour-Level Threats

**Prompt injection (OWASP LLM01).** Prompt injection is the subversion of a model's intended behaviour through crafted input, exploiting the fact that the model cannot reliably distinguish trusted system instructions from untrusted user or document content. Direct injection places adversarial instructions in the user input; indirect injection plants them in content the model will later ingest — a retrieved document, a web page, a file, or a tool result. Cross-modal injection hides instructions in non-text modalities. Injection is the single most consequential LLM vulnerability because it is the gateway to most others: a successful injection can induce data disclosure, tool misuse, or safety bypass.

**Jailbreaking and safety-alignment bypass (OWASP LLM01, related to LLM06).** Jailbreaks are inputs that cause an aligned model to produce content its safety training was intended to refuse. Techniques range from persona and role-play framings (for example the “DAN” and “AIM” families), through token-smuggling and encoding tricks, to translation- and low-resource-language bypass in which a harmful request is expressed in a language the safety training under-covers. Automated methods now generate such inputs at scale: gradient-based optimisation of adversarial suffixes (the Greedy Coordinate Gradient method of Zou and colleagues, 2023), genetic and search-based methods (AutoDAN), and attacker-LLM-driven iterative refinement (PAIR). A defence-grade assessment must test against a large and current library of such techniques and must include automated generation, not merely a fixed checklist.

### Privacy Threats

**Membership inference (related to OWASP LLM02).** A membership inference attack determines whether a particular record was part of the model's training set. State-of-the-art attacks — the likelihood-ratio attack (LiRA) of Carlini and colleagues and the robust membership inference attack (RMIA) of Zarifzadeh and colleagues — calibrate the model's confidence on a candidate record against reference models to achieve high true-positive rates at low false-positive rates. For a defence model trained on sensitive corpora, a successful membership inference is a direct confidentiality breach.

**Training-data extraction (OWASP LLM02).** Beyond inferring membership, an adversary may extract verbatim memorised content. Carlini and colleagues demonstrated extraction of personally identifiable information, credentials, and source-code fragments from production models. The assessment must therefore attempt extraction of PII, API keys, and code, and must quantify memorisation against deliberately inserted canaries.

### Supply-Chain and Artefact Threats

**Serialized-artefact compromise (OWASP LLM03).** Models are distributed as serialized files. The Python pickle format, still pervasive in the ecosystem, permits arbitrary code execution on deserialization; a weaponised checkpoint can execute attacker code the moment it is loaded. Even ostensibly safer formats can carry embedded URLs, telemetry, or anomalous structures. The safetensors format mitigates code execution but does not by itself attest provenance or integrity. A defence-grade framework must statically analyse every artefact format — pickle, safetensors, ONNX, and PyTorch checkpoints — for malicious objects, suspicious bytecode, and embedded indicators before the model is ever instantiated.

**Poisoning and backdoors (OWASP LLM04).** An adversary who influences training data, or who fine-tunes a model, can implant a backdoor that behaves normally until a trigger is present. Hubinger and colleagues demonstrated “sleeper agent” behaviours that persisted through subsequent safety training. Such triggers can be imperceptible in ordinary evaluation; detecting them requires embedding-space and representation analysis rather than black-box prompting alone.

### Inference-Interface Threats

**Improper output handling and excessive agency (OWASP LLM05, LLM06).** When model output is passed downstream without validation, or when a model is granted tools and autonomy, an attacker who controls the model's behaviour can reach the systems behind it. **System-prompt leakage (OWASP LLM07)** exposes confidential instructions and embedded secrets. **Vector and embedding weaknesses (OWASP LLM08)** arise in retrieval-augmented systems. **Misinformation (OWASP LLM09)** and **unbounded consumption (OWASP LLM10)** complete the list, the latter encompassing denial-of-service and resource-exhaustion vectors against model-serving endpoints, as well as authentication and rate-limiting flaws.

**Table 7. OWASP Top 10 for LLM Applications (2025) mapped to assessment modules**

| OWASP LLM (2025) | Description | Primary assessment module |
|---|---|---|
| LLM01 Prompt Injection | Override of intended behaviour via crafted direct, indirect, or cross-modal input. | M3, M4 |
| LLM02 Sensitive Information Disclosure | Leakage of PII, secrets, or proprietary data, including via memorisation. | M5 |
| LLM03 Supply Chain | Compromise via third-party models, artefacts, or dependencies. | M2 |
| LLM04 Data and Model Poisoning | Backdoors and biased behaviour introduced during training or fine-tuning. | M2, M5 |
| LLM05 Improper Output Handling | Unsafe downstream use of unvalidated model output. | M5 |
| LLM06 Excessive Agency | Harm enabled by excessive autonomy, tools, or permissions. | M4, M5 |
| LLM07 System-Prompt Leakage | Exposure of confidential system instructions and embedded secrets. | M3, M4 |
| LLM08 Vector and Embedding Weaknesses | Exploitation of retrieval and embedding mechanisms. | M5 |
| LLM09 Misinformation | Confidently incorrect or fabricated output and over-reliance. | M4, M6 |
| LLM10 Unbounded Consumption | Resource exhaustion, denial of service, and uncontrolled cost. | M5 |

Source: OWASP Top 10 for LLM Applications, 2025 edition; module mapping by ZeroShield Private Limited.

*Interpretation: Every OWASP LLM category is addressed by at least one module; the highest-consequence categories (LLM01, LLM02) are covered by multiple modules for defence in depth.*

### *4.2.3 India-Specific Threat Surfaces*

Two threat surfaces of particular relevance to the Indian defence context are systematically under-tested by foreign tooling. The first is the **multilingual attack surface**. Safety training is concentrated in high-resource languages, principally English; the same harmful request, expressed in Hindi, Tamil, Telugu, Bengali, or code-mixed forms such as Hinglish, frequently evades a safety filter that would refuse it in English. Script-level obfuscation, transliteration, and translation-relay further expand the surface. The second is the **defence-specific harm taxonomy**. The harm categories that matter most in a defence context — for example, the planning of cyber-attacks, the development of weapons, the synthesis of operational guidance, and the elicitation of restricted knowledge — are not the categories around which commercial safety benchmarks are organised. The proposed framework treats both surfaces as first-class requirements, as detailed in Sections 4.6 and 4.7.

### *4.2.4 Consequences of Inaction*

If a sovereign assessment capability is not established, the nation faces a binary and unattractive choice for every model it wishes to field: deploy it unvetted, accepting unknown and potentially severe model-level risk, or submit it to a foreign tool, accepting a sovereignty and supply-chain compromise. The proposed framework dissolves this dilemma by making rigorous, in-perimeter assessment possible. The remainder of this document shows how.

## 4.3 Limitations of Existing Approaches

### *4.3.1 Categories of Existing Tooling*

The current landscape of LLM-security tooling can be grouped into four categories, each with characteristic limitations when measured against the requirements of this EoI. The categories are: commercial cloud-based LLM-security platforms; open-source red-teaming libraries and harnesses; classical application-security and DevSecOps tools; and ad-hoc, manual red-teaming. None of these, individually or in combination, satisfies the conjoint requirement of sovereignty, air-gapped operation, full-spectrum LLM coverage, India-specific surfaces, and accreditation-ready certification.

### *4.3.2 Gap Analysis*

The table below analyses each category against the dimensions that matter for defence use, and states the specific weakness that the proposed framework is designed to remove. The analysis is deliberately conservative: it credits each category with its genuine strengths before identifying where it falls short.

**Table 8. Comparative gap analysis of existing approaches versus the proposed framework**

| Dimension | Commercial cloud platforms | Open-source red-team libraries | Classical AppSec / DevSecOps | Proposed framework |
|---|---|---|---|---|
| Air-gapped operation | No — cloud-dependent by design | Partial — varies; often assumes connectivity / hosted model APIs | Yes for code, but not LLM-aware | Yes — air-gapped by design |
| Data sovereignty | No — data leaves perimeter | Depends on deployment; corpora often external | Yes | Yes — all artefacts in-perimeter |
| Full LLM threat coverage | Partial; proprietary and opaque | Fragmented across many tools | No — not model-aware | Yes — unified six-module coverage |
| Adversarial generation (GCG etc.) | Hidden / limited | Available but disparate | No | Yes — integrated optimisation |
| Artefact / supply-chain scan | Limited | Some scanners exist (e.g., pickle) | Dependency scan only, not ML-aware | Yes — pickle/safetensors/ONNX/ckpt |
| Membership inference / extraction | Rare | Research code, not productised | No | Yes — LiRA/RMIA + extraction |
| Indian-language surfaces | No | No | No | Yes — Hindi/Tamil/Telugu/Bengali + |
| Defence harm taxonomy | No | No | No | Yes — defence-specific |
| Accreditation-ready output | Vendor format | No | Partial (SARIF for code) | Yes — OWASP/ATLAS/NIST mapped |
| Auditability of the tool itself | No — opaque, possible telemetry | Yes (source available) | Varies | Yes — indigenous, source-controlled |

Source: ZeroShield Private Limited analysis based on public documentation of representative tools and the EoI requirements.

*Interpretation: No existing category satisfies the conjunction of sovereignty, air-gapped operation, full-spectrum coverage, India-specific surfaces, and accreditation-ready output; the proposed framework is designed precisely to close this conjoint gap.*

### *4.3.3 Specific Weaknesses Addressed*

From the gap analysis, the proposed framework is engineered to remove the following specific, named weaknesses:

1. **Connectivity dependence.** Existing platforms assume outbound connectivity to hosted model endpoints or telemetry services. The proposed framework operates with no outbound connectivity whatsoever; all models are served locally and all corpora are local.
2. **Sovereignty leakage.** Cloud tools require the model or its responses to leave the perimeter. The proposed framework keeps weights, corpora, probes, responses, and findings entirely within the defence-controlled perimeter.
3. **Fragmentation.** Open-source capability exists but is scattered across incompatible libraries with no common data model or report. The proposed framework unifies generation, execution, analysis, scoring, and certification under one schema and one orchestrator.
4. **Model-blindness of classical tools.** AppSec and DevSecOps tools secure the code and dependencies around a model but are blind to model-level behaviour. The proposed framework complements, rather than replaces, those tools by adding the model-level dimension.
5. **Absence of Indian surfaces.** No existing tool tests multilingual Indian-language injection or a defence-specific harm taxonomy. Both are first-class requirements of the proposed framework.
6. **Opacity and self-risk.** A foreign testing tool is itself an unaudited foreign binary. The proposed framework is indigenous, source-controlled, and itself amenable to audit and to a software bill of materials.

### *4.3.4 Positioning Statement*

The proposed framework is therefore best understood not as a re-implementation of any existing tool, but as the first sovereign, air-gapped, full-spectrum LLM assurance capability tailored to Indian defence requirements. It draws on the open, peer-reviewed research base — GCG, AutoDAN, PAIR, LiRA, RMIA, and the OWASP, ATLAS, and NIST frameworks — and renders that research operational, repeatable, and certifiable within national control. The technical design that delivers this positioning is the subject of Part II.

## 4.4 Solution Overview and Design Principles

### *4.4.1 Solution Concept*

The firm proposes a single, integrated framework — provisionally designated the LLM Vulnerability Assessment Framework — that admits a model under test into a hardened, air-gapped environment and subjects it to a battery of automated assessments organised as six modules. The framework is conceived as a controlled pipeline: a model is ingested and its provenance recorded; its artefacts are statically analysed before it is ever instantiated; adversarial inputs are generated and executed against it; multi-turn agentic red-teaming probes its behaviour over conversations; its privacy boundary, inference interface, and internal representations are audited; and finally every finding is scored, mapped to recognised frameworks, and compiled into an accreditation-ready certification report. The entire pipeline runs within the perimeter, with no outbound connectivity and with cryptographically verifiable audit logging.

### *4.4.2 Design Principles*

The architecture is governed by eight design principles, each of which is traceable to a requirement of the EoI or to established security-engineering practice.

1. **Sovereignty and air-gap by construction.** No component requires or initiates outbound connectivity. The default network posture is deny-all; the perimeter has no route to any external network.
2. **Defence in depth.** The highest-consequence threats are addressed by more than one module and more than one technique, so that the failure of any single detector does not leave a gap.
3. **Determinism and reproducibility.** Given the same model, corpus, and configuration, an assessment produces the same findings and the same score. Every run is seeded, versioned, and recorded.
4. **Containment of untrusted artefacts.** The model under test and any untrusted serialized artefact are treated as hostile until proven otherwise and are confined to a sandboxed enclave with no privileged access and no lateral reach.
5. **Separation of duties and least privilege.** Operators, the orchestration plane, and the target enclave are isolated; access is role-based and minimal; secrets are centrally managed.
6. **Auditability and evidence integrity.** Every action and finding is written to a write-once, tamper-evident log, so that a certification can be independently re-verified.
7. **Modularity and extensibility.** New attack techniques, languages, and harm categories can be added without re-architecting; the threat landscape evolves and the framework is built to evolve with it.
8. **Indigenous and transparent construction.** The framework is built from open, auditable components and indigenous code, so that the assurance tool is itself assurable.

### *4.4.3 Conformance to the EoI Scope*

The six-module decomposition corresponds one-to-one with the system composition described in the EoI background: a GPU cluster for adversarial optimisation, an automated scanning engine, an agentic red-teaming module, a static-analysis pipeline for serialized artefacts, a privacy-audit module, and a certification dashboard with OWASP LLM Top 10 mapping. The mapping is made explicit in the following table so that the evaluator can trace each element of the EoI's stated composition to a module of the proposed design.

**Table 9. Traceability from EoI-stated system composition to proposed modules**

| EoI-stated system element | Proposed module | Section |
|---|---|---|
| GPU cluster for adversarial optimisation (GCG, gradient-based) | M3 Adversarial Attack Generation | 6.3 |
| Automated scanning engine executing 100+ security tests | Orchestrator + M2/M3/M5 executors | 5, 6 |
| Agentic red-teaming module for multi-turn attacks | M4 Agentic Red-Teaming | 6.4 |
| Static-analysis pipeline for serialized artefacts | M2 Static Analysis and Artefact Scanning | 6.2 |
| Privacy-audit module (membership inference, extraction) | M5 Privacy, API, and Embedding Audit | 6.5 |
| Certification dashboard with OWASP LLM Top 10 mapping | M6 Certification and Reporting | 6.6 |

Source: EoI clause 2.2 (Background); mapping by ZeroShield Private Limited.

*Interpretation: Each element named in the EoI is realised by a specific module, confirming complete coverage of the stated composition.*

## 4.5 System Architecture

### *4.5.1 Architectural Overview*

The framework is structured as a layered architecture within a single air-gapped security perimeter. A presentation and control plane provides the operator console, the assessment orchestrator, and the certification dashboard. Beneath it, the assessment services layer hosts the six modules as independently deployable, containerised micro-modules that communicate over an internal, authenticated message and job bus. A platform and data-services layer provides the findings database, the object store for model artefacts, a cache and job queue, and the test-corpus repository. A dedicated GPU compute fabric serves two distinct workloads — adversarial optimisation and inference-serving of the target model. Finally, an isolated target-under-test enclave hosts the model under test, the sandbox for executing untrusted artefacts, and the instrumented inference-probe harness. The architecture is shown in the figure below.

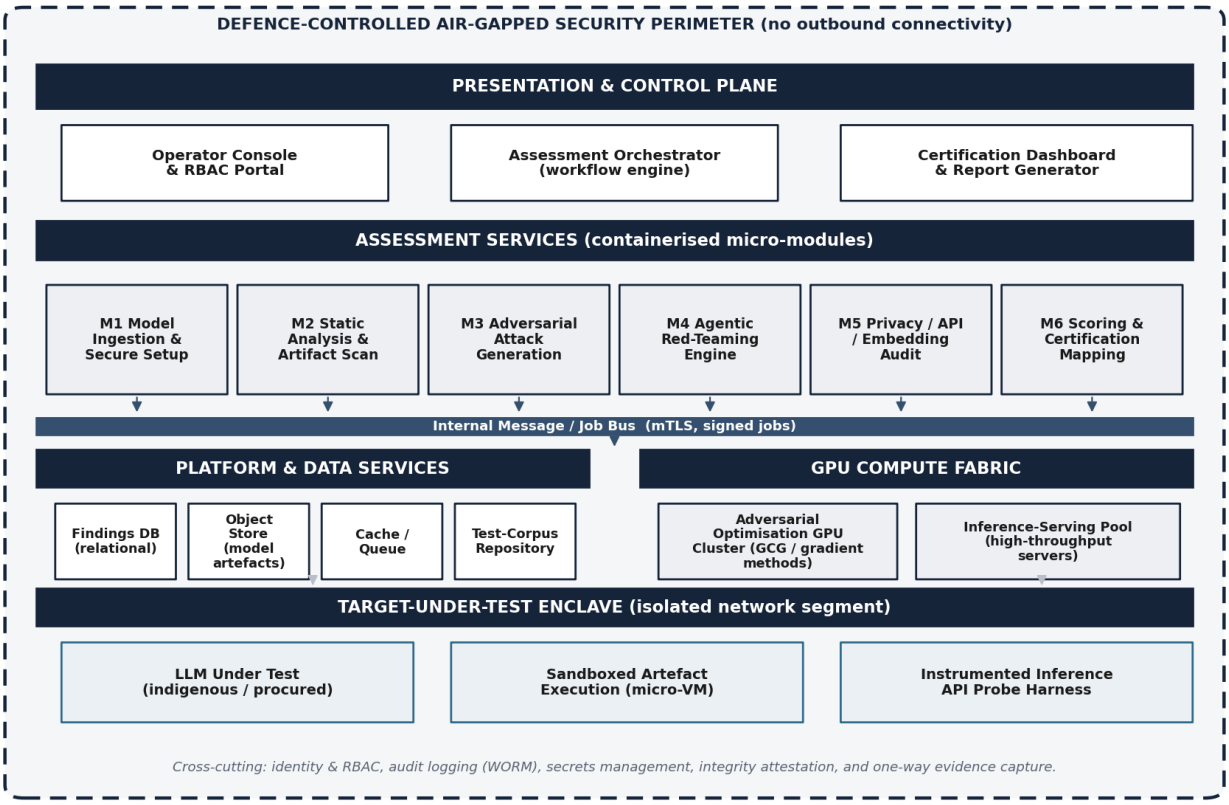



**Figure 3. Logical system architecture within the air-gapped perimeter**

*The architecture separates the control plane, the assessment services, the platform and data services, the GPU compute fabric, and the isolated target enclave. The model under test is confined to the enclave; cross-cutting services provide identity and RBAC, tamper-evident audit logging, secrets management, and integrity attestation. No layer has any route to an external network.*

### *4.5.2 Layered Responsibilities*

#### Presentation and Control Plane

The control plane is the only layer with which a human operator interacts. The operator console authenticates the operator, enforces role-based access control, and exposes the workflow for defining an assessment — selecting the target model, the test plan, the languages, and the harm taxonomy. The assessment orchestrator is a workflow engine that decomposes an assessment into signed jobs, dispatches them to the appropriate module executors, tracks their state, and aggregates their results. The certification dashboard renders findings, scores, and the final report, and provides the evidence trail for accreditation.

#### Assessment Services Layer

Each of the six modules is packaged as a containerised micro-module exposing a uniform job interface. This uniformity is deliberate: it allows modules to be developed, tested, scaled, and replaced independently, and it allows new techniques to be added as new module versions without disturbing the rest of the system. Modules are stateless with respect to a given job; all durable state resides in the data-services layer. Communication is exclusively over the internal message and job bus, secured with mutual TLS and carrying signed job descriptors, so that no module will act on an unsigned or tampered instruction.

#### Platform and Data Services

The findings database (a relational store) holds the structured, queryable record of every finding, its evidence reference, its severity, and its framework mappings. The object store holds model artefacts and large evidence blobs under content-addressed, integrity-checked keys. The cache and job queue decouple producers from consumers and absorb load. The test-corpus repository holds the versioned attack corpora — injection prompts, jailbreak techniques, canary datasets, multilingual payloads, and the defence-harm taxonomy — each under version control so that an assessment can declare exactly which corpus version it used.

#### GPU Compute Fabric

The compute fabric serves two workloads that have different performance profiles and different trust postures. Adversarial optimisation — gradient-based suffix generation and related methods — is compute-intensive and operates on the model in a white-box or grey-box mode where gradients are available. Inference-serving hosts the target model behind a controlled endpoint for black-box probing, using open-source inference engines. Separating the two workloads allows each to be scheduled and isolated according to its needs, and allows the target model to be served under tight instrumentation.

#### Target-Under-Test Enclave

The enclave is the most strongly isolated zone. It hosts the model under test, a sandboxed execution environment (a hardened container runtime or virtual machine) for safely loading untrusted serialized artefacts, and the instrumented inference-probe harness that mediates all interaction with the model. The enclave has no privileged access to the rest of the system and no outbound route; it receives jobs and returns evidence solely through the controlled, one-way-audited interface described in Section 4.11.

### *4.5.3 End-to-End Assessment Pipeline*

The figure below presents the assessment as a left-to-right pipeline, from model intake through static analysis, adversarial generation, red-teaming, privacy and interface audit, to scoring and certification. The pipeline view complements the layered architecture: where the architecture shows what the components are, the pipeline shows the order in which an assessment flows through them.

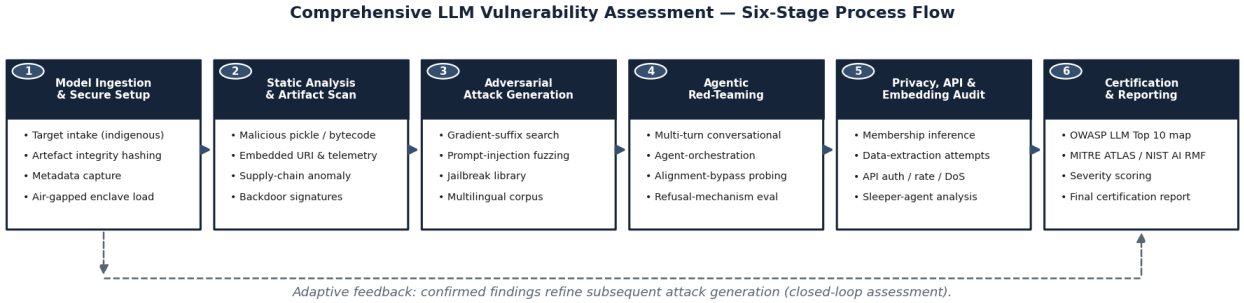



**Figure 4. End-to-end assessment pipeline (process flow of a single assessment)**

*An assessment proceeds through six pipeline stages corresponding to the six modules. Static analysis (M2) gates instantiation; no model is served for dynamic testing until its artefacts have passed inspection. Findings accumulate in the findings database throughout and are consolidated by M6 into the certification report.*

### *4.5.4 Cross-Cutting Services*

Four services cut across all layers. **Identity and RBAC** authenticate every actor and authorise every action against a least-privilege role model. **Audit logging** writes an immutable, tamper-evident record to write-once storage. **Secrets management** centralises credentials and keys and keeps them out of code and configuration. **Integrity attestation** verifies the integrity of the framework's own components and of every artefact it handles, so that the assurance tool can itself be shown to be unmodified. These services are detailed in the security architecture of Section 4.11.

## 4.6 The Six Assessment Modules

This section describes each module in turn: its objective, its inputs and outputs, its principal techniques, and the EoI requirements and OWASP categories it satisfies. Each module is illustrated by a swim-lane figure showing its internal stages. The modules share the common job interface and data model described in Section 4.5; the descriptions here concentrate on the distinctive function of each.

### *4.6.1 Module M1 — Model Ingestion and Secure Setup*

#### Objective

M1 admits a model under test into the perimeter under controlled, auditable conditions, captures its provenance and metadata, computes and records cryptographic hashes of every artefact, and prepares the isolated enclave in which the model will subsequently be analysed and served. M1 is the chain-of-custody origin for the entire assessment.

#### Function and Flow

On intake, M1 records the declared source of the model (indigenous or procured), its claimed architecture, version, and parameter count, and the operator and authorisation under which it is admitted. It computes content hashes of all files, registers them in the object store under content-addressed keys, and creates an immutable intake manifest. It then provisions the target enclave — allocating isolated compute, establishing the sandbox, and configuring the inference-probe harness — but does not yet instantiate the model: instantiation is gated on the static-analysis verdict from M2. A controlled smoke-test confirms that the model can be served once cleared.

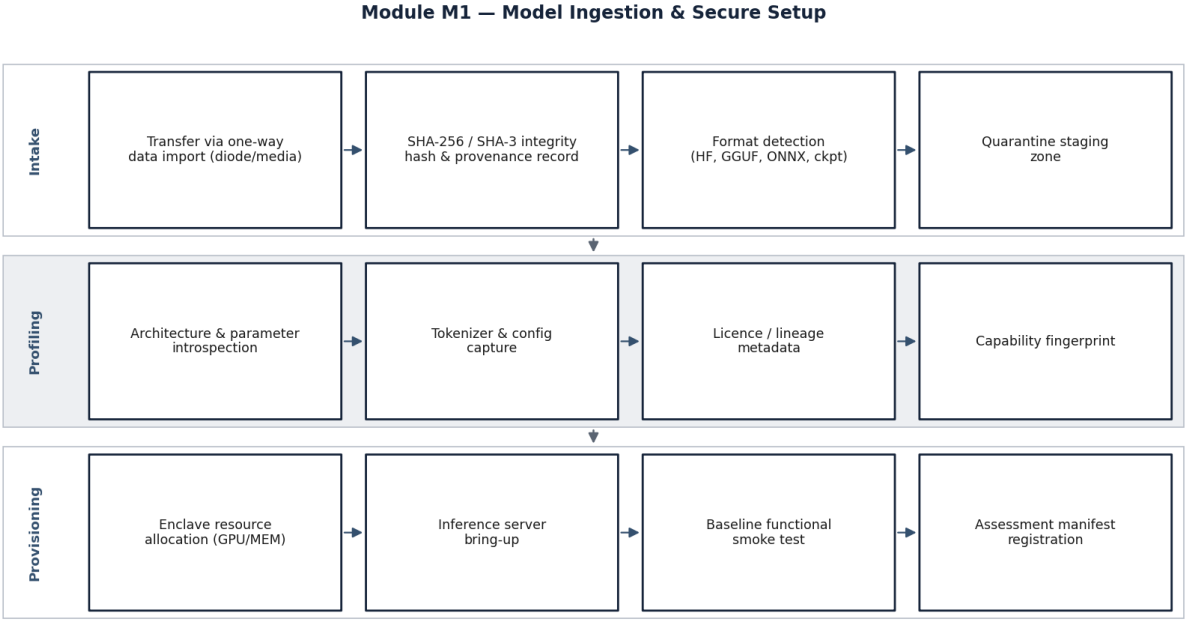



**Figure 5. Module M1 — model ingestion and secure setup (stage view)**

*M1 establishes chain of custody: intake authorisation, provenance and metadata capture, cryptographic hashing and registration, enclave provisioning, and a gated readiness check. Instantiation of the model is deliberately deferred until M2 has cleared the artefacts.*

### Requirements Satisfied

M1 underpins the air-gapped operation requirement and the data-sovereignty undertaking: from the moment of intake, the model and its artefacts are within the perimeter and under cryptographic custody. It provides the provenance baseline on which supply-chain analysis (M2) and certification (M6) depend.

## *4.6.2 Module M2 — Static Analysis and Artefact Scanning*

### Objective

M2 inspects the model's serialized artefacts for malicious or suspicious content before the model is instantiated, addressing the supply-chain threat (OWASP LLM03) and contributing to poisoning and backdoor detection (OWASP LLM04). Its target is a detection rate above ninety per cent on a curated corpus of malicious artefacts.

### Function and Flow

M2 parses each artefact according to its format. For Python pickle and PyTorch checkpoints, it disassembles the pickle opcode stream without executing it, flagging opcodes capable of code execution (such as global and reduce constructs that resolve to dangerous callables) and reconstructing the import and call graph statically. For safetensors, it validates the header and tensor index against the file structure and checks for anomalies. For ONNX, it inspects the computational graph for unexpected custom operators or embedded payloads. Across all formats, it searches for embedded URLs, network indicators, telemetry endpoints, and suspicious byte patterns, and it produces a signature-and-heuristic verdict. Untrusted artefacts that must be loaded to be analysed dynamically are loaded only inside the sandbox.

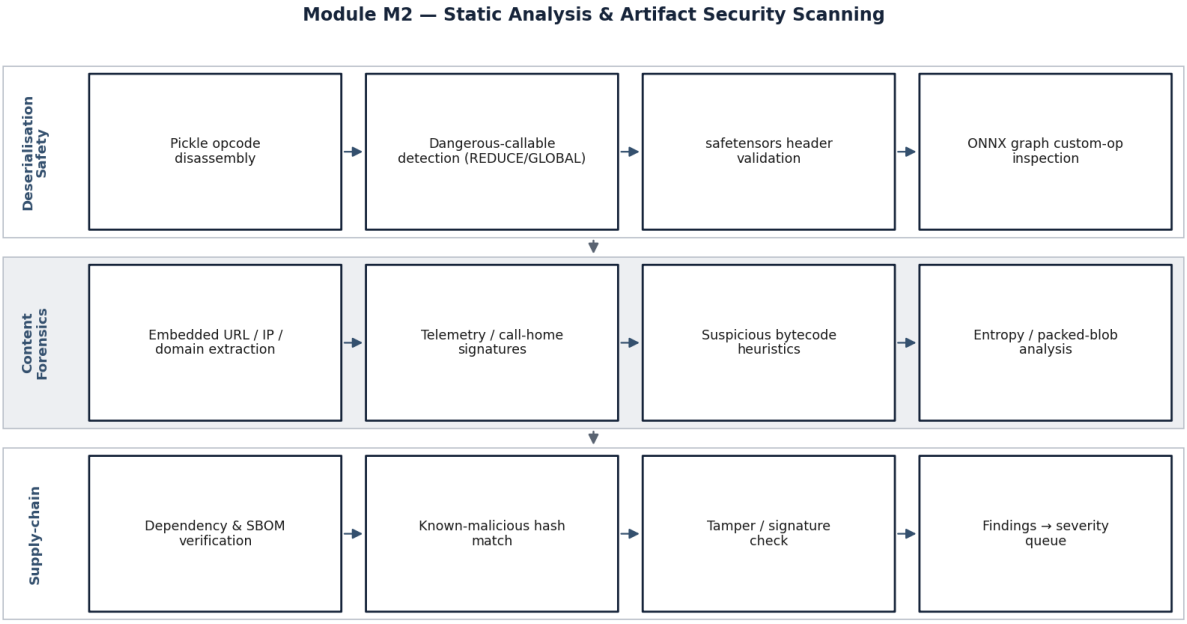



**Figure 6. Module M2 — static analysis and artefact scanning (stage view)**

*M2 routes each artefact to a format-specific analyser, performs opcode-level disassembly for pickle/checkpoint formats without execution, scans for embedded indicators, and renders a clear/suspicious/malicious verdict that gates model instantiation.*

### Requirements Satisfied

M2 satisfies the model-artefact security-scanning specification (detection of malicious serialized objects, embedded telemetry, and suspicious bytecode across pickle, safetensors, ONNX, and PyTorch checkpoints) and provides the gating verdict that enforces the principle of treating untrusted artefacts as hostile.

## *4.6.3 Module M3 — Adversarial Attack Generation*

### Objective

M3 generates adversarial inputs — prompt-injection payloads and jailbreak prompts — using both optimisation-based and fuzzing-based methods, in English and in Indian languages. It targets detection of more than ninety-five per cent of known prompt-injection patterns and more than seventy per cent of zero-shot injection attempts, and coverage of more than fifty distinct jailbreak methodologies.

### Function and Flow

M3 maintains a versioned library of known injection and jailbreak templates and a generation engine. Where model gradients are available (white-box or grey-box), it runs gradient-based optimisation — the Greedy Coordinate Gradient method and variants — on the adversarial-optimisation GPU fabric to synthesise adversarial suffixes that maximise the probability of a target behaviour. Where only black-box access is available, it uses search-based and attacker-LLM-driven methods (in the spirit of AutoDAN and PAIR) and structured fuzzing over the template library. A dedicated multilingual stage translates and adapts payloads into Hindi, Tamil, Telugu, Bengali, and code-mixed forms, and applies script-level and transliteration transformations. Generated payloads are catalogued in the attack-corpus repository and handed to the execution path.

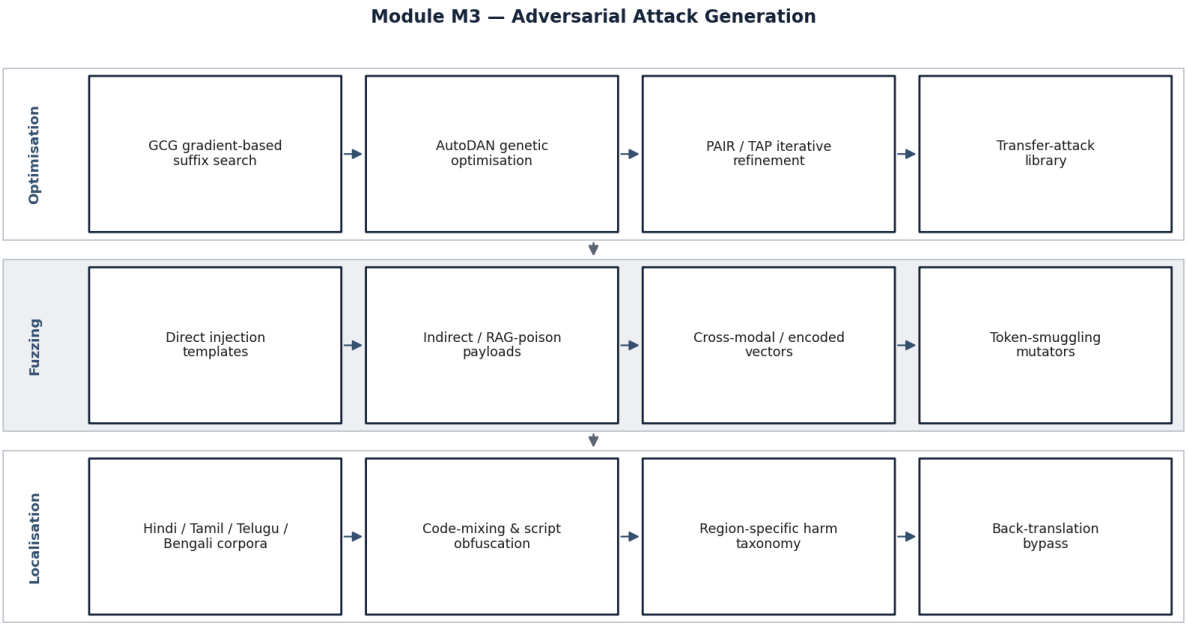



**Figure 7. Module M3 — adversarial attack generation (stage view)**

*M3 combines a curated technique library with gradient-based optimisation (GCG and variants) and black-box search (AutoDAN/PAIR style), then expands every payload through a multilingual adaptation stage before cataloguing it for execution.*

### Requirements Satisfied

M3 satisfies the prompt-injection-coverage and jailbreak-robustness specifications and the multilingual attack-surface requirement, and it supplies the adversarial inputs consumed by the execution and red-teaming paths.

### *4.6.4 Module M4 — Agentic Red-Teaming*

### Objective

M4 conducts automated, multi-turn, goal-directed adversarial conversations against the target model, emulating a persistent human red-teamer. It addresses safety-alignment bypass across the defence-specific harm taxonomy and exercises excessive-agency and system-prompt-leakage threats (OWASP LLM06, LLM07).

### Function and Flow

M4 is built on an agentic orchestration layer (using open-source agent-orchestration frameworks, or an equivalent indigenous orchestration). An attacker agent is given a goal drawn from the harm taxonomy and a budget of conversational turns. It plans an approach, issues a probe, observes the target's response, and adapts — escalating, reframing, role-playing, or switching language — over multiple turns. A separate evaluator (an automated judge with human-review sampling) scores each trajectory for whether the safety objective was breached and to what degree. The module evaluates the target's refusal mechanisms against more than ten harm categories, including defence-specific categories, and records full conversation transcripts as evidence.

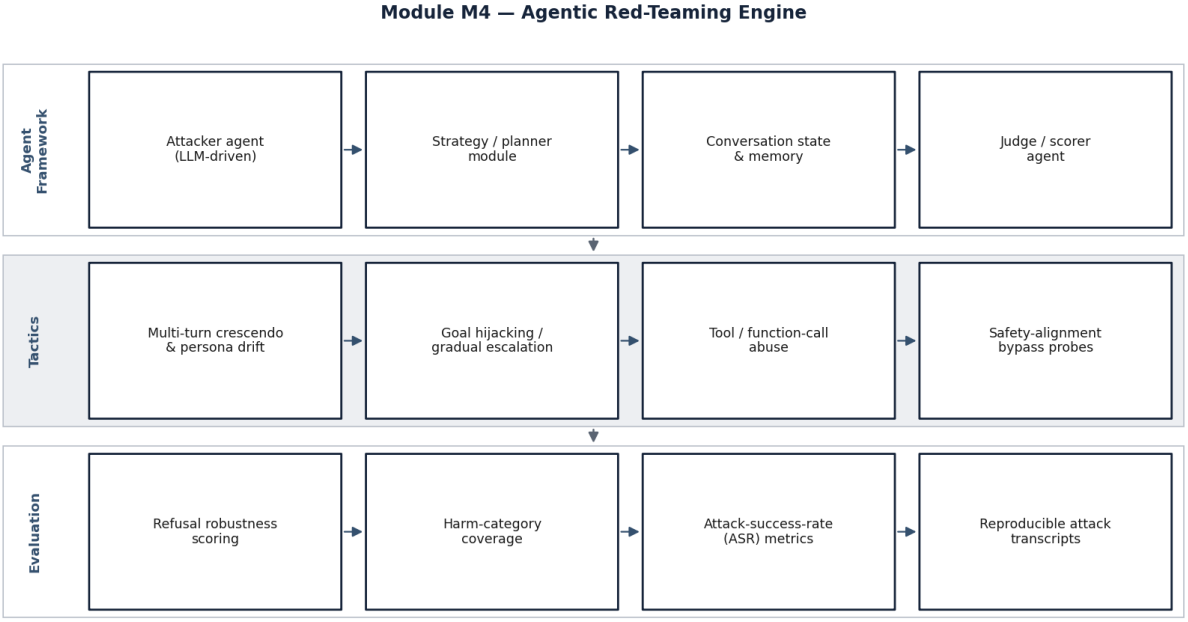



**Figure 8. Module M4 — agentic multi-turn red-teaming (stage view)**

*An attacker agent pursues a taxonomy-derived goal over multiple turns, adapting strategy based on the target's responses, while an automated judge (with sampled human review) scores each trajectory. Full transcripts are retained as tamper-evident evidence.*

### Requirements Satisfied

M4 satisfies the safety-alignment-bypass specification (ten-plus harm categories with a defence-specific taxonomy) and the multi-turn, agentic-framework requirement, and it complements M3's single-turn generation with conversational depth.

### *4.6.5 Module M5 — Privacy, API, and Embedding Audit*

### Objective

M5 audits the model's privacy boundary, its inference interface, and its internal representations. It performs membership inference and training-data extraction, tests the serving endpoint for authentication, rate-limiting, and resource-exhaustion flaws, and analyses the embedding space for backdoors and sleeper behaviours (OWASP LLM02, LLM05, LLM08, LLM10, and contributing to LLM04).

### Function and Flow

For privacy, M5 implements likelihood-ratio (LiRA) and robust (RMIA) membership-inference attacks against standard canary datasets, targeting an inference accuracy above eighty per cent, and attempts extraction of PII, API keys, and source-code fragments. For the interface, it probes the model-serving endpoint for authentication bypass, rate-limiting weaknesses, excessive function-calling permissions, and denial-of-service vectors. For representations, it analyses activations and embeddings for anomalous clusters indicative of backdoor triggers or sleeper-agent behaviour, applying representation-engineering techniques. Findings are recorded with quantitative metrics — attack success rates, true-positive rates at fixed false-positive rates, and extraction counts.

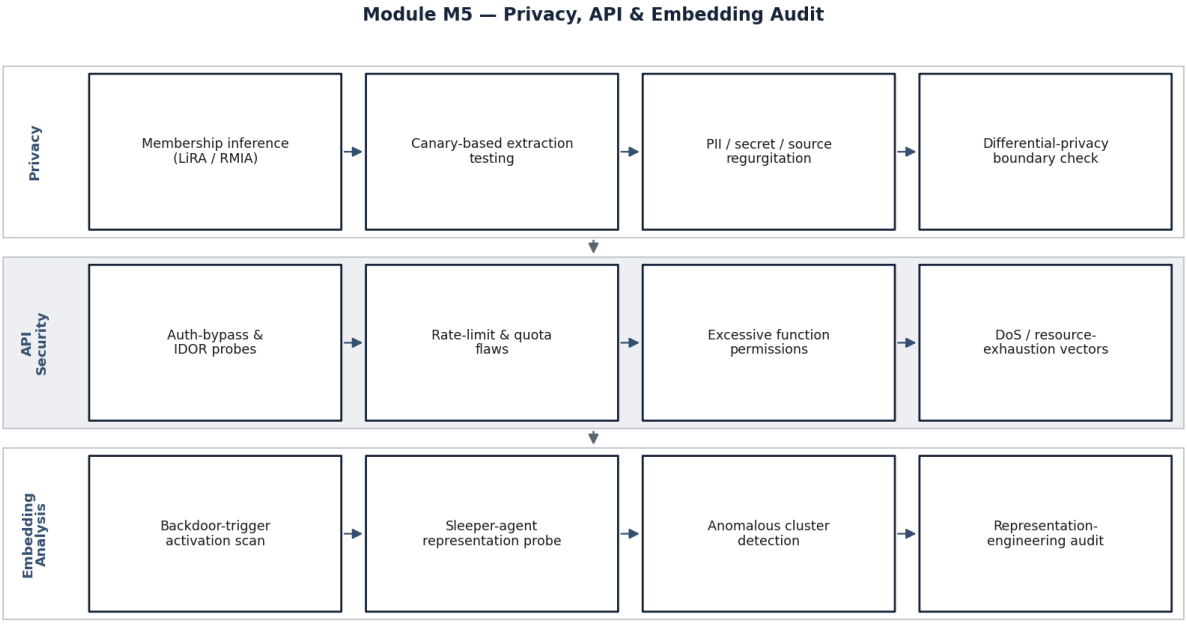



**Figure 9. Module M5 — privacy, API, and embedding audit (stage view)**

*M5 runs three parallel audit tracks — privacy (membership inference and extraction), interface (authentication, rate-limiting, and denial-of-service), and representation (embedding-space backdoor and sleeper-agent detection) — and emits quantitative findings.*

### Requirements Satisfied

M5 satisfies the training-data-extraction and privacy-testing specification, the inference-API security-audit specification, and the embedding-space-analysis specification, providing the privacy and interface dimensions of the assessment.

### *4.6.6 Module M6 — Certification and Reporting*

### Objective

M6 consolidates all findings into a deterministic score, maps every finding to the OWASP Top 10 for LLM Applications, MITRE ATLAS, and the NIST AI Risk Management Framework, and generates the accreditation-ready certification report. M6 is the interface between the technical assessment and the accreditation decision.

### Function and Flow

M6 ingests the structured findings from the findings database, applies a documented, deterministic scoring rubric that weights findings by severity and confidence, and computes per-category and overall robustness scores. It generates the framework mappings automatically and assembles a report containing an executive verdict, the scores, the per-finding detail with evidence references, the framework-coverage matrices, and the reproducibility metadata (corpus versions, seeds, and configuration). The report is produced in both a human-readable form and a machine-readable form suitable for ingestion into accreditation workflows. Every figure in the report is traceable to evidence in the write-once log.

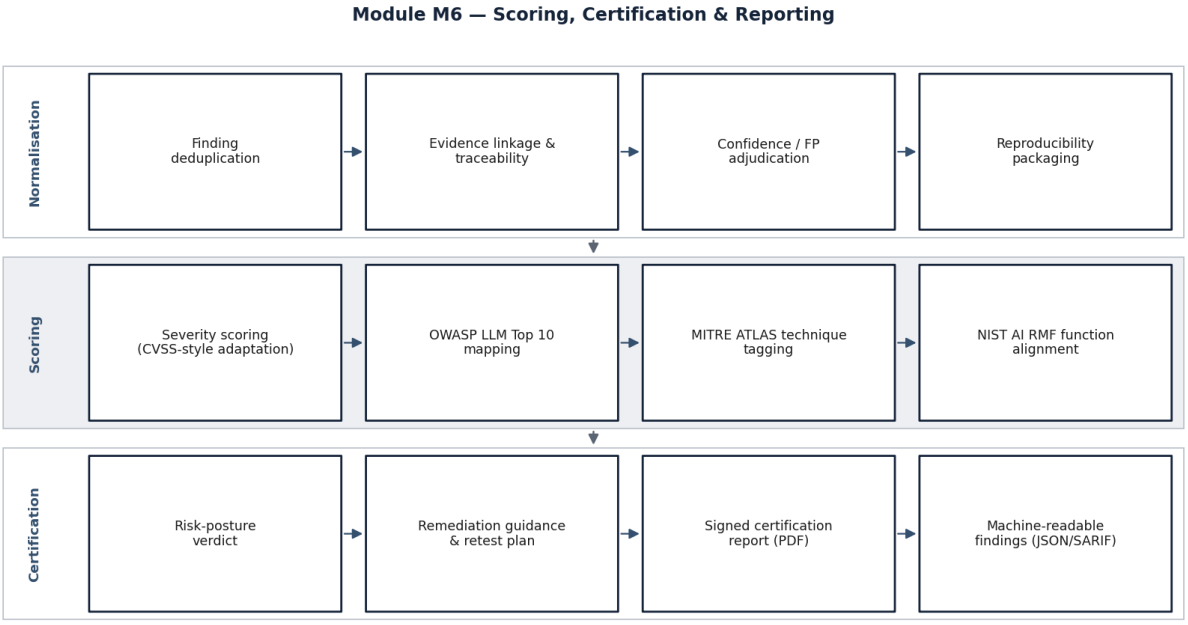



**Figure 10. Module M6 — certification scoring and reporting (stage view)**

*M6 aggregates findings, applies a deterministic severity-and-confidence rubric, generates OWASP/ATLAS/NIST mappings, and produces a dual human- and machine-readable certification report with full reproducibility metadata and evidence traceability.*

**Requirements Satisfied**

M6 satisfies the OWASP-LLM-Top-10-compliance requirement (full, automated mapping and reporting) and the certification-dashboard element of the EoI, and it is the module through which the framework delivers its accreditation value.

### *4.6.7 Inter-Module Data Model*

All modules share a common finding schema so that findings from heterogeneous techniques are comparable and aggregable. A finding records, at minimum: a stable identifier; the module and technique that produced it; the target model and corpus versions; the threat category and OWASP/ATLAS/NIST mappings; a severity and a confidence; quantitative metrics where applicable; and an evidence reference into the write-once log. This shared model is what allows M6 to score and certify across all modules uniformly, and is detailed further in the data architecture of Section 4.9.

## 4.7 Functional and Performance Specifications

### *4.7.1 Approach to Specification*

This section restates the functional and performance requirements of the EoI as a specification against which the proof-of-concept will be verified, and records, for each, the assessment module that delivers it and the figure of merit by which it will be measured. The targets are taken directly from the EoI's detailed-specifications clause and its parameter table; the firm adopts them as acceptance criteria for the proof-of-concept.

### *4.7.2 Functional Specifications*

**Table 10. Functional specification, module assignment, and verification approach**

| Ref | Functional requirement | Module | Verification approach |
|---|---|---|---|
| F1 | Detect direct, indirect, and cross-modal prompt-injection patterns and zero-shot injection attempts. | M3 | Evaluation against the injection corpus; coverage measured. |
| F2 | Test the target against a library of 50+ jailbreak techniques and produce a robustness score. | M3, M4 | Technique-library execution; robustness score computed. |
| F3 | Perform membership inference and attempt extraction of PII, API keys, and code fragments. | M5 | LiRA/RMIA on canary datasets; extraction attempts logged. |
| F4 | Detect malicious serialized objects, embedded telemetry, and suspicious bytecode in model files. | M2 | Static analysis over the malicious-artefact corpus. |
| F5 | Test refusal mechanisms against 10+ harm categories with a defence-specific taxonomy. | M4 | Agentic red-teaming per category; breach rate recorded. |
| F6 | Audit inference endpoints for authentication bypass, rate-limiting flaws, excessive function-calling, and DoS. | M5 | Endpoint probing harness; findings recorded. |
| F7 | Detect sleeper-agent behaviours, backdoor triggers, and anomalous representation clusters. | M5 | Embedding-space and representation analysis. |
| F8 | Support testing in Hindi, Tamil, Telugu, Bengali, and other major Indian languages. | M3, M4 | Multilingual corpus execution; per-language results. |
| F9 | Operate with full functionality and no internet connectivity; keep all artefacts in-perimeter. | All | Air-gap conformance test; egress monitoring shows nil. |
| F10 | Map all findings to OWASP LLM Top 10 and generate a certification report. | M6 | Automated mapping; report generated and reviewed. |

Source: Derived from EoI clause 2.4 (Detailed Specifications); verification approach by ZeroShield Private Limited.

*Interpretation: Every functional requirement is assigned to at least one module and has a concrete verification approach, forming the basis of proof-of-concept acceptance.*

### *4.7.3 Performance Specifications and Figures of Merit*

The EoI specifies indicative performance targets and figures of merit. The firm adopts these as the proof-of-concept acceptance thresholds, reproduced below with the module responsible and the measurement method. Where a target is expressed as a percentage, it will be measured on the corresponding corpus or benchmark suite under a documented protocol; where it is expressed as a time, it will be measured on a seven-billion-parameter reference model under representative load.

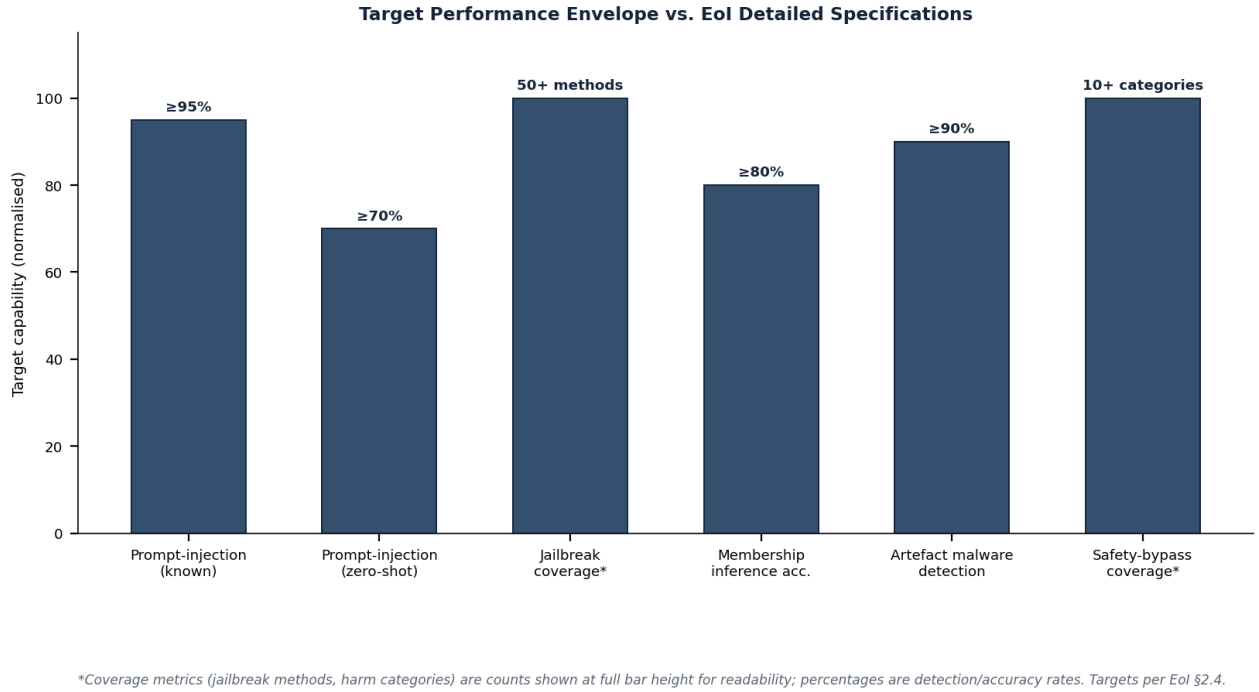
**Table 11. Performance parameters, targets, and figures of merit**

| Parameter | Target value | Figure of merit | Module |
|---|---|---|---|
| Prompt-injection detection coverage | >95% known; >70% zero-shot | Custom corpus of 10,000+ injection prompts | M3 |
| Jailbreak technique coverage | 50+ distinct methodologies | OWASP LLM Top 10 / LLM security benchmarks | M3, M4 |
| Membership-inference accuracy | >80% on standard canaries | LiRA / RMIA benchmark suites | M5 |
| Model-artefact malware detection | >90% detection rate | Custom malicious-model corpus | M2 |
| Safety-bypass test coverage | 10+ harm categories (incl. defence) | Indian defence harm taxonomy | M4 |
| Multilingual attack support | 5+ Indian languages | Regional-language prompt corpus | M3, M4 |
| Scan duration (7B model) | <4 hours for full suite | Operational-efficiency metric | Orchestrator |
| Air-gapped security | Zero external data exfiltration | Security-audit certification | All |
| OWASP LLM Top 10 compliance | Full coverage (LLM01–LLM10) | Automated mapping and reporting | M6 |

Source: EoI clause 2.4 parameter table; module assignment by ZeroShield Private Limited.

*Interpretation: The proof-of-concept will be accepted against these thresholds; they are conservative, published-benchmark-aligned targets, not aspirational figures.*

The relationship between these targets and the typical capability of the existing tooling categories analysed in Section 4.3 is shown graphically below. The intent of the figure is not to claim superiority in the abstract, but to make explicit the coverage gap that the targets are designed to close.




**Coverage metrics (jailbreak methods, harm categories) are counts shown at full bar height for readability; percentages are detection/accuracy rates. Targets per EoI §2.4.*

**Figure 11. Target capability profile versus typical existing-tool coverage**

*Indicative comparison across the principal assessment dimensions. The existing-tool profile is a conservative composite drawn from public documentation; the proposed-target profile reflects the EoI's figures of merit. The figure illustrates breadth of coverage, not a head-to-head benchmark, which will be conducted during the proof-of-concept.*

### *4.7.4 OWASP, ATLAS, and NIST Coverage*

The framework is designed for explicit, auditable coverage of the recognised frameworks. The figure below shows the coverage map across the OWASP Top 10 for LLM Applications, the relevant MITRE ATLAS tactics, and the functions of the NIST AI Risk Management Framework, indicating which module provides coverage for each. This map is the basis of the certification produced by M6 and of the compliance matrix in Annexure B.

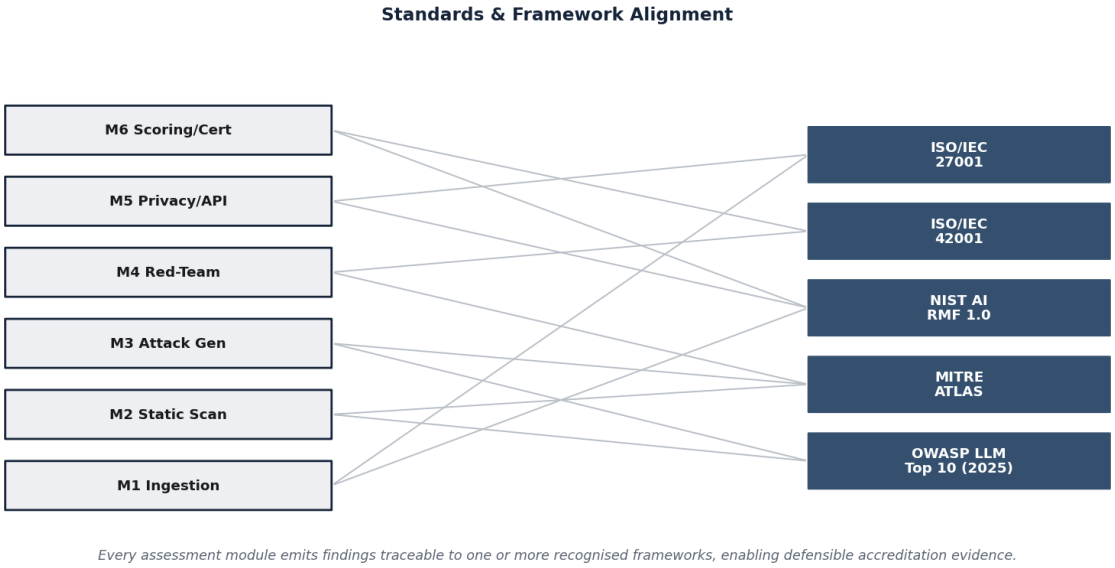



**Figure 12. Framework coverage map (OWASP LLM Top 10, MITRE ATLAS, NIST AI RMF)**

*Each recognised-framework element is mapped to the module that provides coverage. The map demonstrates complete coverage of the OWASP LLM Top 10 and alignment to ATLAS tactics and to the Govern, Map, Measure, and Manage functions of the NIST AI RMF.*

### *4.7.5 Multilingual Specification*

The multilingual requirement is specified in its own right because it is both a functional requirement (support for the named languages) and a quality requirement (the testing must be linguistically valid, not merely machine-translated). The framework's multilingual stage covers the named languages — Hindi, Tamil, Telugu, and Bengali — and code-mixed forms, and applies script-level, transliteration, and translation-relay transformations to expand the attack surface. Per-language results are reported separately so that a model's safety can be assessed language-by-language rather than in aggregate.

**Multilingual Attack-Surface Coverage**

| | English | Hindi | Tamil | Telugu | Bengali | Code-mixed |
|---|---|---|---|---|---|---|
| Direct injection | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Jailbreak / role-play | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Encoding / script | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Back-translation | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Harm-category probes | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

*Indian-language corpora and code-mixing/script-obfuscation mutators extend each attack class beyond English-only coverage provided by foreign tools.*

**Figure 13. Multilingual attack-surface coverage and transformation pipeline**

*The multilingual stage expands each base payload across the named Indian languages and code-mixed forms and applies a set of linguistic transformations. Results are reported per language so that language-specific safety gaps are visible rather than averaged away.*

## 4.8 Technology Stack

### *4.8.1 Selection Principles*

The technology stack is selected against four principles: it must be open and auditable, so that the assurance tool is itself assurable and carries no foreign telemetry; it must be operable fully offline, with all dependencies vendorable into the perimeter; it must be mature and widely supported, to reduce delivery risk; and it must be efficient on the available GPU fabric. Python is the mandatory implementation language for the assessment logic, consistent with the EoI and with the ecosystem of LLM and security tooling.

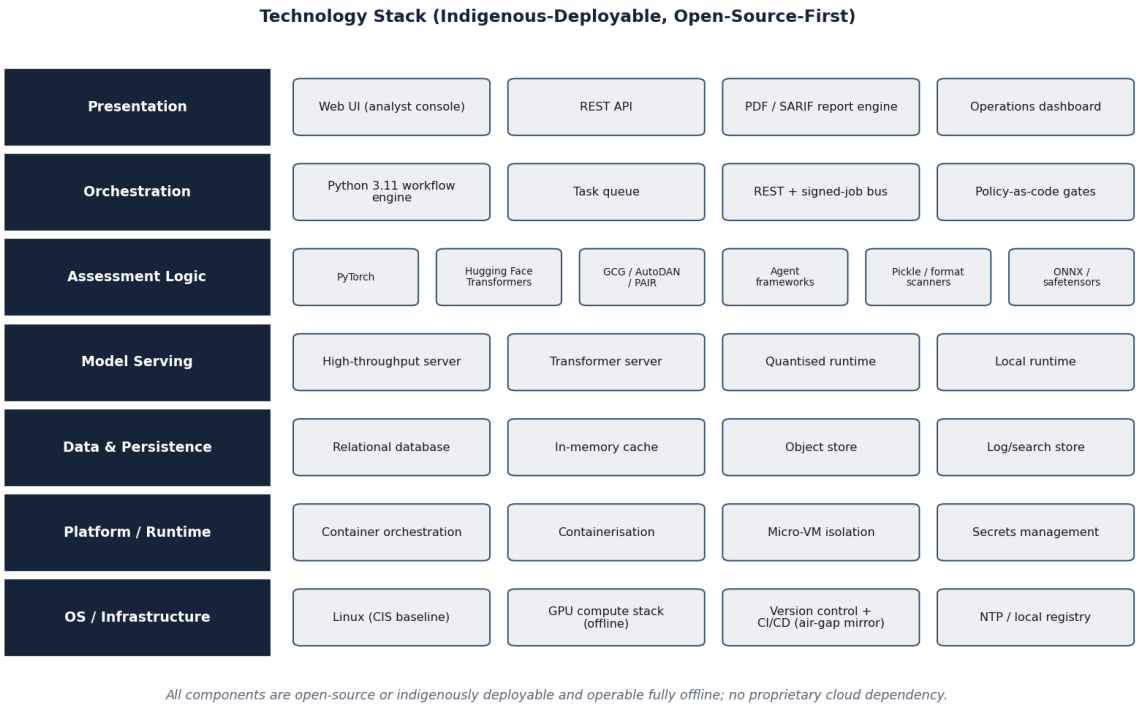



**Figure 14. Indicative technology stack across the architectural layers**

*The stack is organised by layer: presentation and orchestration; assessment-module runtimes; model-serving and adversarial-optimisation engines; data and platform services; and the underlying container and compute substrate. All components are open-source and can be vendored for fully offline installation.*

### *4.8.2 Stack by Layer*

**Table 12. Technology stack by architectural layer with rationale**

| Layer | Representative technologies | Rationale |
|---|---|---|
| Implementation language | Python (mandatory); selective native/compiled extensions where needed | Ecosystem alignment for ML and security; EoI requirement. |
| Model serving (black-box) | High-throughput inference-serving frameworks (offline) | High-throughput inference for hosting test LLMs offline. |
| Adversarial optimisation | PyTorch with GPU acceleration; gradient-based attack implementations | White/grey-box gradient methods (GCG and variants). |

| Layer | Representative technologies | Rationale |
|---|---|---|
| Agentic red-teaming | Multi-agent orchestration frameworks (open-source) | Multi-step, multi-turn attacker-agent orchestration. |
| Artefact analysis | Pickle disassembly, safetensors/ONNX parsers, sandboxed loaders | Format-specific static analysis without execution. |
| Orchestration / workflow | Containerised services with an internal job/message bus and queue | Decoupled, signed-job dispatch across modules. |
| Data services | Relational database; object store; in-memory cache | Structured findings, artefact blobs, and caching. |
| Container / runtime | OCI containers; hardened runtime / VM sandbox with strong isolation | Isolation of untrusted artefacts and module separation. |
| Reporting | Server-side document generation for human- and machine-readable reports | Deterministic, evidence-linked certification output. |

Source: ZeroShield Private Limited technology selection.

*Interpretation: Every layer is realised with open-source, offline-installable components; named products are representative and may be substituted with equivalents during detailed design.*

### *4.8.3 Offline Dependency Management*

Because the framework must operate air-gapped, all dependencies are managed for offline installation. A controlled software bill of materials is maintained; dependencies are mirrored into an in-perimeter package repository; and the integrity of every component is verified on installation. No build or run-time step reaches an external network. This discipline is part of the framework's own supply-chain hygiene and is described further in Section 14.1.

## 4.9 Data Architecture and Data-Flow

### *4.9.1 Data Domains*

The framework manages five principal data domains: model artefacts (the weights and serialized files of the model under test); attack corpora (injection prompts, jailbreak techniques, canary datasets, multilingual payloads, and the defence-harm taxonomy); run-time data (probes issued and responses received during an assessment); findings (the structured results); and evidence and audit data (the immutable record). Each domain has a defined owner, retention rule, and access policy, and each resides exclusively within the perimeter.

### *4.9.2 Data Life-Cycle*

The life-cycle of assessment data — from intake, through generation and execution, to findings, certification, and controlled retention or disposal — is shown below. The life-cycle is designed so that sensitive data (model weights, responses) is minimised, access-controlled, and never leaves the perimeter, while evidence necessary for certification is preserved immutably.

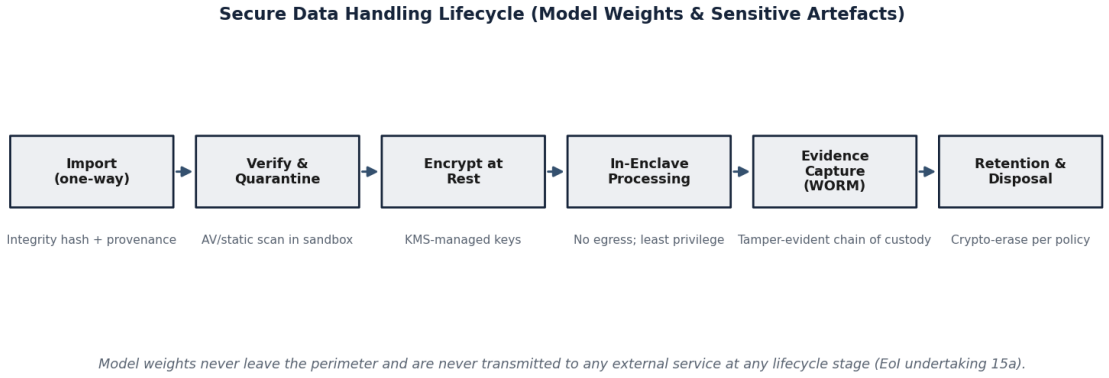



**Figure 15. Data life-cycle from model intake to certified disposal**
*Data flows from controlled intake through generation, execution, and analysis into findings and certification, with evidence written to write-once storage. Sensitive artefacts are retained under access control for the assessment's life and disposed of under a documented rule; evidence is preserved for re-verification.*

### *4.9.3 Context-Level Data-Flow (Level 0)*

The level-0 data-flow diagram presents the framework as a single process interacting with its external entities — the operator, the model under test, and the accreditation consumer of the certification report — and with its principal data stores. It establishes the boundary of the system and the high-level flows across that boundary.

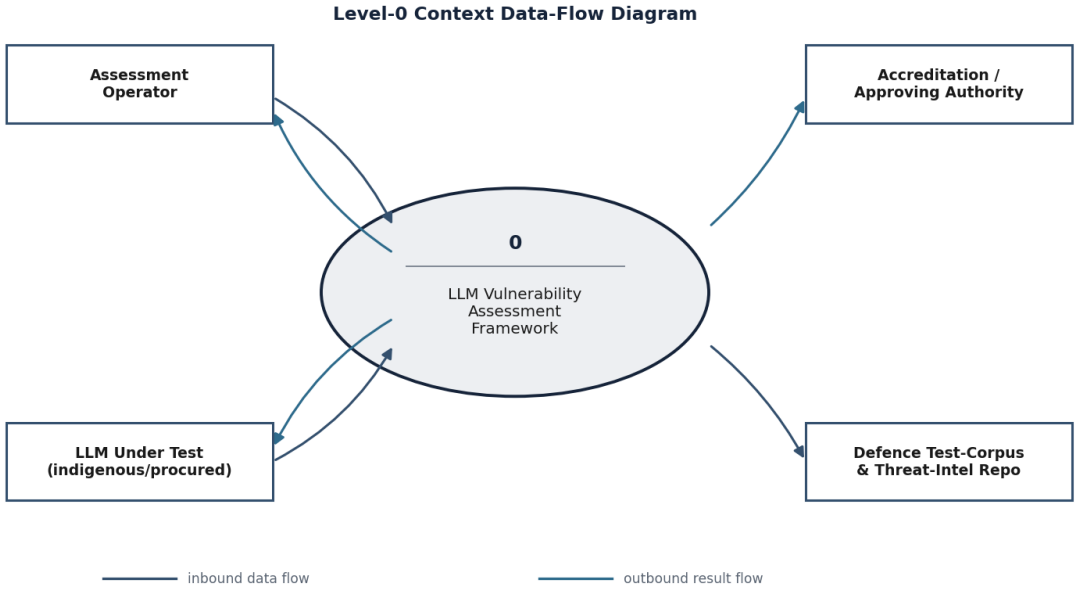



**Figure 16. Level-0 (context) data-flow diagram**

*The context diagram shows the assessment framework as one process bounded by the air-gapped perimeter, receiving an assessment request and a model from the operator, exercising the model under test, and producing a certification report. All external entities are in-perimeter; there is no flow across the perimeter boundary.*

### *4.9.4 Detailed Data-Flow (Level 1)*

The level-1 diagram decomposes the single process of the context diagram into the six module-processes and the data stores that connect them — the artefact and object store, the attack-corpus repository, the findings database, and the write-once audit and evidence log. It shows how an assessment's data moves between the modules and where it comes to rest.

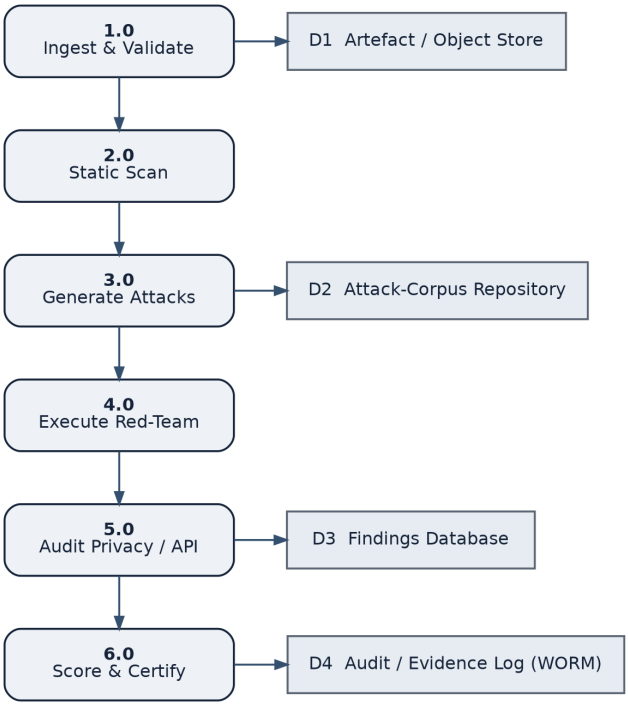



**Figure 17. Level-1 data-flow diagram (decomposition into module-processes)**

*Processes 1.0–6.0 correspond to modules M1–M6. Open-ended rectangles denote persistent data stores. Static scan (2.0) gates the flow into dynamic testing; findings converge on the findings database and the certification process; all stores reside inside the perimeter.*

## 4.10 Detailed Design

### *4.10.1 Component Model*

The component diagram below shows the principal software components of the framework and the interfaces between them. It refines the layered architecture of Section 4.5 into named components with explicit provided and required interfaces, so that the system can be developed and tested component-by-component.

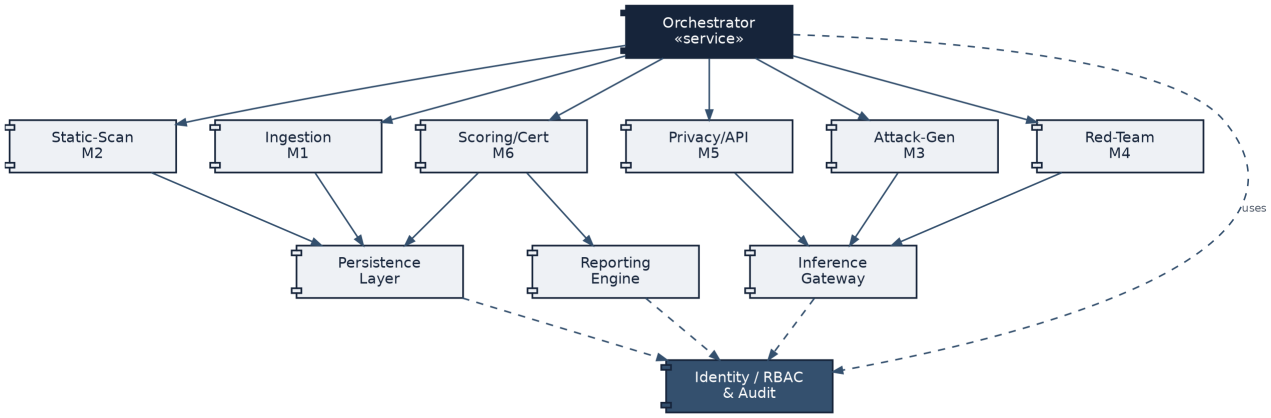



**Figure 18. UML component diagram of the assessment framework**

*Components communicate through explicit interfaces over the internal job/message bus. The orchestrator depends on the module components; all components depend on the data-services and cross-cutting security components. The target-enclave components are reachable only through the controlled inference-probe interface.*

### *4.10.2 Domain Model*

The class diagram below captures the core domain entities and their relationships: the assessment, the target model, the test plan and the tests it contains, the findings produced, the evidence that supports them, and the certification that consolidates them. This domain model is the conceptual basis of the shared finding schema introduced in Section 4.6.

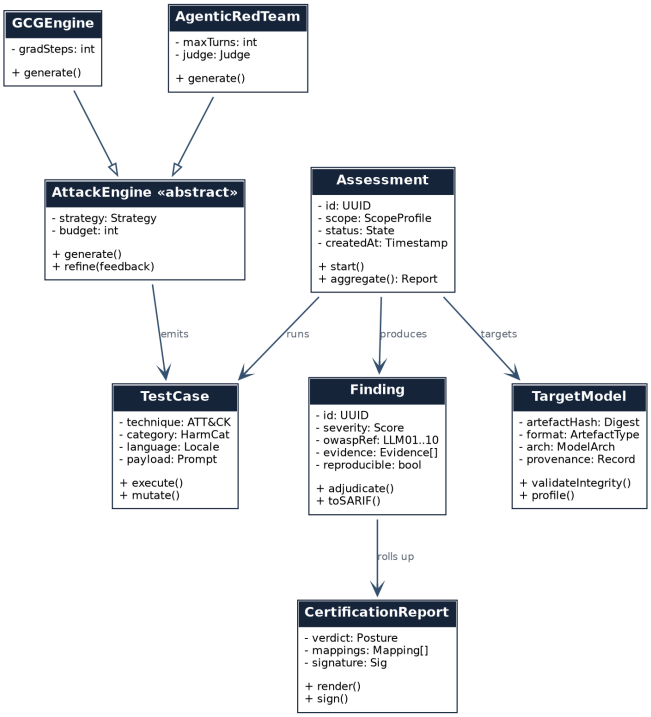



**Figure 19. UML class (domain) diagram of core entities**

*The domain model centres on the Assessment, which references a TargetModel and a TestPlan and aggregates Findings; each Finding carries framework mappings, a severity, and a reference to Evidence; a Certification consolidates the Findings into a scored verdict.*

### *4.10.3 Behavioural Design — Assessment Run*

The sequence diagram below shows the interactions among the operator, the orchestrator, the ingestion module, the scan and attack modules, the target model, and the findings database during a single end-to-end assessment. It makes explicit the order of operations and the points at which evidence is persisted.

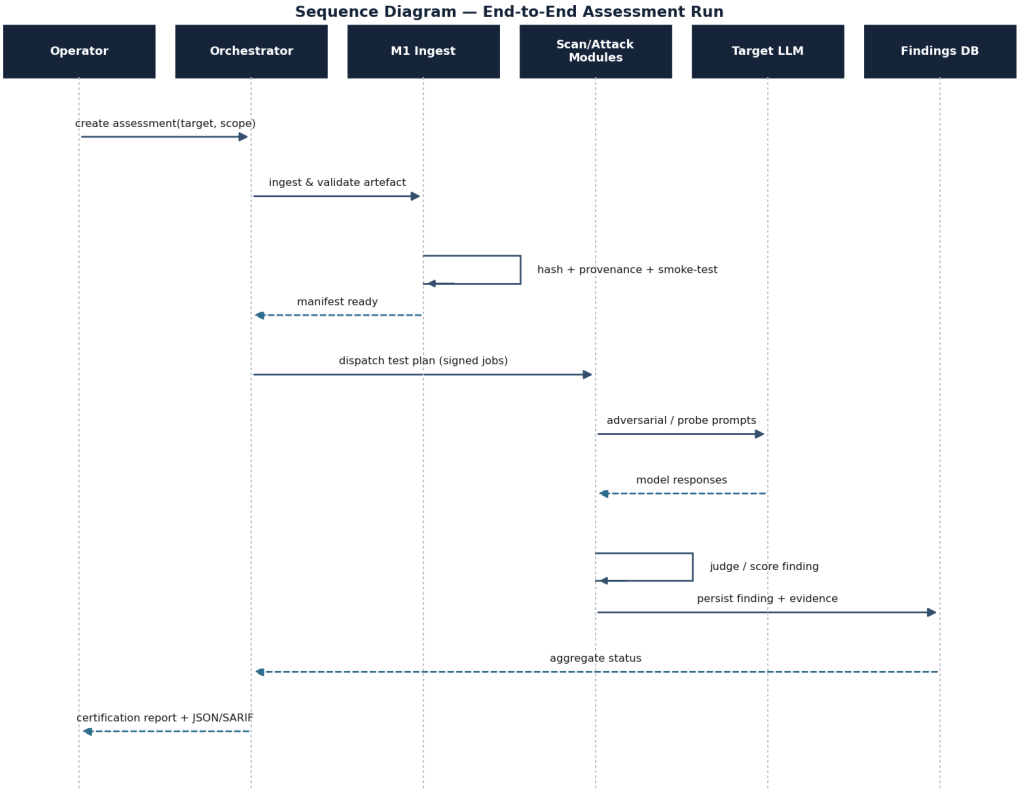



**Figure 20. Sequence diagram — end-to-end assessment run**

*The operator creates an assessment; the orchestrator drives ingestion and, once cleared, dispatches signed test jobs; the scan and attack modules exercise the target model and persist findings and evidence; the orchestrator aggregates status and the certification report is returned. Dashed arrows denote returns.*

### *4.10.4 Behavioural Design — Agentic Red-Team Loop*

The second sequence diagram details the multi-turn red-teaming loop of module M4: the iterative exchange between the attacker agent, the target model, and the evaluator, over a bounded number of turns, with each trajectory scored and recorded.

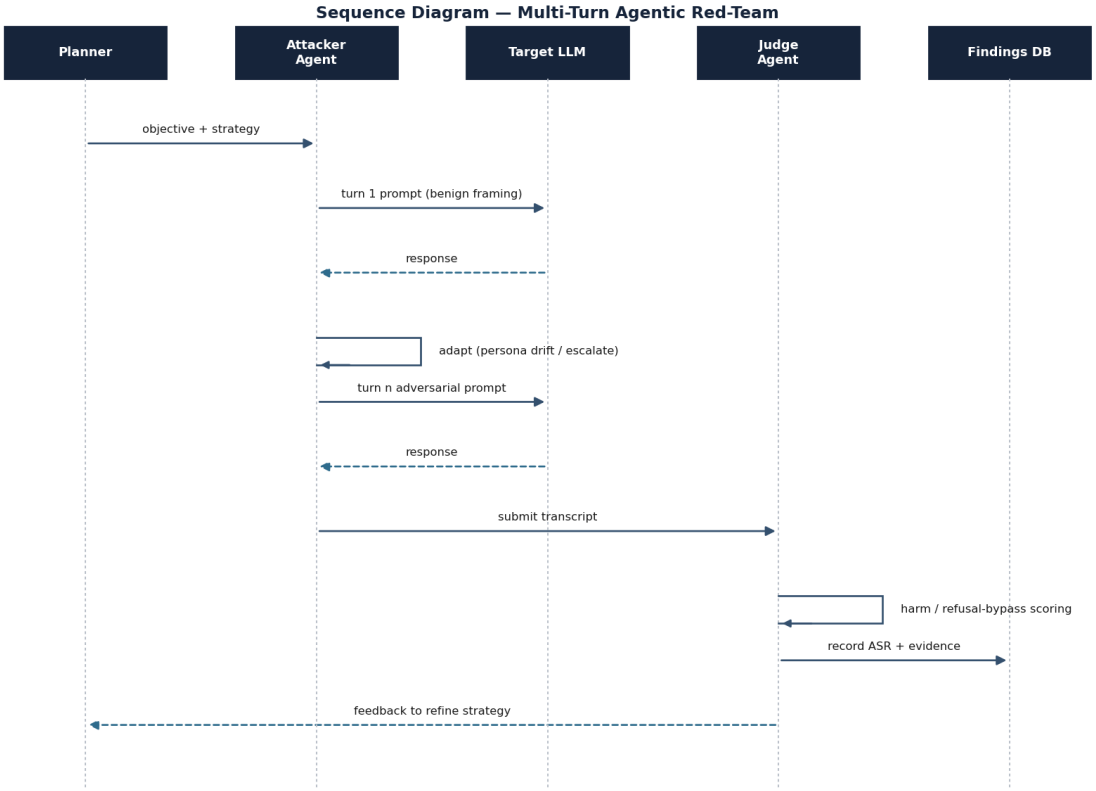



**Figure 21. Sequence diagram — multi-turn agentic red-team loop**

*The attacker agent plans and issues probes, observes responses, and adapts over multiple turns; the evaluator scores each trajectory; the loop terminates on success or turn-budget exhaustion, and the transcript and score are persisted as evidence.*

### *4.10.5 Assessment State Model*

The state machine below defines the life-cycle states of an assessment — from created, through ingesting, scanning, generating, executing, auditing, and scoring, to certified or failed — and the transitions between them, including the gating transition that blocks dynamic testing until the static scan has cleared the model.

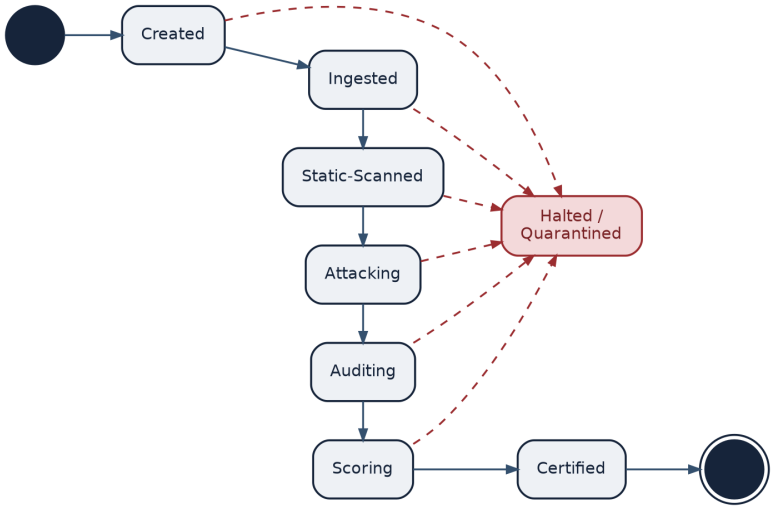



**Figure 22. Assessment life-cycle state machine**

*The state machine enforces the gating discipline: the transition from static analysis to dynamic testing is conditional on a clear verdict; a malicious-artefact verdict transitions directly to a terminal failed state with the finding recorded.*

## 4.11 Security Architecture

### *4.11.1 Security Objectives*

The framework's security architecture has two distinct objectives. The first is to protect the framework and the facility from the model under test, which must be treated as potentially hostile. The second is to protect the sensitive data the framework handles — weights, responses, and findings — and to guarantee the integrity of the evidence on which certification rests. The architecture below addresses both.

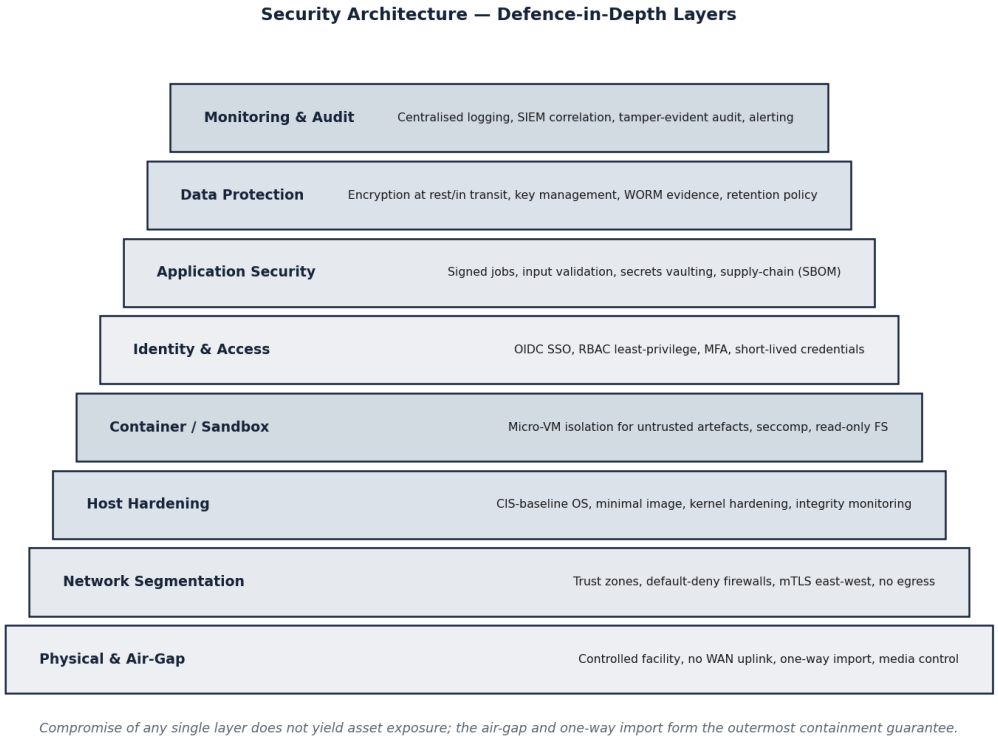



**Figure 23. Security architecture and trust zones**

*The architecture defines concentric trust zones: the operator zone, the control and services zone, and the strongly isolated target enclave. Controls — RBAC, mutual TLS, signed jobs, sandboxing, secrets management, integrity attestation, and write-once audit logging — are placed at the zone boundaries. The perimeter has no outbound route.*

### *4.11.2 Trust Zones and Isolation*

Three trust zones are defined. The operator zone contains the consoles from which operators interact; it can reach the control plane but not the target enclave. The control and services zone contains the orchestrator, the modules, and the data services. The target enclave contains the model under test, the artefact sandbox, and the probe harness, and is the least trusted zone. Inter-zone communication is mediated, authenticated with mutual TLS, and restricted to the minimum necessary flows; the enclave can receive jobs and return evidence but cannot initiate connections outward or laterally.

### *4.11.3 Controls*

**Table 13. Security controls, purpose, and placement**

| Control | Purpose | Placement |
|---|---|---|
| Air-gap / deny-all egress | Prevent any data exfiltration; enforce sovereignty. | Perimeter |
| Role-based access control | Limit actions to authorised roles; separation of duties. | Control plane and services |
| Mutual TLS on internal bus | Authenticate every service-to-service call. | Services |
| Signed job descriptors | Ensure modules act only on authentic, untampered instructions. | Orchestrator → modules |
| Sandboxed artefact execution | Contain untrusted serialized artefacts; no privileged or lateral access. | Target enclave |
| Secrets management | Centralise and protect credentials and keys. | Cross-cutting |
| Integrity attestation | Verify framework components and artefacts are unmodified. | Cross-cutting |
| Write-once audit log | Provide tamper-evident evidence for re-verification. | Cross-cutting |
| Data minimisation and retention | Limit retention of sensitive data to assessment life. | Data services |

Source: ZeroShield Private Limited security design, aligned to ISO/IEC 27001:2022 control themes and NIST AI RMF.

*Interpretation: Controls are placed at zone boundaries and applied cross-cuttingly, realising defence-in-depth and the air-gap-by-construction principle.*

### *4.11.4 Threats to the Framework and Mitigations*

The framework anticipates that a malicious model or artefact may attempt to escape containment, exfiltrate data, or corrupt evidence. A weaponised pickle is neutralised by static analysis before instantiation (M2) and, if dynamic loading is unavoidable, by sandbox confinement. An attempt by a running model to reach the network is defeated by the deny-all egress posture and the absence of any outbound route. An attempt to tamper with findings is defeated by the write-once, integrity-attested evidence log. These mitigations are the security counterpart of the design principles in Section 4.4.

## 4.12 Deployment and Network Topology

### *4.12.1 Deployment Model*

The framework is delivered as software deployable on air-gapped GPU infrastructure within laboratory premises, Service AI units, or designated cyber-defence organisations. It is packaged as a set of containerised services orchestrated on an in-perimeter cluster, with the GPU fabric serving adversarial-optimisation and inference workloads. The deployment topology below shows the physical and logical placement of components across the cluster and the enclave.

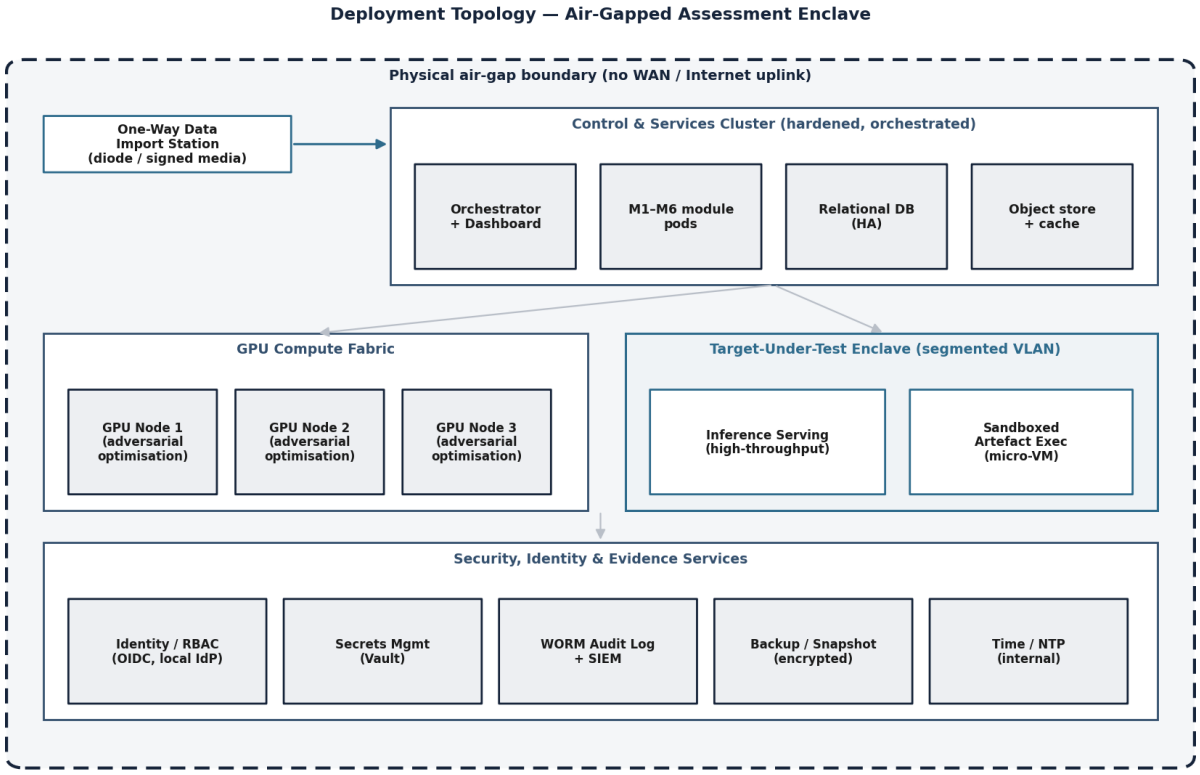



**Figure 24. Deployment topology on air-gapped GPU infrastructure**

*Control-plane and services nodes, data-services nodes, and GPU nodes are placed on an isolated cluster; the target enclave is a dedicated, strongly isolated segment. The diagram shows where each module and service runs and how the GPU fabric is partitioned between optimisation and serving.*

### *4.12.2 Network Topology and Segmentation*

The network is segmented into the operator, control/services, data, GPU, and enclave segments, with controlled, authenticated flows between them and no route to any external network. The network diagram below shows the segments, the permitted flows, and the enforcement points. The defining property of the topology is that there is no egress path: the perimeter is closed by construction.

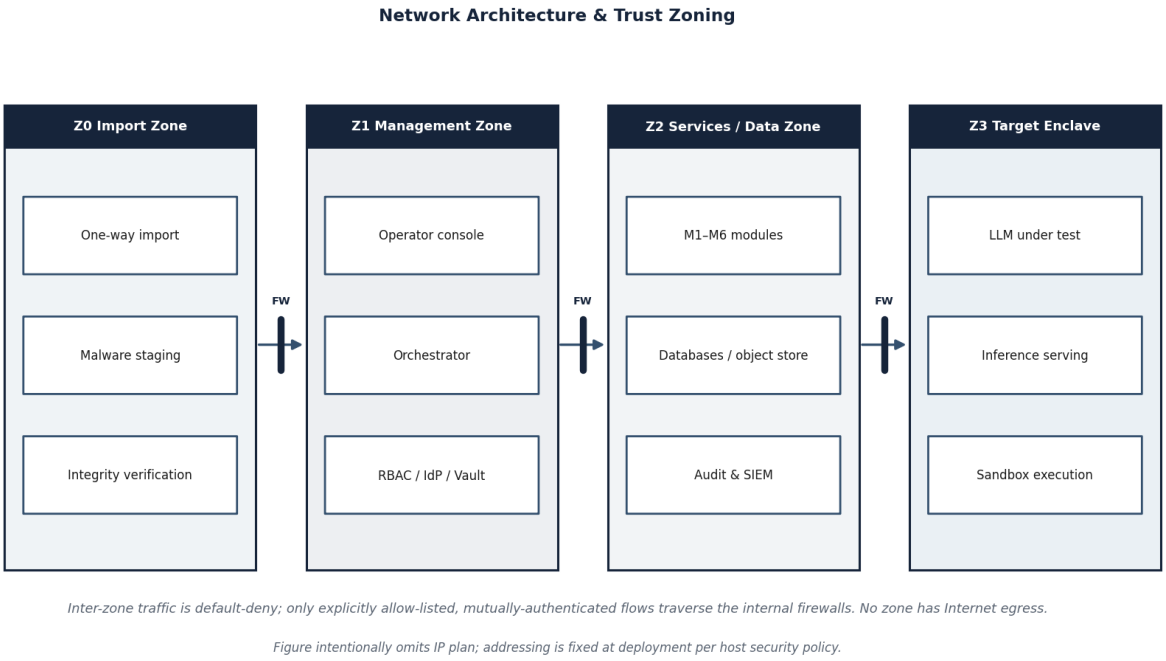



**Figure 25. Network topology and segmentation within the air-gapped perimeter**
*Network segments are separated by enforcement points that permit only the minimum necessary, authenticated flows. The target-enclave segment is the most restricted. No segment has a route to any external network; the perimeter boundary carries no egress.*

### *4.12.3 Air-Gap Conformance*

Air-gap conformance is treated as a verifiable property, not an assumption. During the proof-of-concept, egress from every segment is monitored and is required to be nil; the absence of any outbound route is confirmed by inspection of the network configuration; and the offline dependency-management discipline (Section 4.8.3) ensures that no build or run-time step requires connectivity. The conformance result is recorded as part of the certification evidence and satisfies the zero-exfiltration figure of merit.

### *4.12.4 Scalability and Portability*

The containerised design allows the framework to be deployed at different scales — from a single multi-GPU workstation for a small assessment to a multi-node cluster for larger models — without change to the application. Because all dependencies are vendorable and the components are open-source, the framework is portable across compliant Indian infrastructure and can be re-deployed into different accredited facilities. Performance targets, including the sub-four-hour full-suite scan for a seven-billion-parameter model, are met by partitioning the GPU fabric and parallelising independent tests across module instances.

## 4.13 Work Breakdown Structure

### *4.13.1 Decomposition Approach*

The programme of work is decomposed into a four-level work breakdown structure (WBS). Level 0 is the programme itself; Level 1 comprises eight work packages that group related engineering effort; Level 2 breaks each work package into activities; and Level 3 identifies the discrete deliverables and verifiable outcomes that constitute the definition of done for each activity. The decomposition is deliverable-oriented rather than organisation-oriented, so that every element of scope in the EoI maps to exactly one work package and so that progress can be measured by the completion of tangible artefacts rather than by elapsed time. The structure is shown in the figure below and elaborated in the work-package register that follows.

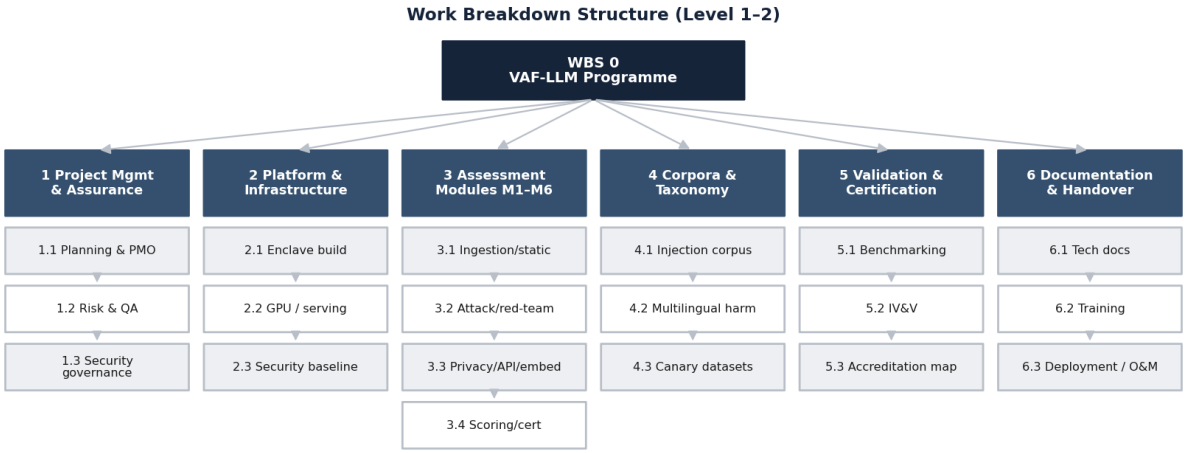



**Figure 26. Work breakdown structure for the proof-of-concept programme**

*Level 0 denotes the programme; Level 1 denotes the eight work packages (WP1–WP8); the indicative activities beneath each work package are Level 2. The mapping of work packages to the six assessment modules and to the EoI scope items is given in the work-package register and in the traceability matrix at Annexure B.*

### *4.13.2 Work-Package Register*

The eight work packages and their principal activities, deliverables and the modules they realise are recorded below. Each work package has a single accountable owner within the engineering team and a defined set of entry and exit criteria; the exit criteria are the deliverables listed in the right-hand column.

**Table 14. Work-package register and mapping to assessment modules**

| WP | Work package | Principal activities | Primary deliverables |
|---|---|---|---|
| WP1 | Programme management and engineering governance | Planning, configuration management, milestone tracking, risk and quality governance, stakeholder reporting, IV&V; coordination. | Project management plan; configuration and version baselines; milestone and risk reports; review records. |
| WP2 | Secure platform and air-gapped environment | Provisioning of the isolated environment; container and orchestration baseline; offline artefact repository; identity, logging and evidence services (M1). | Hardened platform build; offline dependency mirror; environment acceptance record; egress-nil verification. |
| WP3 | Model ingestion and static artefact scanning | Secure ingestion workflow; format-aware parsers for pickle / safetensors / ONNX / checkpoint artefacts; malicious object, embedded-URL and bytecode detection; sandboxed deserialisation (M1, M2). | Ingestion service; static-analysis engine; signature and heuristic rule-set; artefact-scan report schema. |
| WP4 | Adversarial attack generation | Gradient and discrete optimisation (GCG-class); prompt-injection fuzzing (direct, indirect, cross-modal); jailbreak technique library (50+); multilingual corpus integration (M3). | Attack-generation engine; technique library; injection and jailbreak corpora; robustness-scoring routine. |
| WP5 | Agentic red-teaming | Multi-turn conversational attack orchestration; agent framework integration; refusal-mechanism and safety-bypass evaluation across harm categories (M4). | Red-teaming agent harness; scenario library; multi-turn transcript store; safety-bypass findings schema. |
| WP6 | Privacy, API and embedding audit | Membership-inference (LiRA / RMIA); training-data and secret extraction; inference-API security checks; embedding-space and backdoor-trigger analysis (M5). | Privacy-audit module; canary harness; API-audit checks; representation-analysis routines. |
| WP7 | Certification, scoring and reporting | Severity scoring; OWASP LLM Top 10, MITRE ATLAS and NIST AI RMF mapping; certification dashboard; evidence packaging (M6). | Scoring engine; mapping tables; certification dashboard; machine-readable and human-readable report templates. |
| WP8 | Validation, corpus curation and proof-of-concept | Curation and validation of the test corpus; assembly of the ~100 validated test cases; end-to-end runs against open-weight reference models; documentation and demonstration. | Validated test-case set; benchmark results; user and operations documentation; proof-of-concept demonstration package. |

Source: EoI scope of work (Sections 4.2.1–2.5) and the six-module architecture of Section 4.6.

*Interpretation: Each work package owns a coherent slice of scope and exits on tangible deliverables. WP2–WP7 realise modules M1–M6 respectively; WP1 and WP8 provide governance and validation that span all modules.*

### *4.13.3 Definition of Done and Acceptance*

An activity is complete only when its deliverables exist under configuration control, have passed the relevant verification, and have been recorded in the evidence repository. For engineering components this means the code is integrated, tested and documented; for corpora it means the content is curated, labelled and validated; and for the framework as a whole it means the figures of merit defined in Section 4.7.3 have been measured against the reference models. Acceptance at the programme level is the successful demonstration of the end-to-end proof-of-concept and the delivery of the approximately one hundred validated test cases required by the EoI.

## 4.14 Detailed OWASP LLM Top 10 (2025) Coverage and Test Approach

### *4.14.1 Purpose*

This section elaborates, for each risk in the OWASP Top 10 for Large Language Model Applications (2025 edition), how the proposed framework tests for the risk, which module is responsible, the figure of merit by which coverage is judged, and the form that a finding takes. It supplements the coverage map of Section 4.7.4 with the depth required for technical evaluation, and demonstrates that the framework's scope is anchored to a recognised, industry-standard taxonomy rather than to an ad-hoc checklist. No operational attack content is reproduced here; the description is confined to method and coverage.

### *4.14.2 Risk-to-Module-to-Metric Mapping*

**Table 15. OWASP LLM Top 10 (2025) mapped to modules and figures of merit**

| OWASP ref | Risk | Primary module(s) | Figure of merit |
|---|---|---|---|
| LLM01 | Prompt Injection | M3, M4 | Detection rate on known and zero-shot injection corpora. |
| LLM02 | Sensitive Information Disclosure | M5 | Extraction success rate; membership-inference accuracy on canaries. |
| LLM03 | Supply Chain | M2 | Malicious-artefact detection rate; SBOM verification coverage. |
| LLM04 | Data and Model Poisoning | M2, M5 | Backdoor-trigger detection; anomalous-representation flag rate. |
| LLM05 | Improper Output Handling | M3, M4 | Rate of unsafe / unsanitised outputs elicited under test. |
| LLM06 | Excessive Agency | M4, M5 | Tool / function-abuse success rate; over-privilege findings. |
| LLM07 | System-Prompt Leakage | M3, M4 | System-prompt extraction success rate. |
| LLM08 | Vector and Embedding Weaknesses | M5 | Embedding-anomaly and trigger-activation detection. |
| LLM09 | Misinformation | M4 | Rate of confidently false / fabricated outputs under probing. |
| LLM10 | Unbounded Consumption | M5 | Resource-exhaustion / denial-of-service susceptibility. |

Source: OWASP Top 10 for LLM Applications (2025) and the module design of Section 4.6.

*Interpretation: Every OWASP risk is owned by at least one module and is associated with a measurable figure of merit, so that coverage can be demonstrated rather than asserted.*

### *4.14.3 Per-Risk Test Approach*

The following describes, for each risk, the testing approach and the evidence a finding records. The descriptions are deliberately at the level of method.

### LLM01 — Prompt Injection

The framework tests direct injection (adversarial instructions in the user input), indirect injection (adversarial instructions in content the model retrieves or is given), and cross-modal and encoded variants. Module M3 generates and mutates injection vectors from a curated corpus and through fuzzing; Module M4 exercises multi-turn injection through agentic conversation. Coverage is measured as the detection rate against the known-pattern corpus (target above ninety-five percent) and against held-out zero-shot patterns (target above seventy percent). A finding records the injection class, the eliciting vector in sanitised form, the model's deviation from intended behaviour, and reproducibility.

### LLM02 — Sensitive Information Disclosure

Module M5 probes for disclosure of training data, personally identifiable information, secrets, and source material. Membership inference (LiRA and RMIA) quantifies whether specific records were in the training set; canary-based extraction measures verbatim regurgitation of planted secrets. Coverage is measured by membership-inference accuracy on canary datasets (target above eighty percent) and by extraction success rate. A finding records the disclosure type, the canary or probe used, and the leaked content category (without reproducing sensitive values).

### LLM03 — Supply Chain

Module M2 inspects the provenance and integrity of model artefacts and their dependencies: serialised-object safety, embedded URLs and telemetry, tampering, and software-bill-of-materials verification. Coverage is measured by the malicious-artefact detection rate (target above ninety percent on a labelled corpus) and by SBOM verification completeness. A finding records the artefact, the specific supply-chain indicator, and the containment action taken.

### LLM04 — Data and Model Poisoning

Modules M2 and M5 together test for poisoning and backdoors: static indicators in the artefact, trigger-phrase activation, and anomalous internal representations consistent with sleeper-agent behaviour. Coverage is measured by trigger-detection and anomalous-representation flag rates against seeded test models. A finding records the suspected trigger or anomaly and the evidence of altered behaviour.

### LLM05 — Improper Output Handling

Modules M3 and M4 elicit outputs that, if passed unsanitised to downstream systems, would be dangerous — for example outputs containing executable content, markup, or injection payloads targeting consumers of the model. Coverage is measured by the rate at which such unsafe outputs are elicited. A finding records the eliciting prompt class and the unsafe output category.

### LLM06 — Excessive Agency

Where the target exposes tools or function-calling, Module M4 tests whether the model can be induced to take actions beyond its intended authority, and Module M5 audits the permission surface. Coverage is measured by the tool-abuse success rate and the count of over-privilege findings. A finding records the action induced and the authority boundary crossed.

### LLM07 — System-Prompt Leakage

Modules M3 and M4 attempt to extract the model's hidden system prompt or configuration through direct and conversational techniques. Coverage is measured by the system-prompt extraction success rate. A finding records the extraction technique class and the category of leaked configuration (without reproducing sensitive instructions).

#### LLM08 — Vector and Embedding Weaknesses

Module M5 analyses the embedding space for weaknesses exploitable in retrieval-augmented or embedding-dependent deployments: anomalous clusters, trigger-activated regions, and representation-engineering susceptibility. Coverage is measured by embedding-anomaly and trigger-activation detection. A finding records the anomaly and its potential exploitation path.

#### LLM09 — Misinformation

Module M4 probes the model's propensity to produce confident but false or fabricated output, including in defence-relevant and Indian-context domains. Coverage is measured by the rate of confidently false outputs under structured probing. A finding records the probe domain and the nature of the fabrication.

#### LLM10 — Unbounded Consumption

Module M5 tests susceptibility to resource-exhaustion and denial-of-service through adversarial inputs that induce excessive computation or output. Coverage is measured by susceptibility under controlled load probes. A finding records the consumption vector and the observed resource impact.

### *4.14.4 ATLAS and NIST Alignment*

Beyond OWASP, each finding is tagged with the relevant MITRE ATLAS tactic and technique, and is aligned to the Measure and Manage functions of the NIST AI Risk Management Framework. This dual tagging allows the End-User to consume the framework's evidence within whichever taxonomy its accreditation process uses, and ensures that the certification output is defensible against more than one recognised reference. The alignment is summarised in Section 4.17.4.

## 4.15 Evaluation Methodology, Metrics and Statistical Validation

### *4.15.1 Principles of Measurement*

The framework's claims rest on measurement, not assertion. Every figure of merit defined in the EoI and in Section 4.7.3 is associated with a precisely defined metric, a measurement procedure, a reference dataset, and a statistical treatment that expresses uncertainty. The firm distinguishes between detection metrics (how well the framework finds known issues), discovery metrics (its ability to surface previously unknown issues), and efficiency metrics (the time and resources a scan consumes). Results are reported with the assumptions and the dataset on which they were obtained, so that a result is interpretable and reproducible rather than a bare number.

### *4.15.2 Metric Definitions*

The principal metrics used across the modules are defined below. These definitions are standard in the security and machine-learning literature and are adopted without modification so that the framework's results are comparable with published work.

**Table 16. Definitions of the principal evaluation metrics**

| Metric | Definition | Used for |
|---|---|---|
| Detection rate | Proportion of known-malicious or known-unsafe cases correctly flagged. | Prompt injection, artefact malware, supply chain. |
| Attack-success rate (ASR) | Proportion of attack attempts that achieve the adversarial objective against the target. | Jailbreak, red-teaming, output handling. |
| Precision | Proportion of flagged cases that are true positives (1 – false-discovery rate). | All detection modules; controls false alarms. |
| Recall (sensitivity) | Proportion of all true issues that are detected. | All detection modules; controls misses. |
| F1 score | Harmonic mean of precision and recall. | Balanced summary of detection quality. |
| TPR at low FPR | True-positive rate at a fixed, low false-positive rate (e.g. 1% or 0.1%). | Membership inference (the meaningful operating point). |
| AUC | Area under the receiver-operating-characteristic curve. | Membership inference; threshold-independent quality. |
| Coverage | Count or proportion of techniques, languages, or harm categories exercised. | Jailbreak library, multilingual, harm taxonomy. |
| Scan duration | Wall-clock time for a defined test profile on a model of stated size. | Efficiency / the sub-four-hour target. |
| Reproducibility rate | Proportion of findings that reproduce on independent re-execution with the recorded seed. | Evidence integrity across all modules. |

Source: Standard definitions from the security and machine-learning literature.

*Interpretation: Adopting standard metric definitions makes the framework's results interpretable, comparable, and defensible under technical scrutiny.*

### *4.15.3 Statistical Validation*

Because attack generation and several detection methods are stochastic, single-run numbers are insufficient. The framework reports metrics over repeated runs with fixed and recorded random seeds, and accompanies point estimates with confidence intervals obtained by bootstrap resampling where appropriate. For membership-inference results, performance is reported at fixed low false-positive operating points and as an AUC, in line with current best practice, rather than as a single accuracy figure that can be misleading. Where a target is approached but not met, the interim value is reported honestly with its interval rather than rounded up or presented selectively.

Baselines are established before improvement is claimed: a model's behaviour is characterised on a benign baseline so that the effect of an attack is measured as a deviation from that baseline rather than in absolute terms. Reference open-weight models are used for development and calibration so that the methodology can be validated against models whose properties are publicly understood before it is applied to sensitive targets.

### *4.15.4 Acceptance Methodology per Figure of Merit*

The table below binds each EoI figure of merit to its metric, its target, and the procedure by which the proof-of-concept will demonstrate it. This is the acceptance basis against which the framework's performance will be judged.

**Table 17. Acceptance methodology binding figures of merit to metrics and procedures**

| Figure of merit | Metric | Target | Demonstration procedure |
|---|---|---|---|
| Known prompt-injection detection | Detection rate | > 95% | Evaluate against the labelled known-injection corpus; report rate with interval. |
| Zero-shot injection detection | Detection rate | > 70% | Evaluate against held-out, unseen injection patterns. |
| Jailbreak technique coverage | Coverage (count) | 50+ techniques | Execute the technique library; enumerate techniques exercised. |
| Membership-inference accuracy | TPR at low FPR / AUC | > 80% on canaries | Run LiRA and RMIA on canary datasets; report at fixed FPR with AUC. |
| Artefact-malware detection | Detection rate | > 90% | Evaluate against the labelled malicious-artefact corpus. |
| Harm-category coverage | Coverage (count) | 10+ categories | Exercise the harm taxonomy including defence-specific categories. |
| Language coverage | Coverage (count) | 5+ Indian languages | Execute multilingual corpora across the required languages. |
| Full-suite scan time (7B model) | Scan duration | < 4 hours | Time the standard full-suite profile on a seven-billion-parameter reference model. |
| Zero data exfiltration | Egress count | Nil | Monitor all segments for outbound traffic; require nil throughout the scan. |
| OWASP LLM coverage | Coverage | LLM01–LLM10 | Demonstrate at least one operative test per OWASP risk (Section 4.14). |

Source: EoI detailed specifications (§2.4) and the metric definitions of Section 4.15.2.

*Interpretation: Each acceptance criterion is unambiguous and measurable; the proof-of-concept succeeds against a criterion only when the stated metric meets the stated target by the stated procedure.*

### *4.15.5 Measurement Integrity*

All measurement is performed within the air-gapped environment and recorded in the tamper-evident evidence log, so that a reported figure can be traced to the run that produced it. The separation of the act of building a capability from the act of measuring it (Section 14.1.5) ensures that performance claims are independent of development optimism. This discipline is the basis on which the End-User, or an independent verifier, can rely on the framework's reported results.

## 4.16 Detailed Module Specifications

### *4.16.1 Specification Format*

Sections 4.6 described the function of each module. This section specifies each module in the form of its inputs, its principal processing steps, its outputs, and the acceptance criteria by which the module is judged complete. The specifications are written so that they can be carried forward, with refinement, into the detailed design at the RFP stage.

### *4.16.2 M1 — Model Ingestion and Secure Setup*

**Table 18. Specification of module M1**

| Aspect | Specification |
|---|---|
| Inputs | Target model artefacts (weights, tokenizer, configuration) in supported formats; assessment scope; operator authorisation. |
| Principal processing | Controlled one-way import into the enclave; integrity hashing and provenance recording; format detection and structural validation; architecture and capability profiling; allocation of enclave resources; bring-up of the inference server; baseline functional smoke test; registration of the assessment manifest. |
| Outputs | A loaded, profiled target ready for testing; an integrity and provenance record; an assessment manifest; a baseline behavioural profile. |
| Acceptance criteria | Artefact integrity is verified and recorded; the target serves inference within the enclave; egress from the enclave is nil; the manifest is registered and auditable. |

Source: Module design (Section 4.6) and functional specification (Section 4.7.2).

*Interpretation: The module is complete when its outputs are produced from its inputs by the specified processing and the acceptance criteria are met under measurement.*

### *4.16.3 M2 — Static Analysis and Artefact Scanning*

**Table 19. Specification of module M2**

| Aspect | Specification |
|---|---|
| Inputs | Imported model artefacts and their dependency manifest; known-indicator and signature sets. |
| Principal processing | Safe disassembly and inspection of serialised objects; detection of dangerous callables, embedded URLs, telemetry and suspicious bytecode; software-bill-of-materials verification; known-malicious hash matching; sandboxed deserialisation of anything not provably safe; quarantine of suspect artefacts. |
| Outputs | A static-analysis report enumerating indicators; a quarantine decision per artefact; findings queued with provisional severity. |
| Acceptance criteria | Malicious-artefact detection rate meets the target on the labelled corpus; no unsafe artefact is loaded without quarantine; SBOM verification is complete for supplied dependencies. |

Source: Module design (Section 4.6) and functional specification (Section 4.7.2).

*Interpretation: The module is complete when its outputs are produced from its inputs by the specified processing and the acceptance criteria are met under measurement.*

### *4.16.4 M3 — Adversarial Attack Generation*

**Table 20. Specification of module M3**

| Aspect | Specification |
|---|---|
| Inputs | The loaded target; injection and jailbreak corpora; the multilingual corpus; an attack budget. |
| Principal processing | Gradient and discrete optimisation of adversarial suffixes; prompt-injection fuzzing across direct, indirect and cross-modal vectors; execution of the jailbreak technique library; application of multilingual and code-mixed mutators; robustness scoring of the target's responses. |
| Outputs | A set of executed attacks with outcomes; a robustness score; findings for successful attacks with sanitised reproduction evidence. |
| Acceptance criteria | Known and zero-shot injection detection rates meet their targets; the jailbreak library covers at least the required number of techniques; the required languages are exercised. |

Source: Module design (Section 4.6) and functional specification (Section 4.7.2).

*Interpretation: The module is complete when its outputs are produced from its inputs by the specified processing and the acceptance criteria are met under measurement.*

### *4.16.5 M4 — Agentic Red-Teaming Engine*

**Table 21. Specification of module M4**

| Aspect | Specification |
|---|---|
| Inputs | The loaded target; an objective and strategy; the harm taxonomy; conversation budget. |
| Principal processing | Multi-turn conversational attack orchestration with planning and adaptation; persona drift and gradual escalation; tool and function-call abuse probing where applicable; refusal-mechanism and alignment-bypass evaluation; automated judging of transcripts. |
| Outputs | Reproducible multi-turn transcripts; attack-success-rate metrics by harm category; findings with judged evidence. |
| Acceptance criteria | Harm-category coverage meets the target including defence-specific categories; transcripts are reproducible; judging is consistent and recorded. |

Source: Module design (Section 4.6) and functional specification (Section 4.7.2).

*Interpretation: The module is complete when its outputs are produced from its inputs by the specified processing and the acceptance criteria are met under measurement.*

### *4.16.6 M5 — Privacy, API and Embedding Audit*

**Table 22. Specification of module M5**

| Aspect | Specification |
|---|---|
| Inputs | The loaded target and, where exposed, its inference API; canary datasets; reference statistics. |
| Principal processing | Membership inference (LiRA and RMIA); canary-based extraction testing; PII, secret and source regurgitation probing; inference-API authentication, authorisation, rate-limit and denial-of-service checks; embedding-space anomaly and trigger-activation analysis. |
| Outputs | Privacy-leakage metrics at defined operating points; API-security findings; embedding-analysis findings; extraction evidence by category. |
| Acceptance criteria | Membership-inference accuracy meets the target on canaries at a fixed low false-positive rate; API and embedding checks complete; findings are reproducible. |

Source: Module design (Section 4.6) and functional specification (Section 4.7.2).

*Interpretation: The module is complete when its outputs are produced from its inputs by the specified processing and the acceptance criteria are met under measurement.*

### *4.16.7 M6 — Scoring, Certification and Reporting*

**Table 23. Specification of module M6**

| Aspect | Specification |
|---|---|
| Inputs | Findings from all modules with their evidence; the OWASP, ATLAS and NIST mapping tables; the severity-scoring rules. |
| Principal processing | Finding deduplication and confidence adjudication; severity scoring; mapping to OWASP LLM Top 10, MITRE ATLAS and NIST AI RMF; construction of the certification verdict; packaging of human-readable and machine-readable reports. |
| Outputs | A severity-scored, standards-mapped certification report; a machine-readable findings file; a reproducible evidence package; a risk-posture verdict with remediation guidance. |
| Acceptance criteria | Every finding is scored and mapped; the report renders end-to-end; the machine-readable output validates against its schema; the evidence package is complete and auditable. |

Source: Module design (Section 4.6) and functional specification (Section 4.7.2).

*Interpretation: The module is complete when its outputs are produced from its inputs by the specified processing and the acceptance criteria are met under measurement.*

### *4.16.8 Inter-Module Contract*

The modules communicate only through the internal signed-job bus and the shared persistence layer, never directly, so that each module can be developed, tested and replaced independently. A finding produced by any module conforms to the common finding schema (Section 4.18.4), which carries the module of origin, the OWASP and ATLAS tags, the severity, the evidence references, and the reproducibility flag. This contract is what allows the certification module (M6) to aggregate findings from heterogeneous modules into a single, coherent, traceable certification result.

## 4.17 Security Controls Catalogue and Standards Cross-Reference

### *4.17.1 Control Framework Approach*

The framework is itself a security-critical system handling highly sensitive assets, and is therefore engineered to a defined set of controls. This section catalogues those controls by area and maps them to the control themes of ISO/IEC 27001:2022 and to the functions of the NIST AI Risk Management Framework. The firm claims alignment with these references, not certification against them (Section 14.1).

### *4.17.2 Controls Catalogue*

**Table 24. Security controls catalogue for the assessment framework**

| Control area | Control | Implementation in the framework |
|---|---|---|
| Containment | Physical air-gap; no WAN uplink | The entire environment operates without outbound connectivity; egress is verified nil (Sections 4.12, 4.11). |
| Controlled import | One-way data import | Artefacts enter only through a one-way station with integrity verification; no return path exists. |
| Network | Default-deny segmentation | Trust zones separated by default-deny firewalls; only allow-listed, mutually authenticated flows (Section 4.12.2). |
| Host | Baseline hardening | Minimal, CIS-aligned host images; kernel hardening; integrity monitoring. |
| Isolation | Sandboxed execution | Untrusted artefacts run under micro-VM isolation with restricted syscalls and read-only file systems. |
| Identity | Least-privilege RBAC | Role-based access with multi-factor authentication and short-lived credentials; no shared accounts. |
| Secrets | Centralised secret management | Secrets held in a vault, never in code or images; scoped, rotated, and audited. |
| Integrity | Signed jobs and artefacts | Internal jobs are signed; artefact integrity is hashed and verified before use. |
| Supply chain | Vendored, verified dependencies | All dependencies are mirrored offline with integrity verification and SBOM (Section 4.8.3). |
| Data protection | Encryption and key management | Sensitive data encrypted at rest and in transit within the perimeter; keys managed under policy. |
| Evidence | Tamper-evident audit (WORM) | All actions and findings are recorded in a write-once audit log for chain of custody. |
| Monitoring | Centralised logging and alerting | Logs aggregated to a SIEM within the perimeter; anomalous activity alerts the operator. |
| Disposal | Policy-based crypto-erase | Sensitive material is cryptographically erased per the retention and disposal policy (Section 4.9.2). |

Source: Security architecture (Section 4.11) and deployment design (Section 4.12).

*Interpretation: The controls implement defence-in-depth around the sensitive assets, with the air-gap and one-way import as the outermost containment guarantee.*

### *4.17.3 ISO/IEC 27001:2022 Theme Cross-Reference*

The controls above align with the four control themes of ISO/IEC 27001:2022 Annex A, as cross-referenced below. The mapping is indicative and is offered to show the breadth of alignment, not as a certification claim.

**Table 25. Indicative alignment with ISO/IEC 27001:2022 control themes**

| ISO/IEC 27001:2022 theme | Representative alignment in the framework |
|---|---|
| Organisational controls | Access policy, supplier / supply-chain management, information classification, and documented operating procedures. |
| People controls | Least-privilege roles, authentication, and responsibilities defined for operators and engineers. |
| Physical controls | Operation within a controlled, air-gapped facility; controlled media and one-way import. |
| Technological controls | Cryptography and key management, logging and monitoring, secure configuration, segmentation, malware defence, and secure development. |

Source: ISO/IEC 27001:2022 Annex A control themes.

*Interpretation: The framework's controls span all four control themes of the standard, evidencing a comprehensive, recognised security posture appropriate to the sensitivity of the work.*

### *4.17.4 NIST AI RMF Function Mapping*

The framework supports the four functions of the NIST AI Risk Management Framework, both in how it is built and in what it produces for the End-User.

**Table 26. Mapping of the framework to NIST AI RMF functions**

| NIST AI RMF function | How the framework supports it |
|---|---|
| Govern | Documented controls, roles, and evidence retention establish governance over the assessment process and its outputs. |
| Map | The threat taxonomy and OWASP / ATLAS tagging map the risks applicable to a given target. |
| Measure | The figures of merit and metrics (Section 4.15) measure the target's susceptibility quantitatively. |
| Manage | Severity-scored findings with remediation guidance enable the End-User to prioritise and manage the risks identified. |

Source: NIST AI Risk Management Framework 1.0.

*Interpretation: The framework's outputs are designed to feed directly into the End-User's own AI-risk governance under the NIST functions, maximising the utility of the assessment evidence.*

## 4.18 Assumptions, Constraints, Dependencies and Interfaces

### *4.18.1 Assumptions*

The proposal is made on the following principal assumptions, stated so that they can be confirmed or adjusted at the RFP stage.

- A secure, air-gapped facility with adequate GPU compute is available for development and, ultimately, for accredited operation, whether provided by the firm for the proof-of-concept or by the End-User for deployment.
- Reference open-weight models and the right to use them for development and validation are available and may be held offline.
- The End-User will define the specific defence-relevant harm categories and any classified test requirements at the appropriate stage, within an appropriately cleared environment.
- The scope of the proof-of-concept is the demonstration of capability against reference models and the delivery of the approximately one hundred validated test cases, not the certification of any specific production model.

### *4.18.2 Constraints*

The development operates under the following constraints, which the design explicitly respects.

- No external or cloud connectivity is available or permitted; all components must operate fully offline (Section 4.8.3).
- No sensitive data — weights, corpora, prompts, findings — may leave the secure perimeter at any time (undertaking 15a).
- The development completes within twenty-one months, inside the twenty-four-month ceiling (Section 5.1).
- No project cost, price or budget is stated anywhere in this Expression of Interest (EoI clause 4.2.1).
- Discovered vulnerabilities may not be disclosed or transferred without End-User authorisation (undertaking 15b).

### *4.18.3 External Dependencies*

The framework depends only on general-purpose, open-source building blocks, all of which are vendored and held offline within the perimeter. The principal dependency categories and the basis on which each is managed are below.

**Table 27. External dependency categories and management basis**

| Dependency category | Examples (general-purpose) | Management basis |
|---|---|---|
| ML frameworks | Deep-learning and model-handling libraries | Vendored offline; integrity-verified; pinned versions. |
| Model serving | Open-source inference servers | Vendored offline; run within the target enclave. |
| Attack methods | Open implementations of recognised attack techniques | Adapted and extended by the firm; held in the code baseline. |
| Platform / runtime | Container and orchestration runtimes | Vendored offline; hardened configuration. |
| Data / persistence | Relational store, cache, object store, search | Vendored offline; deployed within the perimeter. |
| Reference models | Open-weight models for development / validation | Held offline; used for testing only; never transmitted out. |

Source: Technology stack (Section 4.8) and offline dependency discipline (Section 4.8.3).

*Interpretation: No dependency requires connectivity at build or run time and none is foreign-controlled in operation; the offline mirror removes external dependency entirely.*

### *4.18.4 Interface Contracts*

The framework's internal coherence rests on a small number of stable data contracts. The two most important — the finding schema and the certification-report schema — are specified below at the level of their principal fields. These contracts allow modules to interoperate and allow the End-User to consume machine-readable output.

#### Finding Schema (principal fields)

**Table 28. Finding schema — principal fields**

| Field | Description |
|---|---|
| finding_id | Unique identifier for the finding. |
| module | Module of origin (M1–M6). |
| category | Vulnerability category and harm type. |
| owasp_ref | OWASP LLM Top 10 reference (LLM01–LLM10). |
| atlas_technique | MITRE ATLAS technique tag. |
| severity | Severity score and band. |
| evidence_ref | Reference(s) to stored evidence (sanitised). |
| language | Language / locale of the eliciting input, where relevant. |
| reproducible | Whether the finding reproduces on re-execution with the recorded seed. |
| recommendation | Remediation guidance for the finding. |

Source: Module design (Section 4.6.7) and scoring module (M6).

*Interpretation: A single, consistent finding schema across all modules is what makes the aggregate certification result coherent and machine-consumable.*

#### Certification-Report Schema (principal sections)

**Table 29. Certification-report schema — principal sections**

| Section | Content |
|---|---|
| Target identification | Artefact hash, format, provenance, and profile of the assessed model. |
| Scope and method | Test profile executed, modules run, and datasets used. |
| Findings | Severity-scored findings with OWASP / ATLAS tags and evidence references. |
| Figures of merit | Measured values against the EoI targets (Section 4.15.4). |
| Risk posture | Overall verdict and prioritised remediation guidance. |
| Evidence package | References to the reproducible evidence and the machine-readable findings file. |

Source: Scoring and reporting module (M6).

*Interpretation: The report structure gives the accrediting authority a complete, auditable basis for a deployment decision.*

### *4.18.5 Consolidated Exit Criteria*

The programme is complete, and the proof-of-concept accepted, when the following consolidated criteria are met: the six modules are integrated and operate end-to-end within the air-gapped environment; the figures of merit meet their targets, or are reported honestly with their measured interim values and intervals where a target is closely approached; the approximately one hundred validated test cases are delivered; the certification report and machine-readable findings are produced and validate against their schemas; egress from the perimeter is demonstrated nil throughout; and the documentation and evidence package are complete and auditable. These criteria operationalise the EoI's nature-of-deliverables clause and provide an unambiguous definition of success for the subsequent RFP stage.

## 4.19 Comparative Analysis of Existing Approaches and Differentiators

### *4.19.1 Purpose and Method of Comparison*

A central requirement of a credible technical proposal is to set the proposed solution against the alternatives that an organisation might otherwise adopt, to state plainly the weaknesses of those alternatives, and to show how the proposed solution addresses them. This section does so. It considers the four classes of approach that a defence organisation realistically faces when it needs to assess the security of a large language model: foreign-controlled cloud red-teaming services; open-source single-purpose tools; manual expert red-teaming; and conventional application-security scanners repurposed for AI. For each, it identifies the principal limitations and then demonstrates the corresponding capability of the proposed framework. The comparison is made on capability and suitability grounds only; consistent with the EoI, no cost comparison of any kind is made.

### *4.19.2 Limitations of Existing Approaches*

#### 24.2.1 Foreign-controlled cloud red-teaming services

Commercial cloud services for LLM red-teaming require the target model, or at minimum its inference endpoint and the test prompts, to be exposed to externally controlled infrastructure. For a sovereign or classified model this is disqualifying: it breaches data sovereignty, exposes model weights and intellectual property to a foreign-controlled environment, relies on opaque telemetry and logging that the operator cannot inspect, and offers no defence accreditation. Such services are also overwhelmingly English-centric and carry no Indian-language or India-specific harm coverage. They are, in short, unusable for the use case the EoI describes.

#### 24.2.2 Open-source single-purpose tools

A number of valuable open-source tools exist for individual slices of the problem — for example, a library that implements one class of adversarial attack, or a scanner for one class of artefact risk. While useful, they are fragmentary: they cover one technique or one risk rather than the full OWASP LLM Top 10; they are not integrated into a single workflow with consistent findings and scoring; they rarely produce certification-grade, standards-mapped evidence; they are not packaged for hardened, air-gapped operation; and they require substantial expert effort to assemble, operate and interpret. An organisation that relies on them inherits the burden of integration, consistency and accreditation itself.

#### 24.2.3 Manual expert red-teaming

Skilled manual red-teaming is effective and remains an important complement, but on its own it does not scale, is difficult to reproduce, varies with the individual tester, and cannot economically achieve the breadth of coverage — dozens of techniques across multiple languages and ten or more harm categories — that the EoI specifies. Its findings are typically narrative rather than machine-readable and standards-mapped, which complicates accreditation and re-testing.

#### 24.2.4 Conventional application-security scanners

Traditional application-security and vulnerability scanners are designed for conventional software and network surfaces. They do not model the failure modes that are specific to language models — prompt injection, jailbreaks, membership inference, embedding-space weaknesses, model poisoning — and therefore provide little assurance for an LLM. Repurposing them gives a false sense of coverage while leaving the AI-specific attack surface largely untested.

### *4.19.3 Comparative Capability Matrix*

The matrix below summarises, across the dimensions that matter for the EoI's use case, how the four classes of existing approach compare with the proposed framework. The entries are deliberately conservative.

**Table 30. Comparative capability matrix — existing approaches versus the proposed framework**

| Dimension | Foreign cloud service | Open-source point tools | Manual red-team | Proposed framework |
|---|---|---|---|---|
| Air-gapped / sovereign operation | No | Partial (effort) | Possible | Yes — by design |
| Data sovereignty preserved | No | Yes | Yes | Yes |
| Full OWASP LLM Top 10 coverage | Partial | Fragmentary | Depends on tester | Yes (Section 4.14) |
| Adversarial / gradient attack generation | Partial | Partial | Limited | Yes (M3) |
| Agentic multi-turn red-teaming | Partial | Rare | Manual | Yes (M4) |
| Privacy / membership-inference audit | Rare | Partial | Hard | Yes (M5) |
| Artefact / supply-chain scanning | Rare | Partial | Hard | Yes (M2) |
| Indian-language & local harm coverage | No | No | Limited | Yes (M3, M4) |
| Reproducible, automated, scalable | Partial | Partial | No | Yes |
| Standards-mapped certification output | Partial | Rare | Rare | Yes (M6) |
| Indigenous / accreditable | No | Partial | Partial | Yes |

Source: Analysis of approach classes against the EoI requirements (Sections 4.6, 4.7, 4.14).

*Interpretation: No existing class of approach satisfies the combination of sovereign air-gapped operation, full OWASP coverage, Indian-language breadth, and certification-grade output; the proposed framework is designed precisely to close this gap.*

### *4.19.4 Differentiators of the Proposed Framework*

Drawing the comparison together, the framework's differentiators are as follows. Each responds directly to a weakness identified above.

1. **Sovereign, air-gapped by design.** The framework operates entirely within the defence-controlled perimeter with zero egress, removing the data-sovereignty and exfiltration risks that disqualify foreign cloud services (addresses 24.2.1).
2. **Comprehensive and integrated.** Six modules cover the full OWASP LLM Top 10 within a single workflow with consistent findings and scoring, replacing the fragmentary assembly of point tools (addresses 24.2.2).
3. **Automated, reproducible and scalable.** Attack generation and evaluation are automated and seeded for reproducibility, achieving breadth that manual red-teaming cannot economically reach (addresses 24.2.3).

4. **AI-specific.** The framework models the failure modes specific to language models, which conventional scanners do not (addresses 24.2.4).
5. **Indian-context coverage.** Indian-language and India-specific harm coverage extends the attack surface beyond the English-only reach of foreign and most open-source tools.
6. **Certification-grade output.** Severity-scored, standards-mapped, machine-readable evidence supports defence accreditation and re-testing.
7. **Indigenous and accreditable.** The design and intellectual property are indigenous and the system is built to be accredited for defence use.

## 4.20 Proof-of-Concept Definition and Test-Case Taxonomy

### *4.20.1 Scope of the Proof-of-Concept*

The proof-of-concept demonstrates the integrated framework operating end-to-end within the air-gapped environment against reference open-weight models, and delivers a validated body of approximately one hundred test cases spanning the vulnerability classes in scope. Its purpose is to evidence capability and the figures of merit (Section 4.15.4), not to certify any specific production model. The proof-of-concept is the gate between this Expression of Interest and the subsequent development under the Technology Development Fund.

### *4.20.2 Test-Case Taxonomy and Indicative Distribution*

The validated test cases are organised by vulnerability class and are distributed so as to give meaningful coverage of each class while concentrating on the highest-priority risks for defence use. The indicative distribution below totals approximately one hundred cases; the final distribution will be confirmed with the End-User. Test cases describe method and expected outcome and do not, in this document, contain operational exploit content.

**Table 31. Indicative test-case taxonomy and distribution (~100 cases)**

| Vulnerability class | OWASP ref | Module | Indicative cases |
|---|---|---|---|
| Prompt injection (direct / indirect) | LLM01 | M3, M4 | ~20 |
| Jailbreak / safety bypass | LLM01 / LLM05 | M3, M4 | ~18 |
| Sensitive-information disclosure | LLM02 | M5 | ~12 |
| Membership inference / extraction | LLM02 | M5 | ~10 |
| Supply-chain / artefact integrity | LLM03 | M2 | ~10 |
| Data / model poisoning & backdoors | LLM04 | M2, M5 | ~8 |
| Excessive agency / tool abuse | LLM06 | M4, M5 | ~6 |
| System-prompt leakage | LLM07 | M3, M4 | ~6 |
| Embedding / vector weaknesses | LLM08 | M5 | ~5 |
| Misinformation (incl. India context) | LLM09 | M4 | ~5 |
| Unbounded consumption / DoS | LLM10 | M5 | ~4 |
| Multilingual variants (cross-cutting) | LLM01–LLM09 | M3, M4 | Distributed across the above |

Source: EoI proof-of-concept requirement and the module design (Section 4.6).

*Interpretation: The distribution gives coverage across every vulnerability class while weighting the highest-priority risks, and embeds multilingual variants across classes rather than treating language as a separate silo.*

### *4.20.3 Test-Case Structure*

Each validated test case is recorded in a consistent structure so that it is reproducible, reviewable, and mappable to the standards. The principal elements of a test case are below.

**Table 32. Structure of a validated test case**

| Element | Description |
|---|---|
| Identifier | Unique test-case identifier. |

| Element | Description |
|---|---|
| Vulnerability class & OWASP / ATLAS reference | The class exercised and its standard tags. |
| Objective | The adversarial objective the case attempts. |
| Method | The technique and, where relevant, the generation or mutation approach. |
| Language / locale | The language exercised, where relevant. |
| Expected outcome | The behaviour that constitutes a pass or a finding. |
| Evidence | What is captured to substantiate the result (sanitised). |
| Reproducibility | The seed / configuration enabling re-execution. |

Source: Evaluation methodology (Section 4.15) and finding schema (Section 4.18.4.1).

*Interpretation: The consistent structure makes the proof-of-concept body of test cases auditable and directly reusable in subsequent development and operation.*

### *4.20.4 Proof-of-Concept Success Definition*

The proof-of-concept is judged successful when: the six modules operate together end-to-end within the air-gapped environment; the approximately one hundred validated test cases are delivered and execute reproducibly; the figures of merit meet their targets, or are reported honestly with measured interim values and intervals where a target is closely approached (Section 4.15.4); a certification report and machine-readable findings are produced and validate against their schemas; and egress from the perimeter is demonstrated nil throughout. This is the same definition of success consolidated in the exit criteria of Section 4.18.5, expressed specifically for the proof-of-concept.

## 4.21 Deployment, Operations, Training and Handover

### *4.21.1 Deployment into an Accredited Environment*

The framework is packaged for deployment into a hardened, air-gapped environment of the kind described in Section 4.12. Deployment proceeds by establishing the trust zones and the one-way import station, provisioning the control and services cluster and the GPU fabric, instantiating the target-under-test enclave, and bringing up the identity, secrets, evidence and monitoring services. All software is installed from the offline mirror with integrity verification; nothing is fetched from any external network at any point. Deployment concludes with a verification that egress is nil and that the figures-of-merit harness reproduces expected results on a reference model.

### *4.21.2 Operations and Maintenance*

In operation, an assessment is initiated by an authorised operator through the console; the orchestrator dispatches the test profile as signed jobs; the modules execute against the target within the enclave; and the certification report and machine-readable findings are produced. Routine maintenance — the application of vetted updates, the refresh of attack corpora and signatures, and the rotation of credentials — is performed through the same controlled, offline import path used for the initial deployment, so that the air-gap is never compromised by maintenance. Operational health is observed through the internal monitoring and SIEM services, and all activity is recorded in the tamper-evident audit log.

### *4.21.3 Training and Knowledge Transfer*

The firm will provide structured training and knowledge transfer so that the End-User's personnel can operate, interpret and maintain the framework. The training addresses three audiences: operators, who run assessments and interpret reports; administrators, who maintain the environment, identities and corpora; and technical reviewers, who need to understand the methods and the standards mapping in order to rely on the certification output. Training is delivered through documentation, hands-on sessions within the environment, and reference runbooks, all provided within the secure perimeter.

**Table 33. Training and knowledge-transfer plan**

| Audience | Knowledge transferred | Form |
|---|---|---|
| Operators | Initiating assessments; interpreting findings, severity and reports. | Hands-on sessions; operator runbook. |
| Administrators | Environment, identity, secrets, corpora and update management. | Hands-on sessions; administration runbook. |
| Technical reviewers | Methods, metrics and standards mapping underpinning the evidence. | Technical briefings; design and methodology documentation. |

Source: Programme documentation and handover plan (WBS work package 6, Section 4.13).

*Interpretation: Training is matched to each audience's role so that the End-User can operate and rely on the framework independently after handover.*

### *4.21.4 Documentation and Handover Artefacts*

At handover the firm delivers a complete documentation set so that the framework can be operated, maintained and — where required — independently verified without continuing dependence on the firm. The principal artefacts are listed below.

- Architecture and detailed-design documentation, including the data, security and deployment architectures.
- Operator, administration and maintenance runbooks.
- The validated test-case library and the attack corpora, within the secure perimeter.
- The methodology and metrics documentation, and the standards-mapping tables (OWASP, ATLAS, NIST).
- The evidence and audit configuration, and the schemas for findings and reports.
- A maintenance and update procedure consistent with air-gapped operation.

### *4.21.5 Post-Handover Support and Sustainment*

The framework is designed to be sustained by the End-User after handover, with the firm available to provide further development, corpus and signature refreshes, and technical support as separately agreed. Because the design and intellectual property are indigenous (Section 8.1), the End-User is not exposed to foreign-vendor lock-in or to the withdrawal of an external service, and the framework can evolve under national control as the threat landscape changes.

## 4.22 AI Capability Overview and Project Description

The preceding parts of this Expression of Interest describe the framework as a sovereign, air-gapped capability for assessing the security posture of large language models. This part describes the artificial-intelligence core that powers that capability and, in doing so, addresses each specific deliverable item set out in the EoI for the Technical Proposal and the Development Plan. It is presented as a discrete, self-contained part so that an evaluator can map every required item to a numbered section and figure.

### *4.22.1 Restatement of the Problem in AI Terms*

Modern software and firmware are produced and procured at a scale and velocity that manual security review cannot match, and the defensive task spans two distinct artefact types: source code that is available for inspection (Type-I), and binaries or suspected malware for which no source is available (Type-II). Conventional rule-based scanners produce high false-positive rates, miss logic and context-dependent weaknesses, and cannot reason about a binary the way a human reverse engineer can. The problem is therefore to build an AI capability that can read code and binaries with contextual understanding, correlate what it sees against the body of known weaknesses and exploits, reason in multiple steps to confirm whether a weakness is real, and — where the source is available — propose a validated fix, all while operating entirely within a sovereign, air-gapped environment.

### *4.22.2 The Proposed Solution in Brief*

The proposed solution is an AI-powered vulnerability detection, analysis and remediation capability built on four pillars: a transformer-based cybersecurity language model, fine-tuned on a curated security corpus; a retrieval-augmented generation (RAG) layer that grounds the model in an authoritative, continuously curated vulnerability knowledge base; an agentic reasoning layer that performs multi-step analysis and constructs verifiable proof-of-concept chains; and a remediation layer that generates and validates secure code patches for human review. The capability ingests Type-I source code and Type-II binaries, analyses both within the air-gapped perimeter, and produces severity-scored findings, reproducible proof-of-concept evidence, and — for source code — candidate patches with a documented fix rationale. The end-to-end flow is shown in the architecture of Section 4.24.

### *4.22.3 Relationship to the Wider Framework*

This AI core is the analytical engine of the wider Comprehensive Vulnerability Assessment Framework. The same fine-tuned model, RAG layer and agentic reasoning that detect weaknesses in software and binaries also underpin the assessment of language-model systems described in Sections 4.4 to 4.21: the adversarial generation, the agentic red-teaming, and the certification reporting all draw on the same AI foundations and the same air-gapped operating discipline. The framework is therefore a single, coherent capability with two complementary applications — assessing AI systems, and using AI to assess and remediate software and binaries.

### *4.22.4 Mapping of EoI Deliverables to Sections*

The table below maps each deliverable item required by the EoI to the section of this document that addresses it, so that compliance can be verified item by item.

**Table 34. Mapping of EoI-specified deliverable items to the sections that address them**

| EoI deliverable item | Addressed in |
|---|---|
| Project Description | Section 4.22 |

| EoI deliverable item | Addressed in |
|---|---|
| Work Breakdown Structure (WBS) | Sections 4.13 and 4.23 |
| Architecture / Block Diagram | Sections 4.5 and 4.24 |
| Technology Stack | Sections 4.8 and 4.25 |
| Transformer-based LLM Approach | Section 4.26 |
| Supervised Fine-Tuning / Instruction Tuning | Section 4.27 |
| Cybersecurity Corpus Curation | Section 4.28 |
| RAG Design | Section 4.29 |
| Agentic AI Framework | Sections 4.6 and 4.30 |
| Air-Gapped Deployment Strategy | Sections 4.11, 4.12 and 4.31 |
| Vulnerability Detection & Evaluation Plan | Section 4.32 |
| Malware Analysis / Reverse Engineering | Section 4.33 |
| Secure Code Patching Approach | Section 4.34 |
| Development Timeline (≤ 21 months) | Sections 5.1 and 5.2 |
| Milestone-wise Deliverables | Sections 5.1 and 5.2 |
| Suo-moto Similar Product / Technology | Sections 14.1 and 5.2 |
| Proof-of-Concept Plan (Type-I & Type-II) | Sections 4.20 and 4.35 |
| Testing and Qualification Plan | Sections 4.7, 4.15 and 4.36 |

Source: EoI deliverable checklist (Technical Proposal and Development Plan) reconciled against this document.

*Interpretation: Every deliverable item specified by the EoI is addressed by an identifiable section; items with both a general and an AI-specific treatment are cross-referenced to both.*

## 4.23 Work Breakdown Structure of the AI Capability

Section 4.13 presents the programme-level work breakdown structure. This section decomposes the AI capability specifically into its constituent modules, subsystems, software components and supporting hardware, so that the engineering scope of the AI core is explicit and estimable.

### *4.23.1 Decomposition into Modules and Subsystems*

**Table 35. Work breakdown of the AI capability into modules, subsystems and principal elements**

| WBS | Module / subsystem | Principal software / hardware elements |
|---|---|---|
| A1 | Data & corpus subsystem | Ingestion connectors; cleaning and labelling pipeline; relational schema store; object store. |
| A2 | Model subsystem | Base model; domain-adaptive pre-training; SFT and instruction-tuning harness; evaluation harness; GPU training nodes. |
| A3 | Retrieval (RAG) subsystem | Embedding service; vector database; reranker; correlation service. |
| A4 | Agentic reasoning subsystem | Planner–executor orchestrator; tool adapters; PoC-chain and verifier services. |
| A5 | Code-analysis subsystem (Type-I) | Source parsers; static-analysis adapters; detection and ranking services. |
| A6 | Binary-analysis subsystem (Type-II) | Disassembly / lifting adapters; feature extraction; isolated sandbox. |
| A7 | Remediation subsystem | Patch-generation service; build/test/regression validation; fix-rationale generation. |
| A8 | Serving & inference | Inference servers; quantisation; GPU scheduling; batching. |
| A9 | Application & API | Analyst console; REST/gRPC API; RBAC; report and evidence export. |
| A10 | Platform & security | Containerisation; orchestration; secrets; audit logging; one-way import; monitoring. |

Source: Decomposition of the AI core consistent with the architecture of Section 4.24 and the technology stack of Section 4.25.

*Interpretation: The AI capability decomposes into ten subsystems whose software and hardware elements are individually identifiable, supporting estimation, staffing and milestone planning.*

### *4.23.2 Hardware Elements*

The principal hardware elements are GPU training and inference nodes sized for a model in the 30 to 70 billion parameter range, a control-and-services cluster, encrypted storage for corpora and evidence, and a physically separated one-way import station. The detailed environment and topology are described in Sections 4.11 and 12; consistent with the EoI, no cost or price for any element is stated.

## 4.24 Solution Architecture and End-to-End Block Diagram

This section presents the end-to-end architecture of the AI capability and explains the flow of an artefact from intake to findings, evidence and remediation. It complements the logical system architecture of Section 4.5, which covers the assessment framework as a whole.

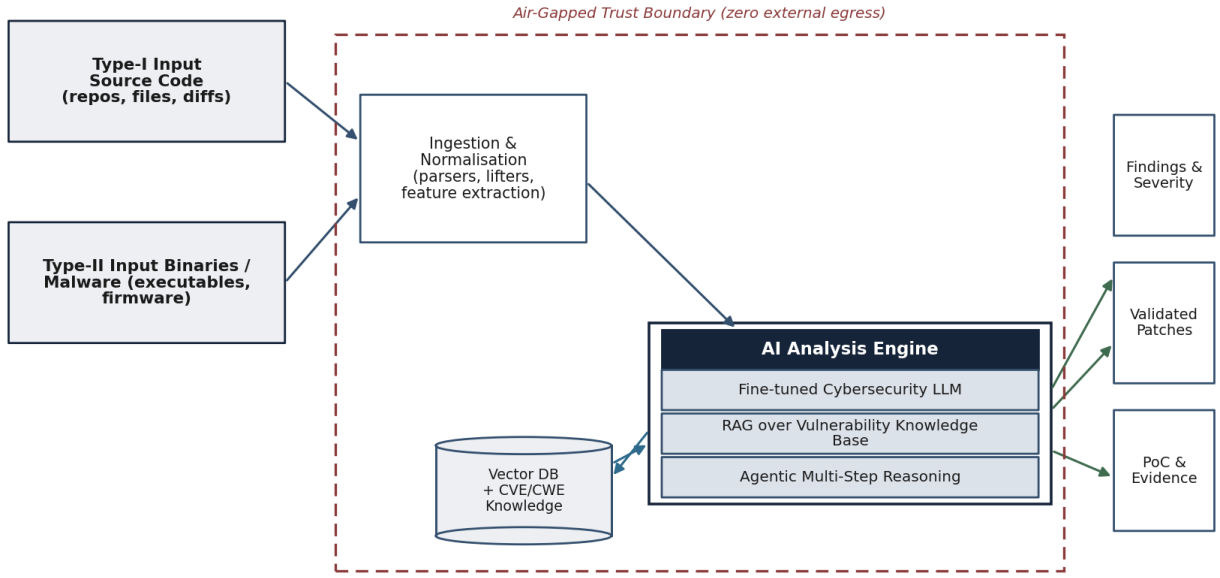



**Figure 27. End-to-end block diagram of the AI vulnerability detection and remediation capability**

*Type-I source code and Type-II binaries enter through ingestion and normalisation; the AI analysis engine (fine-tuned model, RAG over the vulnerability knowledge base, and agentic reasoning) analyses the artefact and produces findings with severity, validated patches, and proof-of-concept evidence. The entire pipeline operates within the air-gapped trust boundary with no external egress.*

### *4.24.1 End-to-End Flow*

An assessment begins when an authorised analyst submits an artefact — a source-code repository or file (Type-I), or a binary or suspected-malware sample (Type-II) — through the console. The ingestion layer normalises the artefact: source code is parsed into syntax trees and intermediate representations, while binaries are triaged, unpacked where possible, and lifted to disassembly and control-flow representations. The normalised artefact is passed to the AI analysis engine, where the fine-tuned model, grounded by retrieved knowledge and directed by the agentic reasoning layer, identifies candidate weaknesses, correlates them against known vulnerabilities, and confirms them through multi-step analysis. Confirmed findings are scored for severity; for Type-I artefacts a candidate patch is generated and validated; and a reproducible proof-of-concept and evidence package is assembled. All outputs are written to the tamper-evident evidence store and surfaced to the analyst for review.

### *4.24.2 Architectural Properties*

The architecture is designed for four properties. It is contained: every component runs inside the air-gapped perimeter, and no data, model weight or telemetry leaves it. It is grounded: the model is always supplied with retrieved, authoritative context, which reduces hallucination and supports correlation. It is auditable: every step, input and output is recorded so that a finding or a patch can be traced and independently reviewed. And it is human-supervised: the framework proposes findings and patches for analyst confirmation and never autonomously modifies an operational system.

## 4.25 Technology Stack

This section specifies the technology stack of the AI capability across its layers: the language model, the retrieval layer, the agentic orchestration, the data and corpus layer, model serving, and the platform and DevSecOps foundation. It complements the framework-level stack of Section 4.8 and lists only mature, well-supported components suitable for hardened, offline operation.

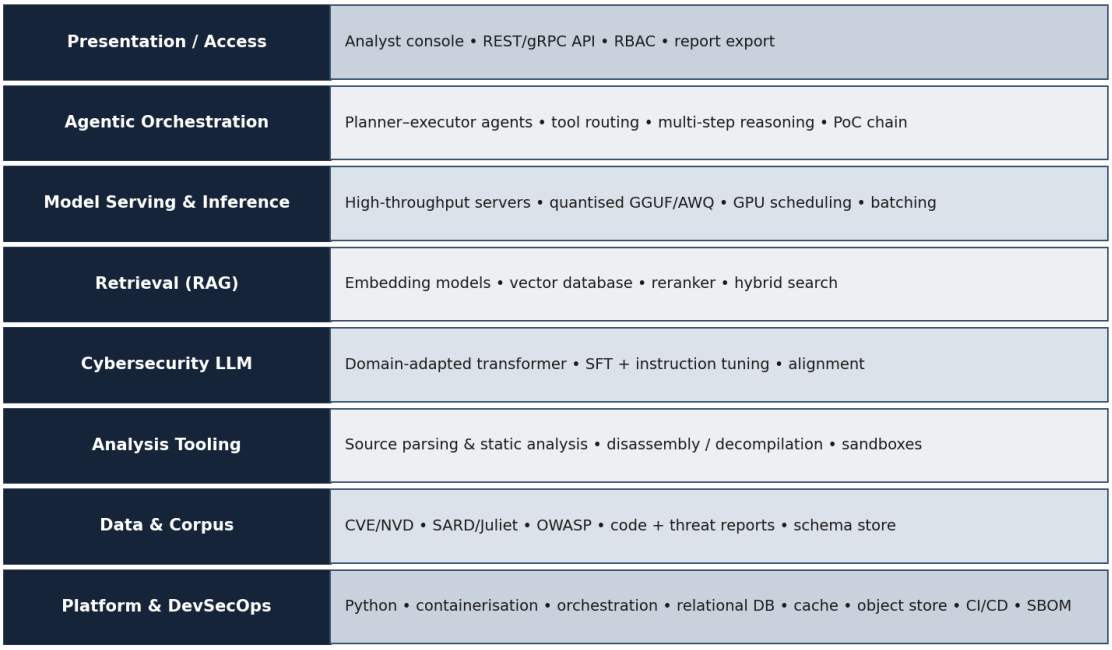



**Figure 28. Layered technology stack of the AI capability**

*The stack is organised from the platform and DevSecOps foundation upward through data and corpus, the cybersecurity language model, retrieval, model serving, agentic orchestration, and the analyst access layer. Each layer names representative, widely used components that can be operated entirely offline.*

### *4.25.1 Component Selection and Rationale*

**Table 36. AI technology stack — representative components and selection rationale**

| Layer | Representative components | Rationale |
|---|---|---|
| Language model | Open-weight transformer (30–70B), domain-adapted | Permissive licensing; on-premises fine-tuning; no dependence on external model APIs. |
| Retrieval (RAG) | Embedding model; vector database; reranker | Mature, offline-capable vector search; grounding and correlation. |
| Agentic orchestration | Planner–executor framework; tool adapters | Multi-step reasoning; controlled tool use; PoC-chain construction. |
| Model serving | Inference servers; quantised GGUF/AWQ | High-throughput inference; fits available GPU memory; deterministic batching. |

| Layer | Representative components | Rationale |
|---|---|---|
| Analysis tooling | Source parsing and static analysis; disassembly / decompilation | Established tooling for source and binary analysis. |
| Data & corpus | Relational database; object store; schema store | Reliable structured storage and object storage for corpora and evidence. |
| Platform / DevSecOps | Python; containerisation; orchestration; cache; CI/CD; SBOM | Standard, supportable, hardened deployment; supply-chain integrity. |

Source: Technology selection for the AI capability (this section) and the deployment constraints of Sections 4.11 and 4.12.

*Interpretation: Each layer is realised with mature, offline-capable components chosen for licensing freedom, supportability and suitability for air-gapped operation.*

## 4.26 Transformer-Based Language-Model Approach

This section describes the design and training approach for the cybersecurity language model at the heart of the capability, demonstrating the methodology by which a general open-weight transformer is adapted into a security-specialised model able to reason about code and binaries.

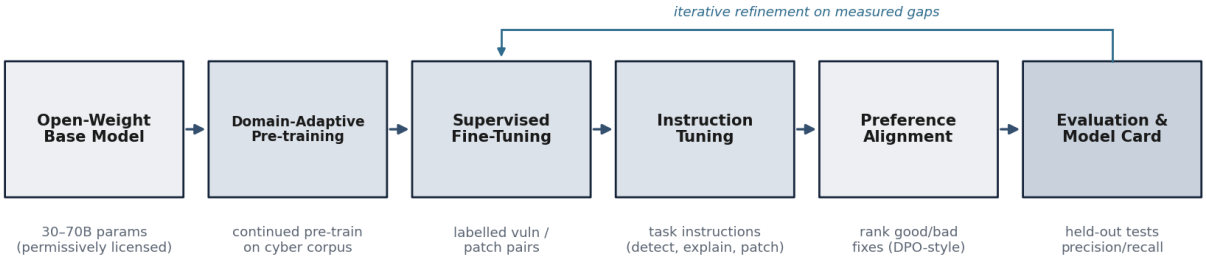



**Figure 29. Model design and training / fine-tuning pipeline**

*A permissively licensed open-weight base model undergoes domain-adaptive pre-training on the security corpus, supervised fine-tuning on labelled vulnerability and patch pairs, instruction tuning for the detect/explain/patch tasks, and preference alignment, with a held-out evaluation stage that feeds measured gaps back into fine-tuning and is recorded in a model card.*

### *4.26.1 Base-Model Selection*

The approach begins from a permissively licensed open-weight transformer model in the 30 to 70 billion parameter range. An open-weight base is essential: it can be fine-tuned and served entirely on-premises, it carries no dependence on an external model API, and its weights can be held within the air-gapped perimeter. Model size in this range balances reasoning capability against the memory and throughput achievable on a practical on-premises GPU fabric, particularly once quantised for inference.

### *4.26.2 Domain-Adaptive Pre-training*

The base model is further pre-trained on the curated cybersecurity corpus (Section 4.28) so that it internalises the vocabulary, idioms and patterns of vulnerable and secure code, of disassembly, and of vulnerability advisories. This domain adaptation materially improves downstream performance relative to fine-tuning a general model directly, because the model begins the supervised stage already fluent in the domain.

### *4.26.3 Parameter-Efficient Fine-Tuning*

Supervised fine-tuning and instruction tuning (Section 4.27) are performed using parameter-efficient methods such as low-rank adaptation, which update a small set of adapter parameters rather than the full model. This makes training tractable on the available hardware, allows multiple task-specific adapters to be maintained and versioned, and keeps the training footprint and reproducibility manageable within the air-gapped environment.

### *4.26.4 Demonstrated Methodology*

The methodology set out here reflects established, published practice for adapting transformer models to specialised technical domains: continued pre-training for domain adaptation, supervised fine-tuning on task-labelled data, instruction tuning for task generalisation, and preference alignment for output quality. The firm proposes to evidence this capability during the proof-of-concept (Section 4.35) by delivering a fine-tuned model with a documented model card and measured detection performance, rather than by asserting unverified prior results.

## 4.27 Supervised Fine-Tuning and Instruction Tuning

This section details the supervised fine-tuning and instruction tuning that specialise the model for vulnerability detection, explanation and patching, and describes how the resulting model is evaluated and documented.

### *4.27.1 Training Tasks and Data*

Supervised fine-tuning uses labelled examples drawn from the curated corpus: vulnerable and non-vulnerable code samples with their weakness labels (mapped to CWE), advisory text linked to affected code, and vulnerable-to-fixed patch pairs with a fix rationale. Instruction tuning casts these as explicit tasks — “identify and locate the weakness”, “explain why this code is vulnerable”, “propose a secure patch and justify it”, and “classify this binary’s behaviour” — so that the model generalises across the operations the framework requires.

### *4.27.2 Evaluation and Model Card*

Each fine-tuned model is evaluated on held-out data and the results are recorded in a model card. The card documents the base model, the training data provenance and governance, the tasks, the evaluation datasets and metrics, known limitations, and the intended use. The indicative model-card structure is below.

**Table 37. Indicative model-card structure for a fine-tuned model**

| Model-card field | Content |
|---|---|
| Base model | Open-weight transformer, parameter count, licence. |
| Adaptation | Domain-adaptive pre-training; PEFT method and configuration. |
| Training data | Corpus version, sources, labelling and governance (Section 4.28). |
| Tasks | Detect / locate; explain; patch; classify binary. |
| Evaluation | Datasets (OWASP Benchmark, SARD, Juliet, CVE patch set) and metrics (precision, recall, F1). |
| Limitations | Known failure modes; out-of-scope artefact types; boundary of assurance. |
| Intended use | Analyst-supervised assessment within the air-gapped environment. |
| Version & provenance | Model version, training run identifier, reproducibility seed. |

Source: Fine-tuning and evaluation methodology (this section) and the evaluation plan (Section 4.32).

*Interpretation: The model card makes each fine-tuned model transparent, evaluable and reproducible, which is essential for evidence that an accrediting authority can rely upon.*

## 4.28 Cybersecurity Corpus Curation

The quality of the model depends on the quality of its training corpus. This section explains how the cybersecurity corpus is assembled from authoritative sources, cleaned and labelled, governed for sensitive content, and stored against a defined schema.

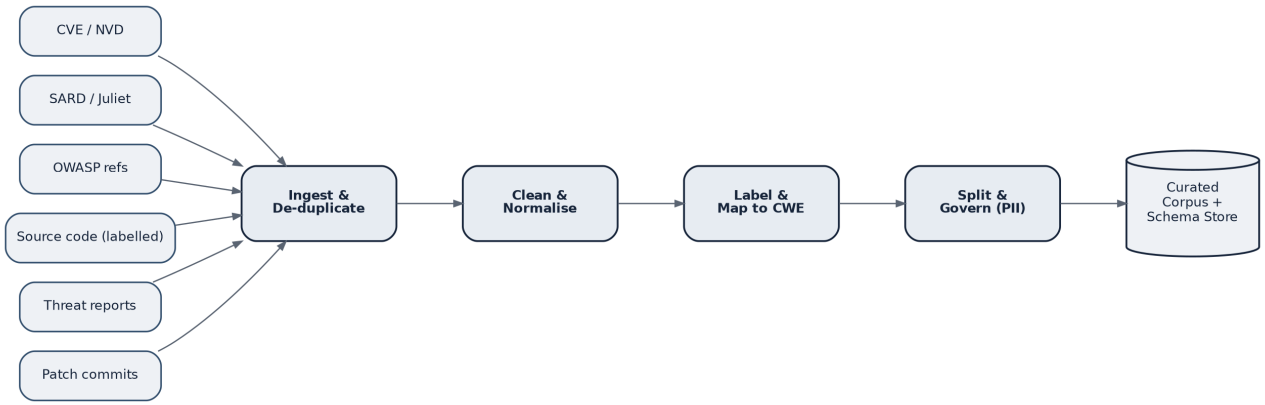



**Figure 30. Cybersecurity corpus curation pipeline**

*Authoritative sources — CVE/NVD, SARD and Juliet, OWASP references, labelled source code, threat reports and patch commits — are ingested and de-duplicated, cleaned and normalised, labelled and mapped to CWE, and split and governed for sensitive data, producing a curated corpus and schema store. All sources are vetted offline and imported through the one-way data diode.*

### *4.28.1 Sources*

**Table 38. Principal corpus sources and their contribution**

| Source | Contribution to the corpus |
|---|---|
| CVE / NVD | Catalogued vulnerabilities with descriptions, severity and references. |
| SARD | Curated test cases with known good and flawed code across many weakness classes. |
| Juliet Test Suite | Large labelled C/C++/Java test cases for specific CWE classes. |
| OWASP references | Weakness taxonomies, benchmarks and secure coding guidance. |
| Labelled source code | Vulnerable and fixed code with weakness labels and patch pairs. |
| Threat reports | Malware behaviour, indicators and tactics/techniques descriptions. |
| Patch commits | Real fix commits providing patch and rationale supervision. |

Source: Public vulnerability and secure-coding datasets and advisories, imported offline.

*Interpretation: The corpus draws on recognised, authoritative datasets, giving broad and defensible coverage of weakness classes for both training and evaluation.*

### *4.28.2 Cleaning, Labelling and Governance*

Ingested data is de-duplicated, normalised to consistent formats, and labelled — each record is mapped to a weakness class (CWE) and, where applicable, a linked CVE and severity. Data is partitioned into training, validation and test splits that are kept strictly separate so that evaluation is honest. Governance controls screen for and exclude sensitive or restricted material, and record the provenance of every record so that the corpus is auditable. The entire process runs offline; sources are vetted and imported through the one-way data path, and nothing is fetched from any external network during training or operation.

### *4.28.3 Sample Record Schema*

Each corpus record conforms to a defined schema so that data is consistent, queryable and traceable. An illustrative schema is shown below.

| Curated Corpus — Record Schema (illustrative) | | |
|---|---|---|
| **Field** | **Type** | **Description** |
| record_id | string | stable unique key |
| artifact_type | enum | source_code \| binary \| advisory |
| language / arch | string | C, C++, Java, Python \| x86-64, ARM |
| cwe_id | string | mapped weakness (e.g. CWE-89) |
| cve_id | string | linked CVE where applicable |
| severity | enum | low \| medium \| high \| critical |
| snippet / ref | text | sanitised code or feature vector |
| vuln_label | bool | vulnerable / not-vulnerable |
| fix_patch | text | reference secure patch (diff) |
| fix_rationale | text | why the fix resolves the weakness |
| provenance | string | source dataset + import batch |
| split | enum | train \| val \| test (governed) |

**Figure 31. Illustrative curated-corpus record schema**

*Each record carries a stable identifier, the artefact type and language or architecture, the mapped weakness (CWE) and any linked CVE and severity, a sanitised code snippet or feature reference, the vulnerability label, the reference patch and its rationale, the provenance, and the governed data split.*

## 4.29 Retrieval-Augmented Generation (RAG) Design

Retrieval-augmented generation grounds the model in an authoritative knowledge base so that its analysis is evidence-based and correlated, rather than reliant on parametric memory alone. This section describes the retrieval and correlation pipeline and the vector database that supports it.

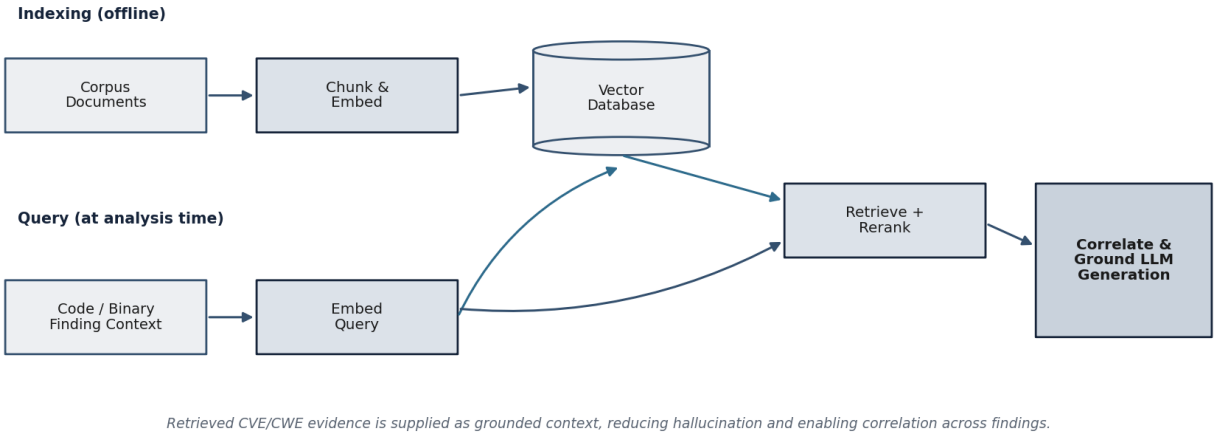



**Figure 32. Retrieval-augmented generation architecture with vector database**
*In the offline indexing path, corpus documents are chunked, embedded and stored in a vector database. At analysis time, the code or binary finding context is embedded, used to retrieve and rerank relevant evidence, and supplied to the model to correlate findings and ground its generation.*

### *4.29.1 Indexing and Retrieval*

In an offline indexing stage, the curated corpus is divided into semantically coherent chunks, each is embedded into a dense vector, and the vectors are stored in a vector database. At analysis time, the context of a candidate finding — a code fragment, a function, or a binary feature set — is embedded and used to retrieve the most relevant corpus entries. A reranking step orders the retrieved candidates by relevance, and a hybrid approach combines dense retrieval with lexical matching on identifiers such as CWE and CVE numbers.

### *4.29.2 Correlation and Grounding*

The retrieved evidence is supplied to the model as grounded context. This serves two purposes. It reduces hallucination, because the model reasons from authoritative retrieved material rather than from memory alone. And it enables correlation — the model can relate a newly observed weakness to known vulnerabilities, affected versions, and documented fixes, which improves both the precision of detection and the quality of the explanation and patch.

### *4.29.3 Knowledge-Base Maintenance*

The vector knowledge base is refreshed as the corpus is updated, through the same vetted, offline import path used for initial population. Because the knowledge base is decoupled from the model weights, new vulnerability information can be incorporated by re-indexing without retraining the model, which keeps the capability current under national control as the threat landscape evolves.

## 4.30 Agentic AI Framework

Detecting a candidate weakness is only the first step; confirming whether it is real, understanding its impact, and demonstrating it safely require multi-step reasoning. This section describes the agentic framework that performs that reasoning and constructs verifiable proof-of-concept chains. It complements the agentic red-teaming of Section 4.6, applying the same agentic principles to code and binary analysis.

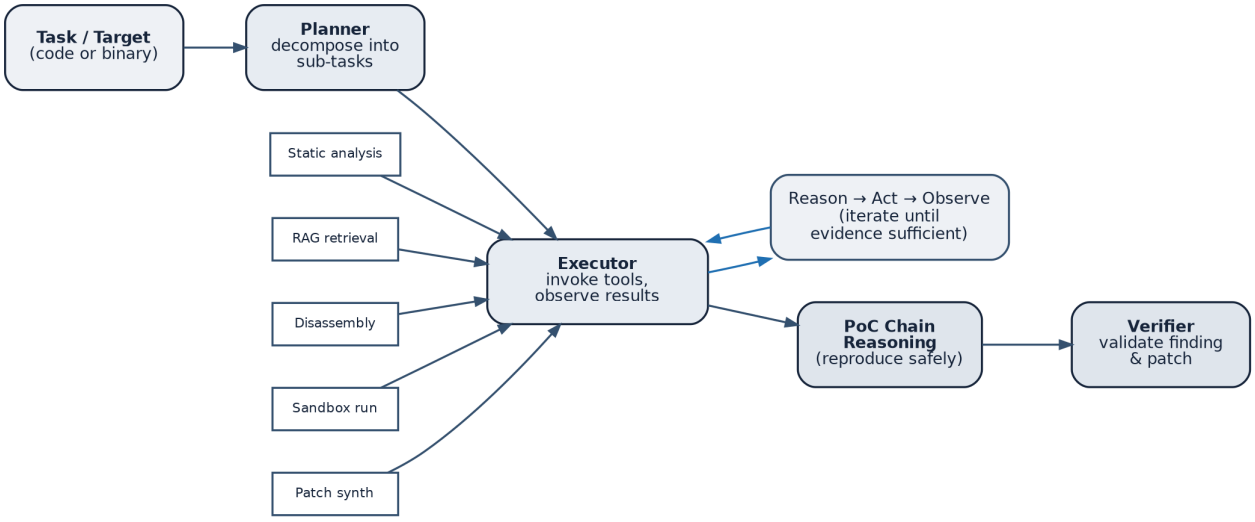



**Figure 33. Agentic multi-step analysis and proof-of-concept chain reasoning**

*A planner decomposes the task into sub-tasks; an executor invokes tools — static analysis, RAG retrieval, disassembler, sandbox, patch synthesis — and observes the results in a reason–act–observe loop; and a proof-of-concept chain is constructed and checked by a verifier before any finding is reported.*

### *4.30.1 Planner, Executor and Tools*

The agentic framework follows a planner–executor pattern. The planner decomposes an analysis objective into ordered sub-tasks; the executor carries them out by invoking a controlled set of tools — static analysers and parsers, RAG retrieval, disassemblers, an isolated execution sandbox, and the patch-synthesis service — and observes the result of each action. The agent iterates through a reason–act–observe loop, refining its hypotheses as evidence accumulates, until it has either confirmed a finding with sufficient evidence or determined that the candidate is a false positive.

### *4.30.2 Proof-of-Concept Chain Reasoning*

Where a weakness is confirmed, the agent constructs a proof-of-concept chain: an ordered, reproducible sequence of steps that demonstrates the weakness within the isolated sandbox. The chain is recorded so that it can be re-executed and independently reviewed. A separate verifier checks the chain and, for source-code findings, checks that a proposed patch genuinely resolves the weakness, before the finding is admitted to the report.

### *4.30.3 Controls on Agency*

The agent operates under strict controls. Its tools are limited to a vetted set; it runs only within the air-gapped sandbox; it cannot reach any external network; and it never modifies an operational system. Every action is logged to the tamper-evident audit trail. These controls ensure that the power of agentic reasoning is exercised safely, consistent with the responsible-use undertakings of this document.

## 4.31 Air-Gapped Deployment Strategy

The capability is designed to operate with no connection to any external or cloud network. Sections 4.11 and 4.12 describe the secure environment and the deployment and network topology in detail. This section states the air-gap strategy specifically as it applies to the AI capability — to model weights, corpora, training and inference.

### *4.31.1 Principles*

Three principles govern the strategy. First, isolation: all components — training, corpora, the vector knowledge base, serving and the analysis tooling — reside within the defence-controlled perimeter, and there is no route from any component to an external network. Second, one-way import: model base weights, corpus updates and software are vetted outside the perimeter and brought in through a controlled, one-way import station with integrity verification; nothing is fetched at train or run time. Third, no external dependence: the model is open-weight and served locally, so the capability never calls an external model API and is not exposed to the withdrawal of any external service.

### *4.31.2 Air-Gap Controls for the AI Capability*

**Table 39. Air-gap controls specific to the AI capability**

| Concern | Control |
|---|---|
| Model weights | Open-weight base imported once and held within the perimeter; fine-tuning performed locally. |
| Training data | Corpus vetted offline and imported one-way; no external fetch during pre-training or fine-tuning. |
| Knowledge base | Vector index built and refreshed offline; decoupled from weights; no external retrieval. |
| Inference | Local serving only; no external model or embedding API is ever called. |
| Updates | Vetted updates applied through the same one-way import path; air-gap never broken for maintenance. |
| Egress verification | Egress demonstrated nil during deployment and monitored continuously thereafter. |

Source: Air-gap strategy (this section) and the secure environment of Sections 4.11 and 4.12.

*Interpretation: Each element of the AI capability that might otherwise require external connectivity is addressed by a control that preserves complete isolation.*

## 4.32 Vulnerability Detection and Evaluation Plan

This section describes how the capability detects vulnerabilities in source code (Type-I) and how detection performance is evaluated against recognised benchmarks and reported with standard metrics.

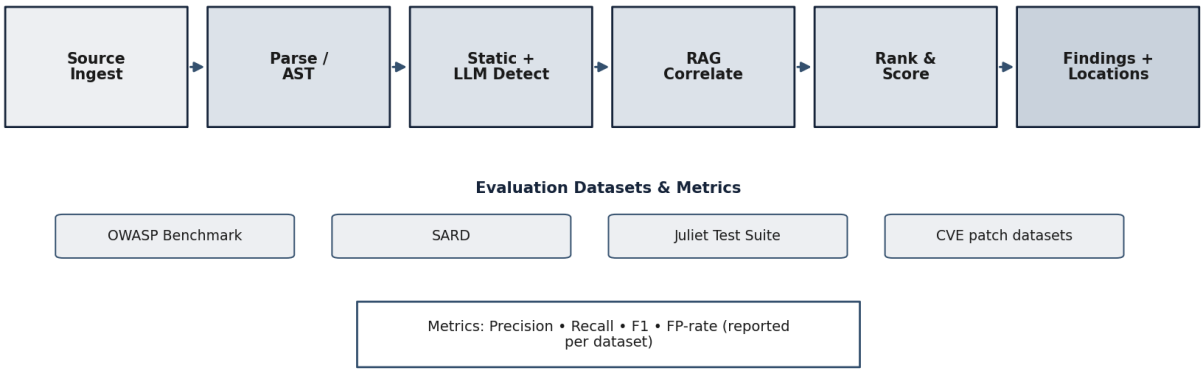



**Figure 34. Source-code vulnerability detection and evaluation pipeline**

*Source is ingested and parsed; static analysis and the fine-tuned model detect candidate weaknesses; RAG correlates them against known vulnerabilities; findings are ranked, scored and located. Detection performance is evaluated on the OWASP Benchmark, SARD, the Juliet Test Suite and CVE patch datasets, and reported as precision, recall and F1 with the false-positive rate.*

### *4.32.1 Detection Approach*

Source code is parsed into syntax trees and intermediate representations; static-analysis adapters surface candidate weaknesses; and the fine-tuned model, grounded by RAG, both detects context-dependent weaknesses that rules miss and filters likely false positives that rules over-report. The agentic layer (Section 4.30) confirms candidates through multi-step analysis. Each confirmed finding is reported with its weakness class, location, severity, an explanation, and — where applicable — a candidate patch.

### *4.32.2 Evaluation Datasets*

**Table 40. Evaluation datasets and their role**

| Dataset | Role in evaluation |
|---|---|
| OWASP Benchmark | Measures detection accuracy against a known set of true and false test cases. |
| SARD | Provides labelled good/flawed cases across many weakness classes. |
| Juliet Test Suite | Provides large labelled C/C++/Java cases for specific CWE classes. |
| CVE patch datasets | Evaluate detection on real vulnerabilities and the correctness of fixes. |

Source: Recognised public vulnerability-detection benchmarks and datasets.

*Interpretation: Using multiple recognised datasets gives a rounded, defensible measure of detection performance across both synthetic and real-world cases.*

### *4.32.3 Metrics and Targets*

Detection performance is reported with standard metrics — precision, recall, the F1 score, and the false-positive rate — computed per dataset so that performance is transparent rather than averaged into a single opaque figure. Indicative targets are shown below; these are honest engineering targets to be confirmed by measurement during the proof-of-concept, and measured interim values will be reported where a target is approached rather than fully met.

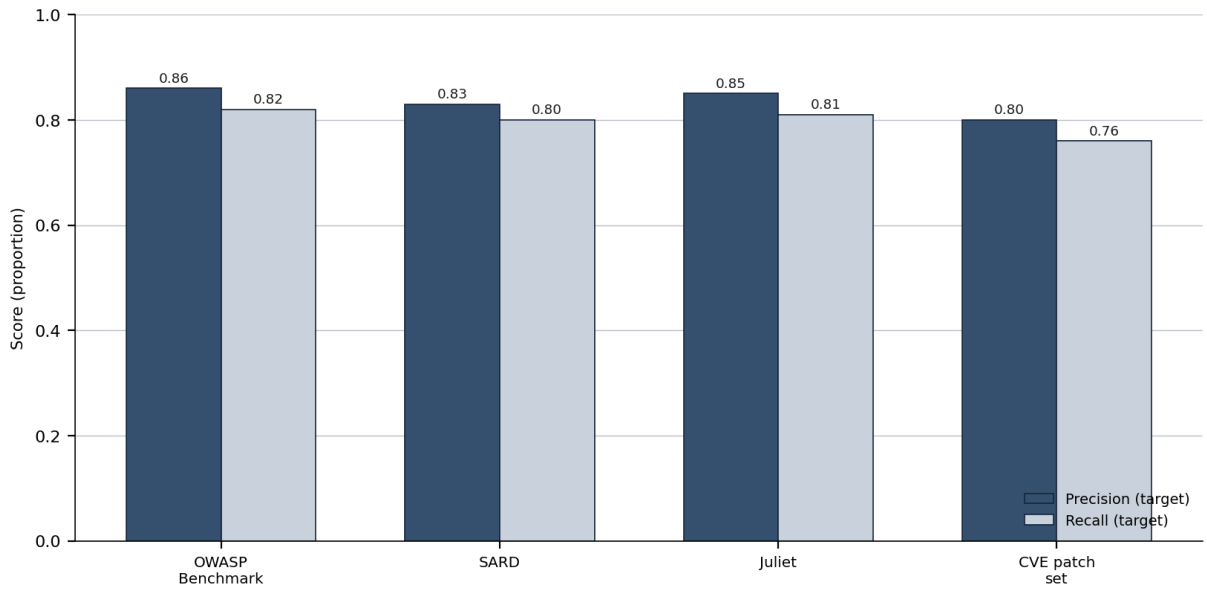



**Figure 35. Indicative detection-performance targets by dataset**

*Indicative precision and recall targets for each evaluation dataset. The targets are deliberately realistic; actual measured values, with confidence intervals, will be reported per dataset during the proof-of-concept and qualification.*

## 4.33 Malware Analysis and Reverse-Engineering Capability

For Type-II artefacts — binaries and suspected malware for which no source is available — the capability applies reverse-engineering tooling together with the AI analysis engine. This section describes the tooling and the analysis workflow.

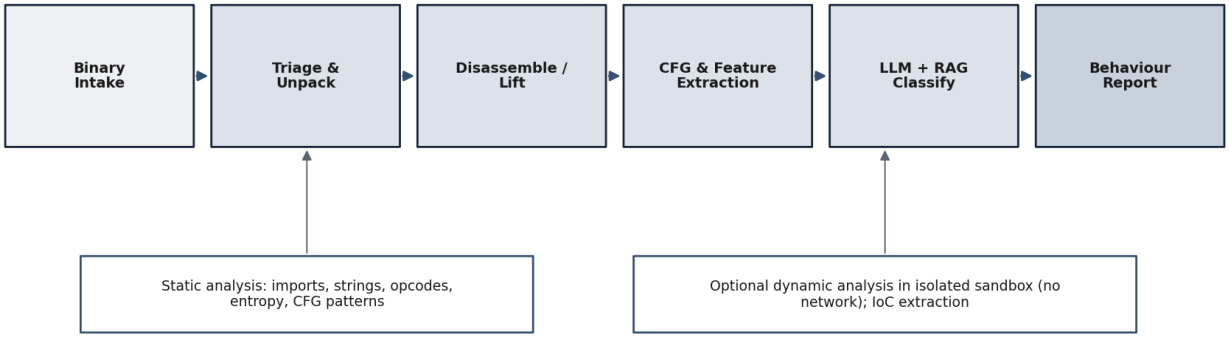



**Figure 36. Malware analysis and reverse-engineering workflow**

*A binary is taken in, triaged and unpacked, then disassembled and lifted using established reverse-engineering tools; control-flow and features are extracted; the fine-tuned model with RAG classifies behaviour; and a behaviour report is produced. Static analysis (imports, strings, opcodes, entropy, control-flow patterns) is complemented by optional dynamic analysis in an isolated, network-free sandbox.*

### *4.33.1 Tooling*

**Table 41. Reverse-engineering capabilities and their use**

| Capability | Use in the workflow |
|---|---|
| Disassembly / decompilation | Disassembly, decompilation and control-flow recovery for multiple architectures. |
| Scriptable static analysis | Scriptable static analysis, disassembly and binary inspection. |
| Interactive RE front-end | Interactive reverse-engineering front-end for analyst-driven inspection. |
| Sandbox | Isolated, network-free execution for optional dynamic behaviour and indicator extraction. |

Source: Established reverse-engineering capabilities integrated into the binary-analysis subsystem (Section 4.23, A6).

*Interpretation: The capability provides the disassembly, decompilation and interactive-analysis functions that professional reverse engineers rely on, integrated with the AI engine for scale and consistency.*

### *4.33.2 Analysis Workflow*

A binary is first triaged — file type, packing and obvious indicators — and unpacked where possible. It is then disassembled and lifted, and its control-flow graph and features (imports, strings, opcode sequences, entropy and structural patterns) are extracted. The fine-tuned model, grounded by retrieved threat knowledge, classifies the binary's likely behaviour and identifies indicators of compromise. Where safe and authorised, optional dynamic analysis is performed in an isolated sandbox with no network access to observe runtime behaviour. The result is a behaviour report with the evidence on which it rests.

### *4.33.3 Safety and Containment*

All binary and malware analysis is conducted within the air-gapped perimeter and, for any dynamic analysis, within an isolated sandbox that has no network path and is reverted to a clean state after each run. This ensures that the handling of potentially malicious artefacts cannot itself cause harm or escape containment.

## 4.34 Secure Code Patching Approach

For confirmed Type-I findings, the capability generates a candidate secure patch, validates it, and documents the rationale for the fix. This section describes that workflow and the controls that keep it safe.

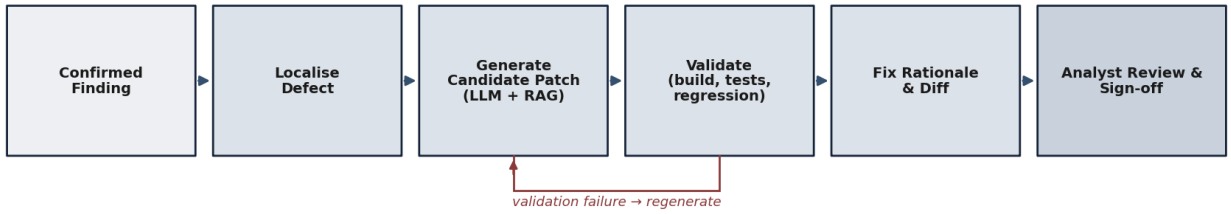



**Figure 37. Secure code patching workflow**

*From a confirmed finding, the defect is localised; a candidate patch is generated by the model grounded with retrieved fix knowledge; the patch is validated by building and running tests and regression checks; a fix rationale and diff are produced; and the patch is presented for analyst review and sign-off. A validation failure returns the patch for regeneration; the framework never auto-commits changes to operational systems.*

### *4.34.1 Patch Generation*

The defect is first localised to the precise code construct responsible for the weakness. The model, grounded by RAG with documented fixes for the same weakness class, then generates a candidate patch as a minimal, targeted change that resolves the weakness while preserving intended behaviour. Generating from retrieved, real fixes improves the correctness and idiomatic quality of the patch.

### *4.34.2 Patch Validation*

A candidate patch is not trusted until it is validated. Validation applies the patch and confirms that the code still builds, that existing tests pass, that regression checks reveal no new failures, and — where a proof-of-concept exists for the weakness — that the weakness is no longer reproducible. A patch that fails validation is returned for regeneration; only a patch that passes is presented to the analyst.

### *4.34.3 Fix Rationale and Human Sign-off*

Every validated patch is accompanied by a fix rationale that explains the weakness, why the change resolves it, and any residual considerations, together with the diff. The patch and its rationale are presented to a human analyst for review and sign-off. The framework proposes; it does not autonomously modify operational systems. This keeps a qualified human in control of any change, consistent with the responsible-use posture of this document.

## 4.35 Proof-of-Concept Plan for the AI Capability

This section sets out the proof-of-concept for the AI capability, covering both a Type-I source-code demonstration and a Type-II binary/malware demonstration. It complements the framework-level proof-of-concept definition of Section 4.20.

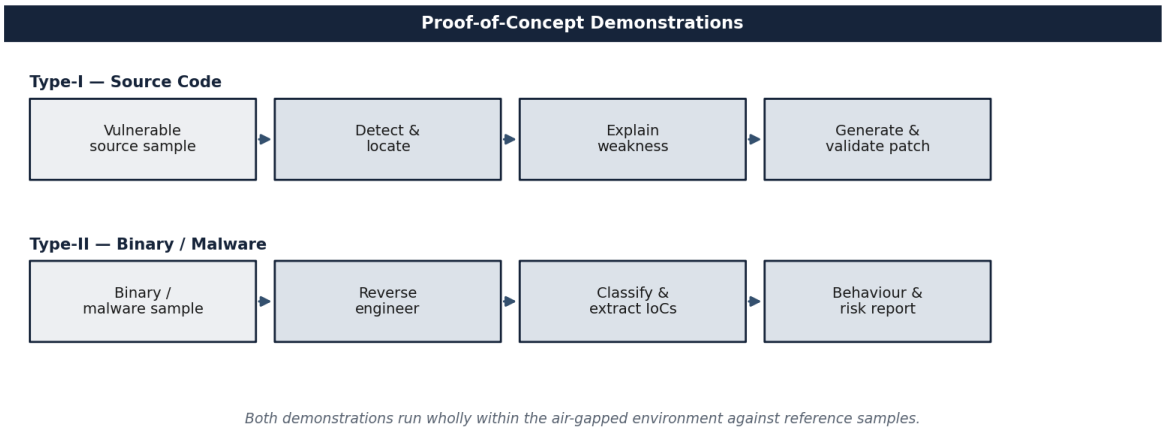



**Figure 38. Proof-of-concept flows for Type-I source code and Type-II binary / malware**
*The Type-I demonstration takes a vulnerable source sample through detection, location, explanation, and validated patch generation. The Type-II demonstration takes a binary or malware sample through reverse engineering, classification and indicator extraction, to a behaviour and risk report. Both run wholly within the air-gapped environment against reference samples.*

### *4.35.1 Type-I Demonstration (Source Code)*

The Type-I demonstration takes a set of vulnerable source-code samples — drawn from recognised datasets and representative weakness classes — and shows the capability detecting and locating each weakness, explaining why the code is vulnerable, generating a candidate patch, and validating that the patch resolves the weakness without breaking the build or tests. Detection performance on the held-out samples is reported with precision and recall (Section 4.32).

### *4.35.2 Type-II Demonstration (Binary / Malware)*

The Type-II demonstration takes a set of binary and benign/malicious samples and shows the capability reverse-engineering each sample with the integrated tooling, extracting features and control-flow, classifying its likely behaviour, and producing a behaviour and risk report with indicators of compromise. Where authorised, optional dynamic analysis in the isolated sandbox is shown.

### *4.35.3 Proof-of-Concept Scope and Success*

**Table 42. Proof-of-concept scope and success definition for the AI capability**

| Aspect | Definition |
|---|---|
| Environment | Wholly within the air-gapped perimeter; egress demonstrated nil. |
| Type-I scope | Detection, location, explanation and validated patching on reference source samples. |
| Type-II scope | Reverse engineering, classification and reporting on reference binary samples. |
| Metrics | Precision / recall / F1 (Type-I); classification and indicator accuracy (Type-II). |
| Evidence | Reproducible PoC chains, model card, and findings / patch artefacts. |
| Success | Targets met or honestly reported with measured interim values; outputs validate against schema. |

Source: Proof-of-concept design (this section) and the evaluation plan (Section 4.32).

*Interpretation: The proof-of-concept evidences both artefact types end-to-end under air-gapped conditions, with success defined by measured, reproducible results rather than assertion.*

## 4.36 Testing and Qualification Plan

This section describes how the capability is tested at each level and how its readiness for qualification is established, combining automated benchmarking with analyst validation. It complements the acceptance methodology of Sections 4.7 and 4.15.

### *4.36.1 Levels of Testing*

**Table 43. Levels of testing for the AI capability**

| Level | Purpose | Method |
|---|---|---|
| Unit | Verify individual components behave correctly. | Automated unit tests in CI. |
| Integration | Verify subsystems interoperate across the pipeline. | Automated integration tests on representative artefacts. |
| Model evaluation | Measure detection / classification performance. | Benchmarking on OWASP Benchmark, SARD, Juliet, CVE patch datasets. |
| Patch validation | Verify generated patches are correct. | Build, test and regression checks; PoC non-reproducibility. |
| Security | Verify the platform and its controls. | Hardening checks; egress verification; audit completeness. |
| Analyst acceptance | Confirm findings and patches are trustworthy. | Expert review of a sampled set of outputs. |

Source: Test strategy (this section) aligned with the engineering discipline of Section 14.1.

*Interpretation: Testing spans component correctness, model performance, patch correctness, platform security, and expert acceptance, giving rounded assurance.*

### *4.36.2 Analyst Validation*

Automated metrics are necessary but not sufficient; a sample of findings, classifications and patches is reviewed by qualified security analysts to confirm that the outputs are accurate, actionable and free of unsafe recommendations. Analyst feedback is fed back into corpus curation and fine-tuning, closing the loop between measurement and improvement.

### *4.36.3 Qualification Readiness*

The capability is considered ready for qualification when its components and pipeline pass their tests; when measured detection and classification performance meets the agreed targets, or is reported honestly with measured interim values where a target is approached; when generated patches are validated and accepted by analysts at the agreed rate; when platform-security checks and egress verification pass; and when the evidence — model card, benchmark results, PoC chains and audit logs — is complete and validates against its schemas. These criteria align with the framework-level exit criteria of Section 4.18.

# 5. Development Estimates (with respect to Time)

***i. Estimated time for development (maximum 24 months), with break-up — the programme is planned for completion within twenty-one months, comfortably inside the EoI's twenty-four-month ceiling, across seven quarterly milestones (M1–M7); the schedule, milestone gates and deliverables are detailed in the sub-sections below.***

***ii. Suo-moto design and development of a similar product / technology — Yes / No.***

**Yes — at component level.** The firm has, on its own initiative, built related cybersecurity-AI components and prototypes (LLM fine-tuning and instruction-tuning exercises with documented evaluation, retrieval-augmented analysis pipelines, agentic tooling, and AI-security / governance product work). It has not previously built the full integrated air-gapped framework that is the subject of this EoI; the proposed programme integrates and scales that prior component work. Anonymised evidence will be furnished at the RFP stage.

## 5.1 Project Schedule, Milestones and Roadmap

### *5.1.1 Scheduling Philosophy*

The schedule is planned for completion within twenty-one months — comfortably inside the twenty-four-month ceiling specified by the EoI, leaving a three-month margin against schedule risk — and is organised so that capability is built incrementally and demonstrated early. The critical path runs through the secure platform (WP2), the attack-generation and red-teaming engines (WP4, WP5), and validation (WP8); the privacy, static-analysis and certification work packages proceed in parallel and converge at the integration milestones. The plan deliberately front-loads the air-gapped environment and the ingestion pipeline, because every downstream module depends on them, and it reserves the final quarter for hardening, full-suite benchmarking and documentation rather than new feature development.

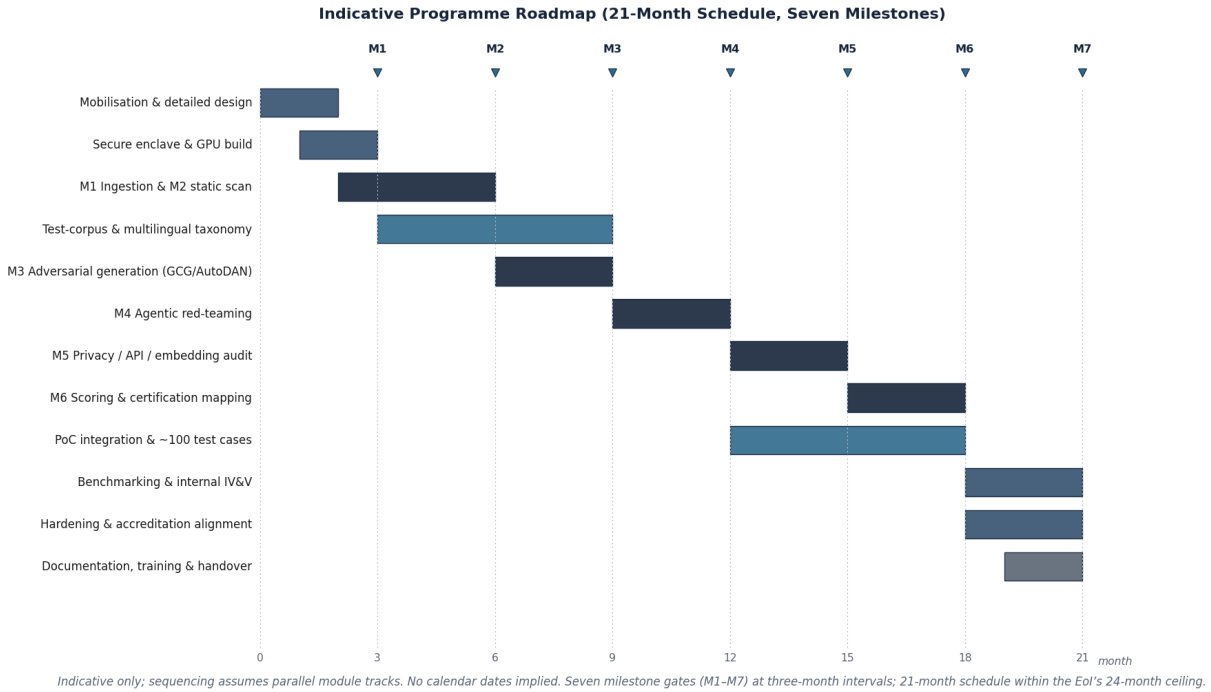



**Figure 39. Indicative twenty-one-month schedule by work package**

*Indicative phasing of the eight work packages across the twenty-one-month schedule (within the EoI's twenty-four-month ceiling). Bars denote periods of principal effort; the seven milestones and their exit criteria are listed in the milestone deliverables table. The schedule is indicative and contains no resource-loading or cost information.*

### *5.1.2 Phases and Milestones*

The work is structured around seven milestones spaced at three-month intervals across the twenty-one-month schedule — one milestone per quarter — each a reviewable gate with explicit exit criteria. Milestones are expressed in elapsed months from programme start and carry no financial connotation.

**Table 44. Programme milestones and exit criteria**

| Milestone | Month | Phase / focus | Exit criteria |
|---|---|---|---|
| M1 | 3 | Secure foundation | Air-gapped environment accepted; offline dependency mirror operational; model-ingestion and static-analysis pipeline scanning reference artefacts; egress verified nil. |
| M2 | 6 | Static analysis & artefact scanning | Modules M1 and M2 operational across pickle, safetensors, ONNX and checkpoint formats; malicious-artefact detection demonstrated on a curated corpus; provenance and integrity checks enforced. |
| M3 | 9 | Adversarial attack generation | Attack-generation engine operational; 50+ technique jailbreak library and prompt-injection corpus integrated; first robustness scores produced on a reference model. |
| M4 | 12 | Agentic red-teaming | Multi-turn agentic red-teaming harness operational across harm categories; attack-success-rate metrics produced; integrated proof-of-concept across modules M1–M4. |

| Milestone | Month | Phase / focus | Exit criteria |
|---|---|---|---|
| M5 | 15 | Privacy, API & embedding audit | Membership-inference and extraction module producing results on canary datasets; inference-API and embedding-space audit functional; differential-privacy checks in place. |
| M6 | 18 | Certification & integration | Severity scoring and OWASP / ATLAS / NIST mapping integrated; certification dashboard rendering end-to-end; multilingual coverage demonstrated across the required languages. |
| M7 | 21 | Validation, benchmarking & programme close | Full security suite executed within target duration on a seven-billion-parameter reference model; figures of merit measured; ~100 validated test cases assembled; internal IV&V; complete; documentation complete; proof-of-concept demonstrated; evidence package delivered; acceptance recorded. |

Source: Derived from the work breakdown structure (Section 4.13) and the EoI performance specifications (Section 4.2.4).

*Interpretation: Each milestone is a quarterly decision gate with verifiable exit criteria. The plan demonstrates working capability from Month 3 and reserves the final quarter for validation, benchmarking and documentation rather than new development, completing in twenty-one months within the EoI's twenty-four-month ceiling.*

### *5.1.3 Deliverables Schedule*

The principal deliverables and the milestone at which each becomes available are summarised below. The schedule culminates in the proof-of-concept artefacts required by the EoI's nature-of-deliverables clause, namely the scanning engine, the red-teaming agentic framework, the static-analysis pipeline, the privacy-audit module, the certification dashboard, and approximately one hundred validated vulnerability test cases.

**Table 45. Principal deliverables and availability by milestone**

| Deliverable | Type | Available by |
|---|---|---|
| Project management and configuration plan | Document | M1 |
| Hardened air-gapped platform and offline mirror | System | M1 |
| Model-ingestion and static-analysis pipeline | Software | M2 |
| Adversarial attack-generation engine | Software | M3 |
| Jailbreak technique library (50+ techniques) | Software / corpus | M3 |
| Prompt-injection test corpus | Corpus | M3 |
| Agentic red-teaming harness | Software | M4 |
| Privacy-audit module (membership inference, extraction) | Software | M5 |
| Inference-API and embedding-space audit | Software | M5 |
| Severity scoring and standards-mapping engine | Software | M6 |
| Certification dashboard | Software | M6 |
| Multilingual attack corpus (required languages) | Corpus | M6 |
| Full-suite benchmark results and figures of merit | Report | M7 |
| ~100 validated vulnerability test cases | Corpus | M7 |
| User, operations and certification documentation | Document | M7 |
| Proof-of-concept demonstration package | System / document | M7 |

Source: EoI nature-of-deliverables clause (Section 2.5) and the work-package register.

*Interpretation: Every deliverable named in the EoI is scheduled and tied to a milestone, giving the evaluator a clear basis for tracking progress at the subsequent RFP stage.*

### *5.1.4 Suo-Moto Design and Development Declaration*

In response to the EoI's development-estimates clause, the firm records the following position on prior suo-moto design and development of the same or a similar product. ZeroShield Private Limited has not previously delivered, and does not claim to have delivered, a completed sovereign LLM vulnerability-assessment framework of the scope described in this EoI. The firm has, however, developed internal capability, design assets and working knowledge across the relevant technology areas — adversarial prompting and jailbreak analysis, serialised-artefact inspection, membership-inference techniques, agentic orchestration, and air-gapped engineering practice — which it brings to this programme. This declaration is made honestly and conservatively; the firm has deliberately avoided overstating prior work, in keeping with the integrity expected of a government technical submission. The detailed statement of experience against the EoI's essential criteria is given in Section 4 and the supporting compliance table.

## 5.2 Development Timeline, Milestone Deliverables and Suo-moto Capability

This section presents the development timeline for the AI capability within the 21-month schedule, the milestone-wise deliverables, and the firm's suo-moto capability in related products and technology. It complements the programme schedule of Section 5.1.

### *5.2.1 Development Timeline (≤ 21 Months)*

The development of the AI capability fits within the 21-month envelope described in Section 5.1 and proceeds in five overlapping phases: foundation and corpus; model adaptation and fine-tuning; detection, RAG and agentic analysis; reverse engineering and patching; and proof-of-concept, testing and qualification. The phase-wise breakdown is below.

**Table 46. Phase-wise development timeline for the AI capability (within 21 months)**

| Phase | Indicative window | Focus |
|---|---|---|
| P1 Foundation & corpus | Months 1–4 | Environment build; corpus curation and schema; data governance. |
| P2 Model adaptation | Months 3–9 | Domain-adaptive pre-training; SFT and instruction tuning; model card. |
| P3 Detection, RAG & agentic | Months 7–14 | RAG layer; source-code detection; agentic analysis and PoC chains. |
| P4 RE & patching | Months 12–18 | Binary analysis and reverse engineering; secure patching and validation. |
| P5 PoC, test & qualification | Months 16–21 | Type-I and Type-II PoC; benchmarking; analyst validation; qualification. |

Source: Programme schedule (Section 5.1) elaborated for the AI capability.

*Interpretation: The five phases overlap deliberately so that corpus, model, analysis and remediation mature in step, with the final phase reserved for demonstration and qualification.*

### *5.2.2 Milestone-wise Deliverables*

**Table 47. Milestone-wise deliverables for the AI capability**

| Milestone | Deliverable |
|---|---|
| M-A (≈ Month 4) | Curated corpus v1 with schema and governance; environment ready. |
| M-B (≈ Month 9) | Fine-tuned model v1 with model card and initial evaluation. |
| M-C (≈ Month 14) | RAG layer and Type-I detection with agentic analysis; API and console. |
| M-D (≈ Month 18) | Type-II reverse-engineering capability and validated secure patching. |
| M-E (≈ Month 21) | Integrated Type-I and Type-II proof-of-concept within the air-gapped environment; benchmarked, analyst-validated and qualification-ready with complete evidence. |

Source: Milestone plan (this section) consistent with the roadmap of Section 5.1.

*Interpretation: Each milestone maps to a concrete, demonstrable deliverable spanning model, pipeline, API, console, PoC, testing and deployment readiness.*

### *5.2.3 Suo-moto Similar Product and Technology*

The firm's suo-moto capability — the indigenous products and technology it has already developed in related areas — is described in Section 14.1. That capability spans AI-security and security-assessment tooling and the engineering foundations (transformer adaptation, retrieval, agentic orchestration, and secure on-premises deployment) on which the proposed AI capability builds. Consistent with the integrity posture of this submission, the firm describes this capability as a portfolio and engineering foundation, and proposes to evidence the specific vulnerability- detection-and-patching capability through the proof-of-concept (Section 4.35) rather than by asserting unverified prior deployments.

# 6. Likely Academic Partners / R&D Institutions Required

The firm is incubated within the **IIT Kanpur** ecosystem through **C3iHub** (the national Technology Innovation Hub for Cyber Security and Cyber Defence), giving ready access to academic and research collaboration. Academic / R&D support may be drawn upon for:

- Large-scale and distributed model training (access to higher-end GPU clusters for full-scale fine-tuning).
- Adversarial-robustness and membership-inference research and formal evaluation methodology.
- Independent benchmarking and review of detection performance and of the proof-of-concept test set.

# 7. Existing Academia / Research Institute Support

**Yes.** The firm is an incubated start-up at **C3iHub, IIT Kanpur**, and operates from the C3iHub premises (2nd Floor, New Technopark Building, IIT Kanpur). This provides mentorship and access to the institute's cyber-security research ecosystem; the incubation / association evidence will be furnished at the RFP stage.

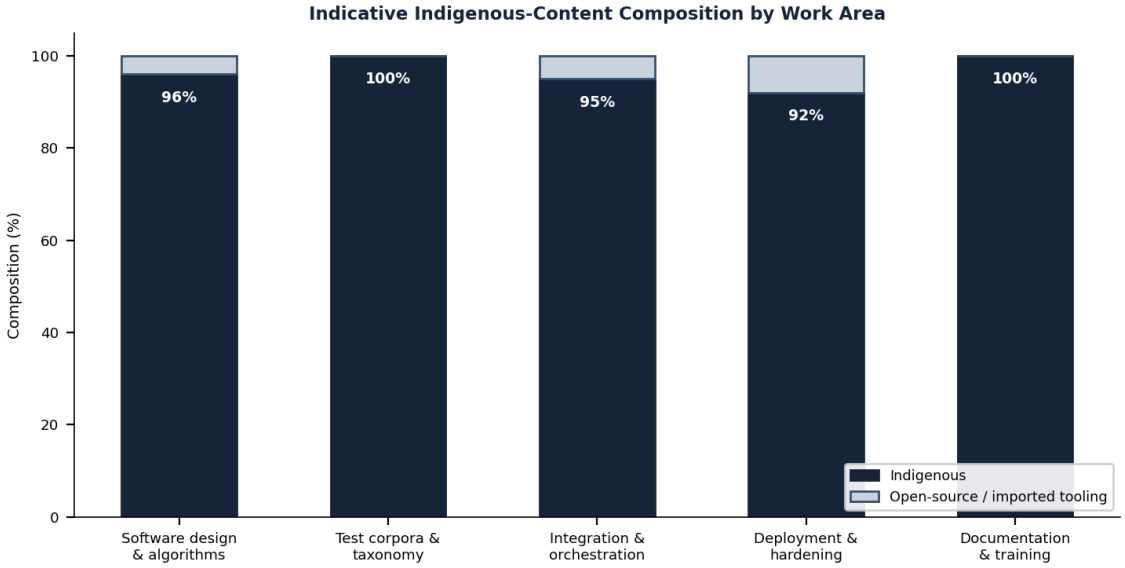
# 8. Likely Indigenous Content in the Proposed Solution

The proposed solution is predominantly indigenous: all bespoke design, fine-tuning, integration, orchestration and analyst-facing development is performed in-house in India; the only non-bespoke elements are permissively-licensed open-source libraries and open-weight base models deployed entirely offline. The indicative composition and declaration follow.

## 8.1 Indigenous Content and Capability Roadmap

### *8.1.1 Indigenous-Content Position*

Indigenous content is central to the rationale of this EoI: the entire purpose of the framework is to remove the dependence on foreign, cloud-bound tools that the problem statement identifies as an unacceptable sovereignty risk. The proposed solution is designed to be substantially indigenous in every dimension that matters for sovereignty — the intellectual property, the software, the operating environment, and the data — and to rely on the global open-source commons only for general-purpose, non-controlled building blocks that carry no sovereignty risk and are themselves vendored and held within the Indian perimeter.




*Imported elements are permissively-licensed open-source libraries deployed offline; no proprietary or foreign-controlled service is embedded.*

**Figure 40. Indicative indigenous-content profile by solution dimension**

*Indicative split between indigenous content and vendored open-source building blocks across the principal dimensions of the solution. The intellectual property, integration, corpora and deployment are indigenous; open-source foundations are general-purpose, non-controlled, and held offline within the Indian perimeter.*

### *8.1.2 Composition of Indigenous Content*

The indigenous character of the solution rests on the following. First, all design and intellectual property — the architecture, the attack-generation and scoring algorithms, the rule-sets, and the integration — is created by the firm in India. Second, the complete software stack of the framework is developed indigenously and runs entirely within the air-gapped environment. Third, the test corpora, and in particular the Indian-language and defence-relevant harm taxonomies, are curated in India and have no foreign analogue. Fourth, the deployment and operation occur wholly within Indian, defence-controlled infrastructure. The only non-indigenous elements are widely used open-source libraries, model-serving runtimes and reference open-weight models used for development and validation; these are general-purpose, are not subject to foreign control in operation, and are vendored offline so that no external dependency exists at run time.

**Table 48. Indigenous and vendored elements by dimension**

| Dimension | Indigenous element | Vendored open-source element |
|---|---|---|
| Intellectual property and design | Architecture, algorithms, scoring, rule-sets, integration logic — created in India. | None. |
| Application software | All framework modules (M1–M6) developed by the firm. | General-purpose libraries and runtimes, vendored offline. |
| Test corpora and taxonomies | Indian-language attack corpus; defence-relevant harm taxonomy; validated test cases. | Reference public attack patterns adapted and extended. |
| Model artefacts (development / validation) | Evaluation methodology and canary constructions. | Open-weight reference models, held offline for testing only. |
| Operating environment and deployment | Air-gapped, defence-controlled Indian infrastructure; deployment automation. | Container and orchestration runtimes, vendored offline. |
| Data and findings | All model weights, prompts, results and reports remain in the Indian secure perimeter. | None. |

Source: Solution architecture (Sections 4.5–4.6) and deployment design (Section 4.12).

*Interpretation: Everything that bears on sovereignty — IP, software, corpora, environment and data — is indigenous; the only foreign content is general-purpose open-source building blocks that are vendored and confined offline, consistent with the EoI's air-gap and data-sovereignty requirements.*

### *8.1.3 Capability-Maturity Roadmap*

Beyond the proof-of-concept, the firm sees a clear maturation path for the capability, which it summarises here to demonstrate the depth of its thinking rather than to make any commitment beyond the scope of this EoI. The framework matures from a validated proof-of-concept, through hardened and accredited operation in a defence facility, towards a continuously updated sovereign capability whose corpora and techniques track the evolving threat. The indicative stages are shown below.

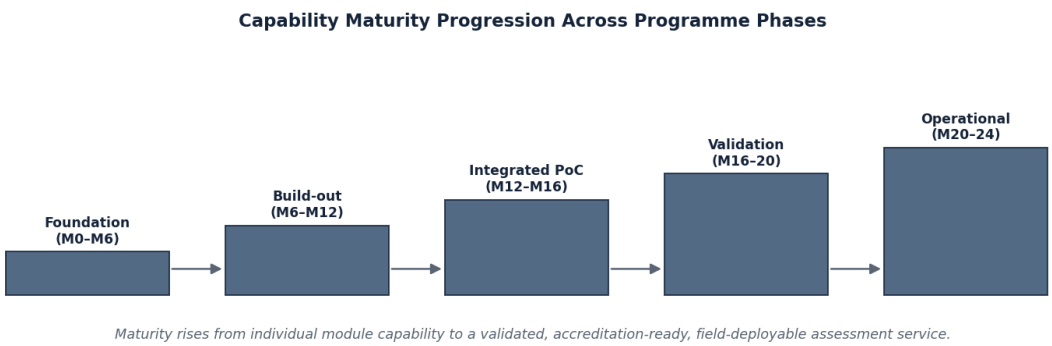



**Figure 41. Indicative capability-maturity roadmap for the framework**

*Indicative maturation of the capability from proof-of-concept towards a continuously maintained sovereign assessment service. The roadmap is illustrative of direction and contains no commitment or cost beyond the scope of the present EoI.*

### *8.1.4 Strategic Value of Indigenous Capability*

An indigenous framework does more than avoid the risks of foreign tooling. It creates a national capability to certify any model — indigenous or procured — before deployment in a national-security context; it keeps the knowledge of how sovereign models fail, and of the zero-day vulnerabilities discovered in them, within the country and under the End-User's control; and it establishes a basis on which Indian defence accreditation processes for AI systems can be built. These outcomes are the strategic justification for the indigenous content of the proposed solution and align it with the intent of the Technology Development Fund.

## 8.2 Indigenous Content Declaration

Section 8.1 describes the indigenous nature of the proposed capability in narrative terms. This section provides an explicit self-declaration of indigenous content, with an indicative percentage estimate and a breakdown by element, as required by the EoI.

### *8.2.1 Declaration*

ZeroShield Private Limited is an Indian-owned and Indian-controlled company, incorporated in India, whose directors are resident Indian nationals. The proposed capability will be designed, developed, integrated, trained and supported in India by an India-based team, using predominantly open-source software and the firm's own intellectual property, and deployed on infrastructure within the sovereign, air-gapped perimeter. On this basis the firm declares that the indigenous content of the proposed solution, measured by development and integration effort and by the value of the software and intellectual property created, is estimated at not less than eighty-five per cent (≥ 85%). This is a good-faith self-declaration; the firm will substantiate it with a detailed computation at the RFP stage on the basis prescribed by the procuring authority.

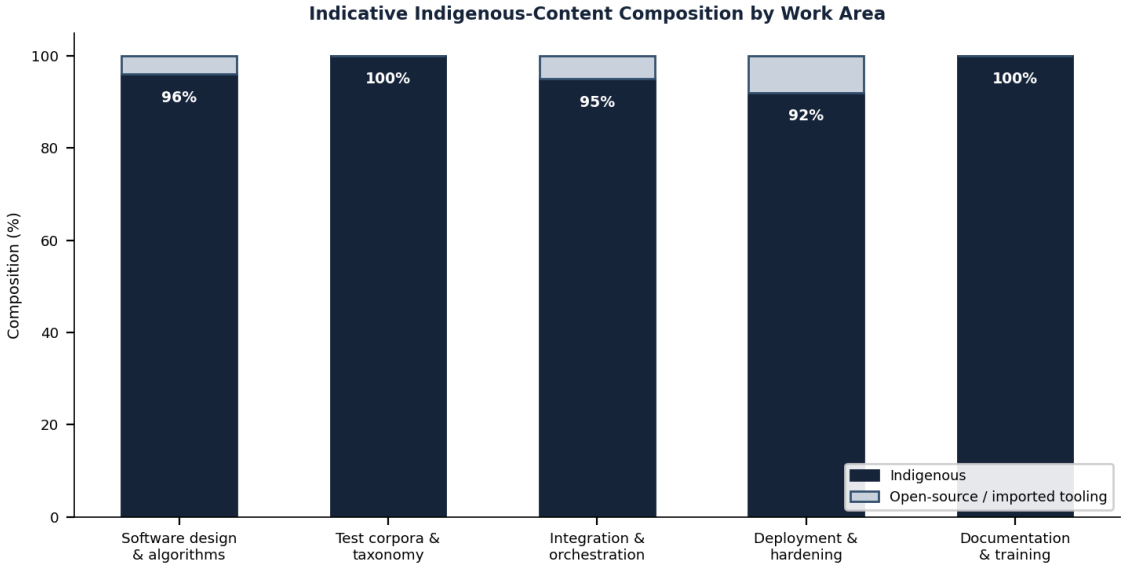
### *8.2.2 Indicative Breakdown*

**Table 49. Indicative indigenous-content breakdown by element**

| Element | Indigenous? | Indicative basis |
|---|---|---|
| Solution design & architecture | Yes | Performed in India by the firm's team. |
| Application, services & integration code | Yes | Developed in India as the firm's intellectual property. |
| Corpus curation, fine-tuning & RAG | Yes | Performed in India within the perimeter. |
| Agentic and analysis pipelines | Yes | Developed and integrated in India. |
| Open-source components | Indigenised in use | Open-source tooling integrated, configured and supported in India. |
| Open-weight base model | Non-indigenous (adapted in India) | Imported once; all adaptation and serving performed locally. |
| Commodity hardware | Largely non-indigenous | COTS compute / storage; integrated and operated in India. |
| Engineering, support & sustainment | Yes | Provided in India for the life of the capability. |

Source: Self-declaration by ZeroShield Private Limited; to be substantiated at the RFP stage on the prescribed basis.

*Interpretation: The substantial majority of the value — design, software, intellectual property, integration and sustainment — is indigenous; the principal non-indigenous elements are the open-weight base model (adapted in India) and commodity hardware.*




*Imported elements are permissively-licensed open-source libraries deployed offline; no proprietary or foreign-controlled service is embedded.*

**Figure 42. Indigenous content and sovereignty of the proposed capability**

*The capability is designed, built, trained, integrated and sustained in India, on Indian-controlled infrastructure, using open-source software and the firm's own intellectual property; the principal imported element is the open-weight base model, which is adapted and served entirely within the sovereign perimeter.*

# 9. Risk Identification and Mitigation Plan

Principal risks are identified, assessed by likelihood and impact, and managed with explicit mitigations and contingencies. The heat matrix and risk register follow.

## 9.1 Risk Identification and Mitigation

### *9.1.1 Risk-Management Method*

Risks are managed on a continuous basis throughout the programme. Each risk is described by its cause, its consequence, and its rating on a five-by-five likelihood-by-impact scale; risks are reviewed at every milestone and whenever a triggering event occurs. The rating determines the response: extreme and high risks carry named mitigations and are tracked at every review, medium risks are monitored with defined triggers, and low risks are accepted with periodic re-assessment. The heat matrix below positions the ten principal risks; the register that follows defines each risk and its mitigation.

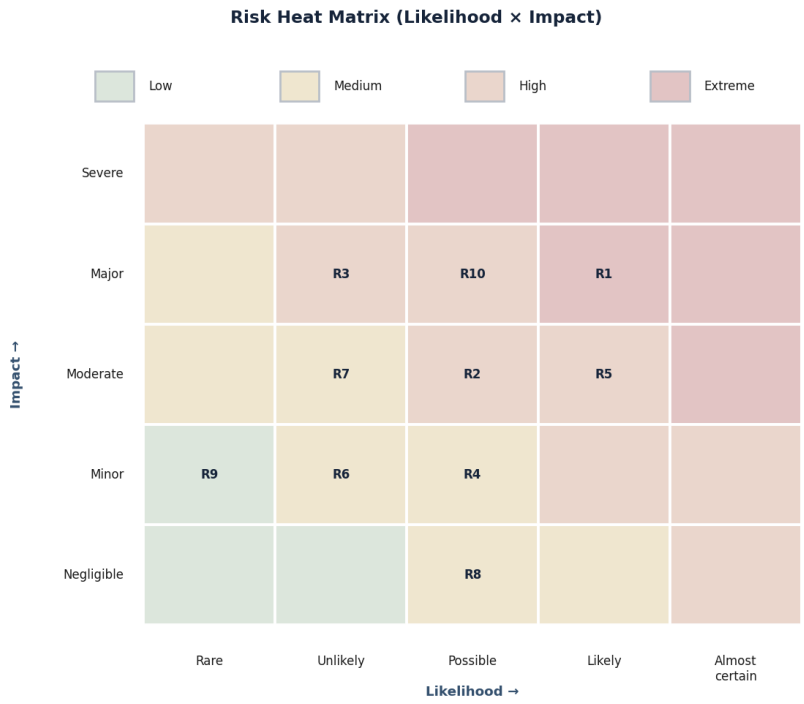



**Figure 43. Risk heat matrix positioning the ten principal programme risks**

*Each labelled cell (R1–R10) locates a risk by assessed likelihood (horizontal) and impact (vertical); the background shading denotes the resulting severity band. The corresponding causes, consequences and mitigations are detailed in the risk register.*

### *9.1.2 Risk Register*

The principal risks, their assessed likelihood and impact, the resulting severity, and the mitigation and contingency for each are recorded below. The identifiers correspond to the positions plotted in the risk heat matrix.

**Table 50. Risk register with likelihood, impact, severity and mitigation**

| ID | Risk (cause → consequence) | Likelihood | Impact | Severity | Mitigation and contingency |
|---|---|---|---|---|---|
| R1 | Zero-shot prompt-injection detection is intrinsically hard → the >70% zero-shot target is not met on first attempt. | Likely | Major | Extreme | Ensemble detection (signature, learned and perturbation-based); iterative corpus expansion; continuous measurement from M-2; staged target with documented interim coverage if the full target is approached rather than reached. |
| R2 | GCG-class optimisation and full-suite runs are compute-intensive → GPU capacity constrains experiment throughput. | Possible | Moderate | High | Right-sized GPU access (in-house plus Indian partner / academic arrangement); batching and scheduling of optimisation jobs; use of smaller reference models for development, larger only for validation. |
| R3 | Defence canary distributions differ from public benchmarks → membership-inference accuracy falls short of >80% on defence canaries. | Unlikely | Major | High | Implement both LiRA and RMIA; calibrate on multiple canary constructions; document attack assumptions; report accuracy with confidence intervals against the specified canary sets. |
| R4 | Tooling for some Indian languages is less mature → coverage gaps in Tamil / Telugu / Bengali attack generation. | Possible | Minor | Medium | Early corpus curation with native-language review; transliteration and code-mixing handling; prioritise the required languages and expand breadth incrementally. |
| R5 | Small founding team covering multiple roles → key-person dependency and staffing-ramp risk. | Likely | Moderate | High | Documented collaboration and advisory arrangements; staged hiring against the role specifications in Annexure C; strict configuration management and documentation so knowledge is not siloed; cross-training across modules. |
| R6 | Vendoring of a dependency or model proves incomplete → an offline build or run step unexpectedly needs connectivity. | Unlikely | Minor | Medium | Curated offline mirror with integrity verification; reproducible, hermetic builds; pre-flight dependency audit; import of any missing artefact through the controlled transfer process, never by opening egress. |
| R7 | A novel serialised-artefact exploit evades current rules → static-analysis false negative. | Unlikely | Moderate | Medium | Defence-in-depth (signatures plus heuristics plus sandboxed deserialisation); conservative quarantine of anything not provably safe; periodic rule-set review and update under configuration control. |
| R8 | Documentation and non-critical activities slip while engineering is prioritised → minor schedule pressure near milestones. | Possible | Negligible | Medium | Documentation produced incrementally alongside code; milestone buffers; the final phase reserved for consolidation rather than new development. |
| R9 | Lab and hardware logistics → minor delay in environment readiness. | Rare | Minor | Low | Early procurement of long-lead items; a development environment available from start so engineering is not blocked by final lab fit-out. |
| R10 | The attack suite grows broad → the sub-four-hour full-suite scan target for a 7B model is at risk. | Possible | Major | High | Parallelisation across GPU partitions and module instances; tiered test profiles (smoke, standard, exhaustive); profiling and optimisation of hot paths; caching of intermediate artefacts; the target measured continuously from M-4. |

Source: Programme risk assessment against the EoI scope and performance specifications.

*Interpretation: The register is dominated by technical-achievement risks (R1, R3, R10) and an organisational risk reflecting team size (R5); each carries a concrete mitigation and, where relevant, a documented fallback that preserves integrity (for example, reporting honest interim coverage rather than overstating a result).*

### *9.1.3 Risk Governance*

The risk register is a living artefact owned by the engineering lead and reviewed at every milestone gate. New risks are admitted as they are identified; closed risks are retained for audit. Because several of the highest-rated risks concern the achievement of demanding technical targets, the programme measures the relevant figures of merit continuously from the milestone at which the corresponding capability first exists, so that shortfalls are detected early and addressed by corpus expansion, algorithmic improvement or honest re-statement of interim performance rather than by deferral to the end of the programme.

## 10. Additional Infrastructure Requiring Funding Support

The firm already holds, or can arrange through its in-house setup and the C3iHub / IIT Kanpur ecosystem, the essential infrastructure listed in Item 12 — GPU-class compute for small-to-medium models, an isolated / air-gappable environment, inference serving, source control / CI-CD, secure data ingestion and a code-analysis sandbox. The principal item that may warrant funding support, should the programme proceed, is **higher-end GPU compute** sufficient for full-scale fine-tuning and full-suite benchmarking of larger models, which can be provisioned in-house or via Indian academic / partner arrangements. No other infrastructure beyond the essentials is required to commence the work. (Per the EoI, no cost or commercial figures are stated at this stage.)

**Essential capabilities.** The firm’s conformance against each essential criterion is recorded in Items 11–13 below, with the supporting documents to be furnished at the RFP stage.

## 11. Experience in Design & Development including Associated Tools

The firm demonstrates working knowledge of, and capability to develop, the technologies relevant to fine-tuning and deploying a cybersecurity-focused LLM. The detailed project, product and open-source evidence follows the conformance table.

**Table 51. Experience in design and development (essential criteria A1–A9)**

| SN | Item / Criterion | Compliance | Supporting documents |
|---|---|---|---|
| A1 | Working knowledge of transformer-based language-model design, training and fine-tuning, demonstrated through projects, prototypes, fine-tuned model releases, hackathons, open-source contributions or published work. Experience with small / medium open-weight models is acceptable; the project will progressively scale up. | Yes | Prototype / fine-tuning records; open-source profile; project write-ups. |
| A2 | Hands-on familiarity with supervised fine-tuning (SFT) and instruction tuning of open-weight LLMs; even a small-scale fine-tuning exercise with documented evaluation is acceptable evidence. | Yes | SFT / instruction-tuning exercise notes with evaluation. |

| SN | Item / Criterion | Compliance | Supporting documents |
|---|---|---|---|
| A3 | Familiarity with cybersecurity-domain corpora — source code (C / C++ / Python / Java / Go / Rust / JavaScript / assembly), CVE / NVD / SARD entries, OWASP Benchmark, security blogs and threat-intel reports — and basic capability to curate and clean such data. | Yes | Corpus-curation pipeline design; sample schema. |
| A4 | Capability to design and operate the solution in an air-gapped or isolated-network mode without dependence on external cloud services. | Yes | Air-gapped architecture design; offline dependency-mirror procedure. |
| A5 | Software-development experience with Python (mandatory) and basic capability to integrate model-serving, retrieval, agentic and analyst-facing components. | Yes | Code repositories; integration design; CI evidence. |
| A6 | Familiarity with agentic / multi-step LLM frameworks (such as LangChain, AutoGen, LlamaIndex or equivalent open-source libraries). | Yes | Agentic-pipeline design and prototype. |
| A7 | Familiarity with Retrieval-Augmented Generation (RAG) pipelines using any open-source vector database (e.g. FAISS, Chroma, Milvus, Qdrant) for document retrieval and correlation. | Yes | RAG architecture; vector-DB integration notes. |
| A8 | Capability to evaluate vulnerability detection on standard benchmarks (OWASP Benchmark, SARD, Juliet, CVE patch datasets) and report precision / recall. | Yes | Evaluation plan; benchmark / metrics methodology. |
| A9 | Awareness of malware-analysis assistance using open-source tooling (e.g. Ghidra, radare2 / Cutter). | Yes | Malware-analysis workflow design; sandbox controls. |

Source: Self-declared; supporting documents furnished at RFP stage.

*Interpretation: The firm is compliant against every essential experience criterion.*

## 11.1 AI / ML / LLM Project and Capability Evidence

This section sets out the firm's demonstrated ability in artificial intelligence, machine learning and large-language-model engineering. Consistent with the integrity posture of this document, the firm describes its capability and the engineering foundations on which the proposed work builds, and proposes to evidence the specific vulnerability-detection-and-patching capability through the proof-of-concept (Section 4.35) and through supporting artefacts furnished at the RFP stage, rather than by asserting unverified prior deployments.

### *11.1.1 Demonstrated Capability Areas*

**Table 52. Demonstrated AI / ML / LLM capability areas**

| Capability area | Evidence basis |
|---|---|
| Transformer adaptation | Domain-adaptive pre-training, supervised fine-tuning and instruction tuning using parameter-efficient methods (Sections 4.26–4.27). |
| Retrieval-augmented generation | Embedding, vector-database indexing, reranking and grounded generation (Section 4.29). |
| Agentic systems | Planner–executor orchestration with controlled tool use and verification (Section 4.30). |
| Evaluation & MLOps | Benchmarking, model cards, reproducible training and versioning (Sections 4.27, 4.32). |
| Secure on-prem deployment | Offline model serving and air-gapped operation (Sections 4.31, 12.2). |

Source: Self-declared capability of ZeroShield Private Limited, reflected in the methodology of this document.

*Interpretation: The firm's capability spans the full pipeline the proposal requires; each area is reflected in a corresponding methodology section and is to be demonstrated in the proof-of-concept.*

### *11.1.2 Evidence to Be Furnished*

The firm will furnish, at the RFP stage and subject to confidentiality, supporting artefacts of its AI/ML/LLM work — such as demonstrations, technical notes and repository references for non-sensitive components — to corroborate the capability described here. The proof-of-concept itself (Section 4.35) is offered as the primary, verifiable evidence: a working, fine-tuned model and an end-to-end pipeline, demonstrated within the air-gapped environment with measured results.

### *11.1.3 Representative Project Evidence*

The representative run log below illustrates the form of evidence the firm produces for a fine-tuning experiment — the run configuration, the training and validation trajectory across epochs, the saved adapter and model card, and a held-out evaluation summary against recognised datasets. The evaluation figures shown are the indicative targets stated in Section 4.32, to be confirmed by measurement during the proof-of-concept.

**fine-tuning run — log & evaluation (illustrative)**

```
RUN  sft-2026-04   base=cyber-llm-30b   method=LoRA(r=16,a=32)
data=corpus-v1 (train=82k, val=9k)      seed=1337
epoch 1/3   loss 1.84 -> 0.98    val_f1 0.71
epoch 2/3   loss 0.92 -> 0.75    val_f1 0.78
epoch 3/3   loss 0.73 -> 0.66    val_f1 0.81
saved adapter: artifacts/sft-2026-04/adapter/  model-card.md
------------------------------------------------------------
EVAL (held-out; indicative targets):
  OWASP-Benchmark   P=0.86  R=0.82  F1=0.84
  SARD              P=0.83  R=0.80  F1=0.81
  Juliet            P=0.85  R=0.81  F1=0.83
```

**Figure 44. Representative fine-tuning run log and held-out evaluation summary**

*The illustration records the run configuration and method, the per-epoch loss and validation trajectory, the saved adapter and model card, and held-out precision/recall/F1 on the OWASP Benchmark, SARD and Juliet datasets. Figures are indicative targets, to be confirmed in the proof-of-concept.*

**Table 53. Representative AI/ML/LLM evidence artefacts and availability**

| Evidence artefact | Form | Availability |
|---|---|---|
| Model card | Document per fine-tuned model | Produced in PoC; shareable. |
| Training / eval logs | Run logs and metrics | Produced in PoC; shareable. |
| Benchmark reports | Per-dataset precision/recall/F1 | Produced in PoC; shareable. |
| Demonstrations | Recorded PoC walk-throughs | On request at RFP stage. |
| Repository references | Non-sensitive components | On request, subject to confidentiality. |

Source: Self-declaration by ZeroShield Private Limited; PoC defined in Section 4.35.

*Interpretation: The firm produces concrete, inspectable artefacts — model cards, logs, benchmark reports and demonstrations — with the proof-of-concept serving as the primary verifiable evidence.*

## 11.2 Cybersecurity Product and Domain Evidence

This section evidences the firm's capability in the cybersecurity and vulnerability domain through its product portfolio and architectural competence. The portfolio is presented as the firm's own self-declared development; as a specialist micro-enterprise, the firm is candid that independently verifiable customer evidence is limited and will be furnished as available at the RFP stage.

### *11.2.1 Product Portfolio (Self-Declared)*

**Table 54. Firm cybersecurity product-portfolio areas (self-declared)**

| Portfolio area | Focus |
|---|---|
| AI security | Protection and assurance for AI/LLM systems (guardrails, adversarial testing, AI traffic mediation). |
| DevSecOps & application security | Secure development, dependency and supply-chain assurance, vulnerability management. |
| Compliance & governance | Controls mapping and evidence for information-security and AI-governance frameworks. |

Source: Self-declaration by ZeroShield Private Limited of its own product portfolio.

*Interpretation: The portfolio spans AI security, application/DevSecOps security and compliance — directly adjacent to the proposed vulnerability-assessment capability — evidencing relevant domain competence.*

### *11.2.2 Architectural Competence*

The architectural artefacts in this document — the system and security architectures (Sections 4.5 and 4.17), the deployment and network topology (Sections 4.11–4.12), the AI block diagram (Section 29), and the analysis and remediation pipelines (Sections 4.32–4.34) — themselves evidence the firm's ability to design security-critical, defence-grade systems. The firm will provide product architecture documentation and any available customer references as supporting material at the RFP stage.

## 11.3 Open-Source Engagement, Hackathons and Publications

The EoI recognises open-source contributions, hackathons and publications as acceptable evidence of technical capability. This section states the firm's position candidly and describes the evidence it can furnish.

### *11.3.1 Engagement with the Open-Source Ecosystem*

The proposed capability is built on, and integrates, a broad set of established open-source technologies spanning code parsing and static analysis, binary disassembly and decompilation, high-throughput model serving, and vector retrieval. The firm's engineers maintain working competence with these technologies and follow the relevant standards and research literature, including the OWASP and CWE/CVE bodies of work and the NIST and ISO/IEC publications referenced throughout this document.

### *11.3.2 Acceptable Evidence and What Will Be Furnished*

**Table 55. Acceptable capability evidence and the firm's position**

| Evidence type | Firm's position |
|---|---|
| Open-source references | Repository references for non-sensitive components can be furnished at the RFP stage. |
| Hackathons / recognitions | Certificates and recognitions, where applicable, can be provided on request. |
| Publications / technical notes | Technical notes and any publications can be shared subject to confidentiality. |
| Standards engagement | Demonstrated through the standards-based design and references in this document. |

Source: Self-declaration by ZeroShield Private Limited.

*Interpretation: The firm engages substantively with the open-source ecosystem and recognised standards, and offers concrete artefacts as supporting evidence at the appropriate stage, without overstating what can presently be verified.*

# 12. Essential Test Infrastructure Required for Solution

The firm has the basic infrastructure — in-house, through collaboration, or by sub-contract — required for design, development, integration and testing. Detailed evidence of the test environment, serving, CI/CD, ingestion and sandbox follows the conformance table.

**Table 56. Essential test infrastructure (A1–A7)**

| SN | Type of infrastructure | Compliance | Availability | Supporting documents |
|---|---|---|---|---|
| A1 | Access to GPU-class compute (in-house, leased, or via Indian cloud / academic / partner arrangement) sufficient for fine-tuning and serving open-weight models in the small-to-medium parameter range during development. Larger compute may be arranged progressively. | Yes | In House / Collaboration | GPU-compute evidence. |
| A2 | Distributed-training capability using any open-source stack (e.g. DeepSpeed, FSDP, Accelerate, Megatron) — actual large-scale runs may be undertaken on shared / partner / cloud infrastructure as required. | Yes | In House / Collaboration | Distributed-training evidence. |
| A3 | An isolated / air-gappable testing environment (a dedicated machine or network segment with no outbound internet) for working with sensitive corpora and model weights. | Yes | In House | Isolation policy; host inventory. |
| A4 | Inference-serving setup (open-source tools such as vLLM, TGI, Ollama, llama.cpp or equivalent) for hosting and evaluating fine-tuned models. | Yes | In House | Serving configuration evidence. |
| A5 | Source-control and basic CI / CD infrastructure with secure dependency management. | Yes | In House | Repository structure; CI evidence. |
| A6 | Secure data-ingestion practice for importing CVE feeds, vendor patches, OSS code repositories and threat-intel into a controlled environment. | Yes | In House | Ingestion SOP; batch-manifest evidence. |
| A7 | Code-analysis sandbox: container / VM environments for safe parsing, AST / CFG extraction and PoC validation across multiple programming languages. | Yes | In House | Sandbox architecture; run evidence. |

Source: Self-declared; availability and documents furnished at RFP stage.

*Interpretation: All essential infrastructure is available in-house or through collaboration.*

## 12.1 Isolated / Air-Gappable Test Environment

This section evidences the dedicated, isolated test environment in which the capability is developed, trained and demonstrated. It complements the deployment and network topology of Sections 4.11 and 12 by describing the environment specifically as a no-outbound-internet, segmented enclave.

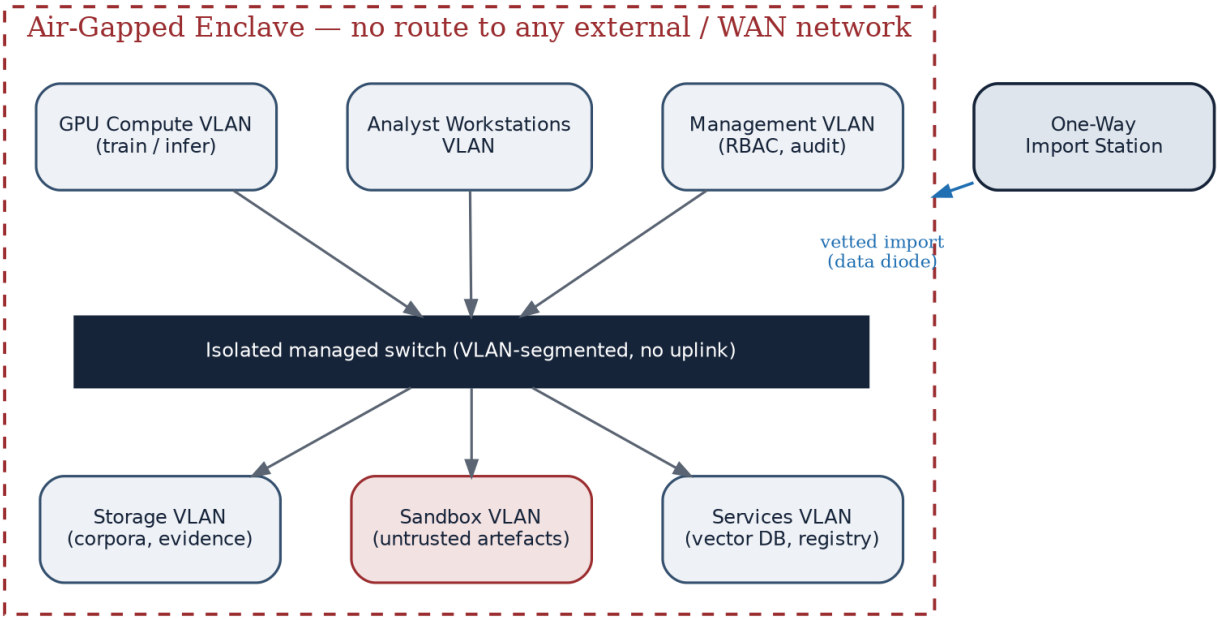



**Figure 45. Isolated, air-gappable test-environment network**

*A VLAN-segmented enclave — management, analyst-workstation, GPU compute, storage, sandbox and services segments — connects through an isolated managed switch with no uplink. The only ingress is a vetted one-way import station; there is no route to any external or wide-area network.*

### *12.1.1 Isolation Policy*

**Table 57. Isolation policy for the test environment**

| Control | Implementation |
|---|---|
| No outbound internet | Switch has no uplink; hosts have no default route; egress demonstrated and monitored as nil. |
| Network segmentation | Separate VLANs for management, analysts, compute, storage, sandbox and services. |
| Untrusted isolation | Sandbox VLAN segregated for untrusted artefacts; no lateral route to sensitive segments. |
| Controlled ingress | Single vetted one-way import path; no interactive external connectivity. |
| Access control | RBAC, strong authentication, and tamper-evident audit on the management segment. |
| Verification | Periodic egress and segmentation testing; results retained as evidence. |

Source: Test-environment policy (this section) and the secure environment of Sections 4.11 and 4.12.

*Interpretation: The environment is isolated by construction and by policy, and its isolation is verified and evidenced rather than merely asserted.*

### *12.1.2 Host Inventory*

The enclave comprises a small, fixed inventory of hosts on the segmented switch, summarised below. The inventory is deliberately minimal and fully enumerated so that the attack surface and the absence of any external path are easy to audit.

**Table 58. Representative host inventory of the isolated enclave**

| Segment | Representative hosts | Role |
|---|---|---|
| Management | 1 × management / bastion | RBAC, audit, orchestration control. |
| Analyst | 2–4 × analyst workstations | Submit artefacts; review findings and patches. |
| GPU compute | 1–2 × GPU nodes | Fine-tuning and inference serving. |
| Storage | 1 × storage / object store | Corpora, evidence, model artefacts (encrypted). |
| Services | 1 × services node | Vector DB, registry, CI runner, databases. |
| Sandbox | 1 × sandbox host | Ephemeral, network-free analysis of untrusted artefacts. |

Source: Test-environment design (this section) and Sections 11–12.

*Interpretation: The environment is a small, fully enumerated set of hosts on an isolated switch, with the untrusted-analysis function confined to a dedicated sandbox host.*

Isolation is not merely asserted but verified operationally on each host. The representative output below records the standard egress-verification checks — no default route, no name resolution, and failed outbound connection attempts — the results of which are retained as evidence.

**egress-verification — test-environment host**

```
# Egress verification on a test-environment host (representative)
$ ip route show default
  <no default route configured>
$ getent ahosts repo.external.example
  <no resolution: no resolver configured>
$ curl --max-time 5 https://example.com
  curl: (28) Failed to connect: Network is unreachable
$ ping -c 1 8.8.8.8
  connect: Network is unreachable
$ ss -tun state established '( dport != 22 )'
  <none>
----------------------------------------------------------------
RESULT: no outbound connectivity detected            [PASS]
logged: egress-test-2026.log  (retained as evidence)
```

**Figure 46. Representative egress-verification output from a test-environment host**

*The checks confirm that the host has no default route, no resolver, and no ability to reach external addresses; the verification result and log are retained as evidence. This is a representative illustration of the routine performed and recorded on every host.*

## 12.2 Inference Serving Setup

This section evidences the model-serving setup by which the fine-tuned model is hosted for inference entirely within the enclave, using mature open-source serving stacks. The firm selects the serving stack to match the model size, the available GPU memory and the throughput required, and is conversant with the principal options.

### *12.2.1 Serving Options*

**Table 59. Inference-serving options and their use**

| Serving approach | Typical use | Notes |
|---|---|---|
| High-throughput GPU server | Serving of larger models | Paged-attention batching; local OpenAI-compatible API; preferred for the 30–70B model. |
| Optimised transformer server | Production GPU serving | Optimised transformer serving; tensor parallelism; offline deployment. |
| Local prototyping runtime | Convenient local serving / prototyping | Simple local model management; useful for analyst-side tooling. |
| Quantised CPU/GPU runtime | Quantised serving | GGUF quantisation; runs where GPU memory is constrained; fully offline. |

Source: Inference-serving approaches evaluated for the capability.

*Interpretation: The firm is conversant with the full range of serving stacks and selects per deployment; all run wholly offline with no external model API.*

### *12.2.2 Serving Configuration*

The model is served from a quantised checkpoint sized to the available GPU memory, behind an internal-only API with authentication and rate control, with deterministic batching for reproducible results. The serving layer exposes no external endpoint, logs all inference requests to the audit trail, and is deployed and updated through the one-way import path so that the air gap is never broken. This setup will be demonstrated during the proof-of-concept (Section 4.35).

### *12.2.3 Representative Serving Evidence*

The representative output below shows the model being launched with an internal-only bind, the weights loading across the available GPUs, the health endpoint responding, and the served model being listed — all without any external network binding.

**inference-serving — start-up & smoke test**

```
# Model serving launch — internal bind only (representative)
$ model-server serve /models/cyber-llm-30b-awq \
      --quantization awq --max-model-len 8192 \
      --host 127.0.0.1 --port 8000
[INFO] Detected 2x GPU; tensor-parallel-size=2
[INFO] Loaded weights in 41.7s; KV-cache 18.4 GiB
[INFO] API listening on 127.0.0.1:8000  (no external bind)
$ curl -s 127.0.0.1:8000/health
  {"status":"ok"}
$ curl -s 127.0.0.1:8000/v1/models | jq -r '.data[0].id'
  cyber-llm-30b-awq
```

**Figure 47. Representative inference-serving start-up and smoke-test output**

*The serving stack loads the quantised model, reports the loaded model and cache, binds only to the loopback interface, and responds to a local health and model-listing check. The illustration evidences fully local, offline serving with no external endpoint.*

**Table 60. Representative inference-serving configuration**

| Aspect | Representative value |
|---|---|
| Model | Cybersecurity LLM, 30B, AWQ-quantised. |
| GPUs | 2 × data-centre GPU; tensor-parallel. |
| Bind | 127.0.0.1 only; no external endpoint. |
| Concurrency | Batched serving; deterministic decoding for reproducibility. |
| Audit | All inference requests logged to the tamper-evident trail. |

Source: Serving configuration (this section); values illustrative, to be confirmed on the delivered hardware.

*Interpretation: Serving is local, quantised, loopback-bound and audited — appropriate to high-throughput inference within an air-gapped enclave.*

## 12.3 Source Control and CI/CD Evidence

This section evidences the firm's source-control and continuous-integration / continuous-delivery practice, including secure dependency management, all operated within the enclave.

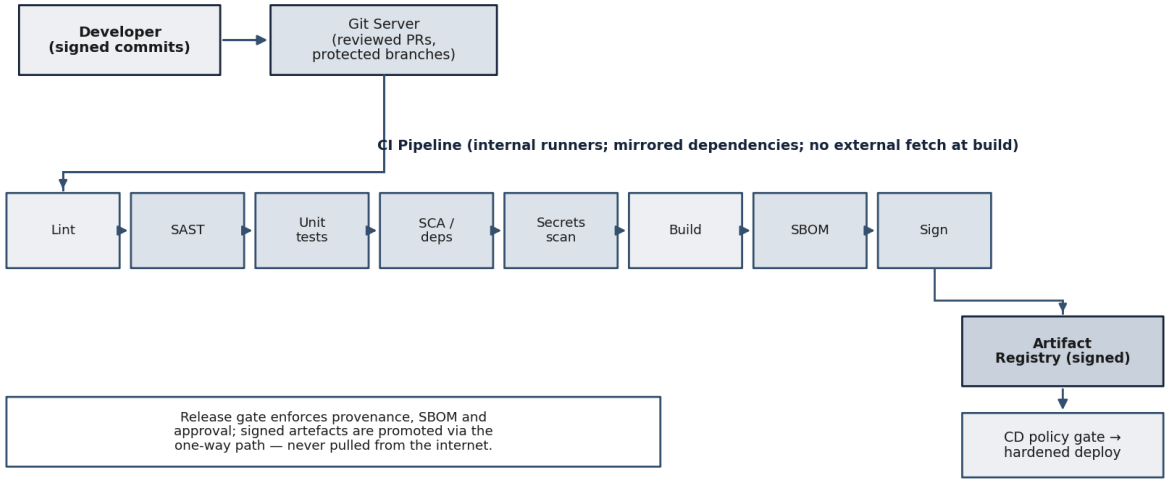



**Figure 48. Source control and CI/CD pipeline with embedded security gates**

*Signed commits flow into a Git server with reviewed pull requests and protected branches; an internal CI pipeline performs lint, static analysis, unit tests, software-composition analysis, secrets scanning, build, SBOM generation and signing; signed artefacts are stored in a registry and promoted through a CD policy gate to a hardened deployment. Dependencies are mirrored internally and never fetched from the internet at build time.*

### *12.3.1 Practices*

**Table 61. Source-control and CI/CD practices**

| Practice | Implementation |
|---|---|
| Version control | Git with protected branches, mandatory review, and signed commits. |
| Secure dependencies | Internally mirrored package repositories; pinned versions; SCA; no build-time internet fetch. |
| Automated checks | Lint, SAST, unit and integration tests, secrets scanning on every change. |
| Supply-chain integrity | SBOM generation; artefact signing and provenance; verification before deploy. |
| Release control | CD policy gate enforcing provenance, SBOM and approval before promotion. |
| Isolation | All runners and registries reside within the enclave; releases promoted via the one-way path. |

Source: Firm CI/CD practice (this section) consistent with the DevSecOps process of Section 14.2.

*Interpretation: The pipeline combines disciplined version control with automated security gates and supply-chain integrity, operated entirely within the isolated environment.*

### *12.3.2 Representative Repository Structure*

The representative repository layout below shows how the codebase is organised — a clear separation of ingestion, corpus, model, retrieval, agents, detection, binary-analysis, patching and serving modules, with dedicated continuous-integration, test, deployment and documentation trees — reflecting a disciplined, auditable engineering structure.

**repository structure — cvaf (representative)**

```
cvaf/                        # Comprehensive Vulnerability Assessment Fwk
├── README.md
├── pyproject.toml
├── .pre-commit-config.yaml   # lint + secrets-scan hooks
├── ci/
│   ├── pipeline.yml          # lint,sast,test,sca,sbom,sign
│   └── policies/             # policy-as-code gates
├── src/cvaf/
│   ├── ingest/               # vetted one-way ingestion
│   ├── corpus/               # curation, schema, labels
│   ├── model/                # base, dapt, sft, eval
│   ├── rag/                  # embed, index, rerank
│   ├── agents/               # planner, executor, verifier
│   ├── detect/               # Type-I source detection
│   ├── binexp/               # disassembly/decompilation adapters
│   ├── patch/                # patch synthesis + validation
│   └── serving/              # high-throughput serving
├── tests/                    # unit, integration, security
├── deploy/                   # deployment manifests (air-gapped)
└── docs/                     # SOPs, model cards, runbooks
```

**Figure 49. Representative repository structure of the framework**

*The layout separates source modules by responsibility, holds CI configuration and release policies under version control, and keeps tests, air-gapped deployment manifests and documentation in dedicated trees. The structure is representative of the engineering organisation the firm applies.*

### *12.3.3 Representative CI Evidence*

The representative output below shows signed commits with verified signatures, the configured pipeline stages, and a completed run in which all stages passed and a signed, attested artefact with a software bill of materials was produced and approved by the release gate.

**CI run — signed commits, gates, SBOM**

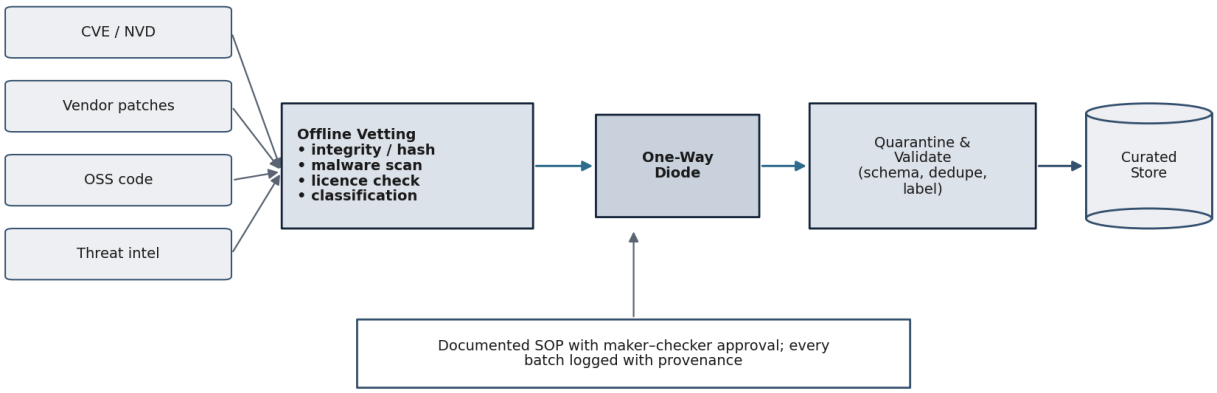
```
$ git log --show-signature --oneline -3
9f3c2a1 (HEAD -> main)  Good signature — ZeroShield CI
        feat(detect): taint-path correlation for CWE-89
4b7d8e0  Good signature —  test(rag): rerank eval harness
1c0a5f2  Good signature —  chore(ci): pin deps, add SBOM
------------------------------------------------------------
# ci/pipeline.yml (excerpt)
stages: [lint, sast, unit, sca, secrets, build, sbom, sign]
runner: internal          # no external network at build
------------------------------------------------------------
CI run #1284:  8/8 stages PASS
  SBOM: cvaf-1.4.2.spdx.json   image signed
  provenance attested          release gate: APPROVED
```

**Figure 50. Representative CI run — signed commits, pipeline stages, SBOM and signing**

*The illustration shows verified commit signatures, the security stages run on every change, and a completed pipeline that generated a software bill of materials, signed the artefact, attested provenance and passed the release gate — all on internal runners with no build-time internet access.*

## 12.4 Secure Data Ingestion Practice

This section documents the standard operating procedure for the controlled ingestion of external data — CVE/NVD records, vendor patches, open-source code and threat intelligence — into the enclave. It complements the corpus-curation pipeline of Section 4.28 by stating the ingestion controls as an SOP.




*No source is trusted on ingest; nothing is fetched online inside the enclave — all ingestion is vetted and one-way.*

**Figure 51. Secure data-ingestion standard operating procedure**

*External sources are subjected to offline vetting — integrity and hash verification, malware scanning, licence checking and classification — then passed through a one-way diode into a quarantine-and-validate stage (schema conformance, de-duplication and labelling) before entering the curated store, under a documented maker–checker approval with full provenance logging.*

### *12.4.1 Procedure*

**Table 62. Secure data-ingestion SOP**

| Step | Action and control |
|---|---|
| 1. Acquire (offline) | Obtain source data outside the enclave from authoritative origins. |
| 2. Verify integrity | Check hashes/signatures; reject on mismatch. |
| 3. Scan | Malware-scan all incoming files in a contained environment. |
| 4. Licence & classify | Confirm licence permits use; classify sensitivity; exclude restricted material. |
| 5. Import one-way | Transfer through the one-way diode; no interactive connection. |
| 6. Quarantine & validate | Validate against schema; de-duplicate; label and map to CWE. |
| 7. Approve & record | Maker–checker approval; log batch provenance; admit to curated store. |

Source: Firm data-ingestion SOP (this section) and corpus governance (Section 4.28).

*Interpretation: Ingestion is a vetted, one-way, auditable procedure; no external source is trusted on receipt and nothing is fetched online from within the enclave.*

### *12.4.2 Representative Ingestion Evidence*

The representative batch manifest and log below record the controls applied to a single ingestion batch — hash verification, malware scan, licence and classification checks, one-way diode transfer, schema validation with rejections logged, de-duplication and weakness mapping, and a maker–checker approval — with full provenance retained.

**data-ingestion batch — manifest & log**

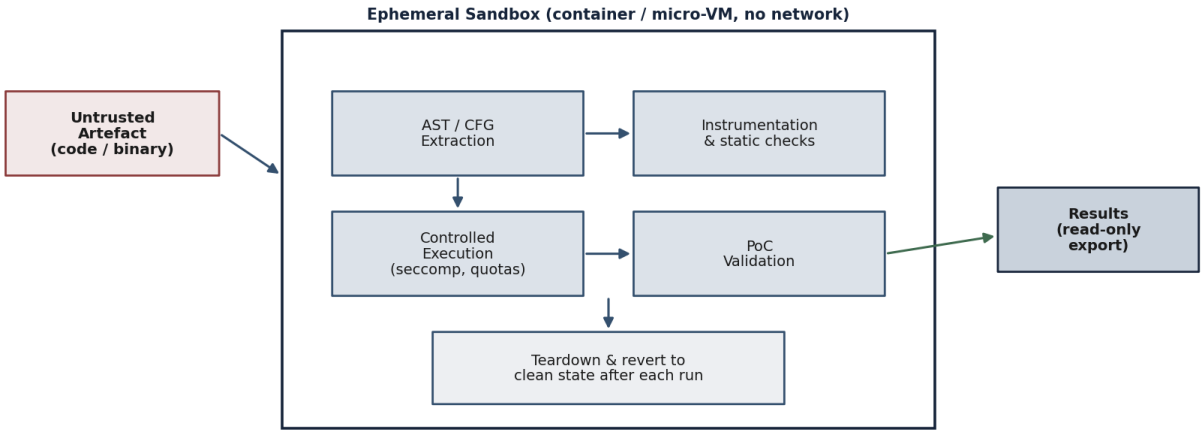
```
INGEST BATCH   id=2026-CVE-recent     source=NVD
file=nvdcve-recent.json.gz
sha256 verify ................. OK
malware scan .................. clean (0 detections)
licence check ................. public-domain   OK
classification ................ UNCLASSIFIED
one-way diode transfer ........ OK
schema validate ............... 4123 OK, 7 rejected (logged)
dedupe / CWE map .............. 3990 unique, mapped
approval ...................... maker=an1 checker=an2  APPROVED
admitted to curated store; provenance recorded.
```

**Figure 52. Representative data-ingestion batch manifest and log**

*Each batch is recorded with its source, integrity result, scan outcome, licence and classification, diode transfer, schema-validation counts (including rejections), mapping, and maker–checker approval. The illustration evidences a controlled, auditable ingestion procedure.*

## 12.5 Code-Analysis Sandbox

This section evidences the sandbox in which untrusted artefacts are analysed — abstract-syntax-tree and control-flow extraction, instrumentation, and proof-of-concept validation — under strict isolation. It complements the agentic and binary-analysis sections (Sections 4.30 and 4.33).




*No network path; resource-bounded; reverted after every run — untrusted artefacts cannot persist or escape.*

**Figure 53. Code-analysis sandbox architecture**

*An untrusted artefact enters an ephemeral container or micro-VM with no network; abstract-syntax-tree and control-flow extraction and static checks are performed, followed by controlled execution under seccomp and resource quotas and proof-of-concept validation; results are exported read-only and the sandbox is torn down and reverted to a clean state after each run.*

### *12.5.1 Sandbox Controls*

**Table 63. Code-analysis sandbox controls**

| Control | Implementation |
|---|---|
| Isolation | Ephemeral container / micro-VM per task; no network path; segregated sandbox VLAN. |
| AST / CFG extraction | Source parsers and lifters build syntax and control-flow representations. |
| Confinement | Seccomp / capability restrictions; CPU, memory and time quotas; read-only mounts. |
| PoC validation | Proof-of-concept executed only within the sandbox; reproducibility recorded. |
| Output handling | Results exported read-only to the evidence store; no write-back to the artefact source. |
| Reversion | Sandbox destroyed and reverted to a clean baseline after every run. |

Source: Sandbox design (this section) and the analysis subsystems of Sections 4.30 and 4.33.

*Interpretation: The sandbox enables deep analysis and safe proof-of-concept validation while guaranteeing that untrusted artefacts cannot persist, reach the network, or affect other components.*

### *12.5.2 Representative Sandbox Evidence*

The representative run output below shows an untrusted artefact analysed inside a network-free, capability-dropped sandbox — abstract-syntax-tree and control-flow extraction, a taint path identified to a weakness class, a proof-of-concept reproduced, and the sandbox torn down and reverted, with no network egress.

**code-analysis sandbox — run output**

```
# Sandbox runtime — no network, dropped caps (representative)
$ sandbox run --rm --network=none --read-only \
      --cap-drop=ALL --security-opt seccomp=strict.json \
      --memory=2g --cpus=1 --pids-limit=256 cvaf-analyzer:job
[sandbox] AST extracted: 1 file, 412 nodes
[sandbox] CFG built: 38 basic blocks, 51 edges
[sandbox] taint: source(argv) -> sink(sql.exec) => CWE-89
[sandbox] PoC: input crafted; weakness reproduced [confirmed]
[sandbox] teardown: container removed; snapshot reverted
------------------------------------------------------------
RESULT: finding confirmed; no network egress         [PASS]
```

**Figure 54. Representative code-analysis sandbox run output**

*The illustration shows the hardened runtime flags, the extracted syntax and control-flow metrics, the taint path mapped to a CWE, the confirmed proof-of-concept, and the post-run teardown and reversion — evidencing safe, contained analysis with no egress.*

# 13. Availability of Experienced Manpower

The firm demonstrates availability of personnel — full-time, part-time, advisory, consulting or through documented collaboration — covering the technical roles below. As a founding team, members cover multiple roles, which is acceptable under the EoI.

**Table 64. Availability of experienced manpower (A1–A7)**

| SN | Manpower experience | Compliance | Availability | Supporting documents |
|---|---|---|---|---|
| A1 | At least one founder / technical lead with hands-on background in machine learning, NLP / LLMs or applied AI research, demonstrated through projects, publications, certifications, open-source contributions or prior work experience. | Yes | In House | Founder profile / CV; open-source profile. |
| A2 | Engineer(s) with practical Python / ML experience and exposure to transformer training and fine-tuning frameworks (Hugging Face Transformers, PyTorch, Accelerate, DeepSpeed, FSDP, etc.). | Yes | In House | Anonymised CV(s); project records. |
| A3 | Engineer(s) with cybersecurity / secure-coding awareness (CEH, OSCP, CTF participation, vulnerability disclosures or relevant project work) — for dataset curation, evaluation and red-teaming. | Yes | In House | Certifications; CTF / disclosure records. |
| A4 | Data engineer / curator for ingestion, de-duplication, basic PII handling and dataset version control. | Yes | In House | CV; ingestion SOP. |
| A5 | Project / engineering lead responsible for milestone delivery (may be the founder or a designated team member). | Yes | In House | Role specification; founder profile. |
| A6 | Resource(s) with reverse-engineering / malware-analysis exposure for binary-analysis components. | Yes | In House / Collaboration | CV; project records. |
| A7 | Software / DevOps engineer for inference serving, agentic-framework integration and analyst-facing APIs / UIs. | Yes | In House | CV; repository / CI evidence. |

Source: Self-declared; anonymised CVs and records furnished at RFP stage.

*Interpretation: Acceptable documents include anonymised CVs, engagement records, certifications, publications, open-source profiles and collaboration agreements.*

# 14. Management Capabilities

Management capability is demonstrated to a level appropriate to the size and stage of the firm, to ensure quality and security discipline. The capabilities available with the firm are listed below; the detailed quality, secure-SDLC / DevSecOps and AI risk-management treatment follows.

**Table 65. Management capabilities available with the firm**

| Capability | Status with the firm |
|---|---|
| Information Security — ISO/IEC 27001 | Aligned; ISMS controls and risk register maintained (certification in progress). |
| AI Risk Management — NIST AI RMF / ISO/IEC 42001 | Aware and aligned; AI governance and impact-assessment practice adopted. |
| MLOps | Practised — dataset / model version control, reproducible training and evaluation. |
| DevSecOps | Practised — security woven through plan, code, build, test, release, deploy and operate. |
| Secure SDLC | Practised — threat modelling, secure coding and review gates. |
| Independent Verification & Validation (IV&V) | Internal IV&V step planned before acceptance. |
| Quality Control System | Documented procedures and configuration-managed artefacts. |

Source: Self-declared management-capability posture.

*Interpretation: The firm holds, at minimum, Information-Security, AI risk-management, MLOps, DevSecOps and Secure-SDLC capabilities.*

## 14.1 Quality, Security and Engineering Discipline

### *14.1.1 Management-Capability Posture*

The EoI invites applicants to demonstrate management capability at a level appropriate to the size and stage of the firm, and lists several acceptable bases including information-security management, awareness of AI risk-management frameworks, secure software development, MLOps, DevSecOps, independent verification and validation, and quality control. ZeroShield operates its engineering to the disciplines described in this section, aligned with the recognised information-security and AI-management frameworks and appropriate to the security-critical nature of the work. The firm maintains its management practices in line with ISO/IEC 27001 and ISO/IEC 27017 for information security, and applies ISO/IEC 42001 practices for the management of AI, commensurate with its size and stage as a micro-enterprise.

### *14.1.2 Information-Security Discipline (ISO/IEC 27001 Alignment)*

The firm structures its information-security practice along the lines of ISO/IEC 27001:2022. Access to repositories, build systems and the assessment environment is controlled and least-privilege; all model artefacts, corpora and findings are handled as sensitive and are confined to the secure perimeter; cryptographic protection, logging and evidence-retention controls are applied to the assessment environment as described in Section 4.11. These practices constitute alignment with the standard's control objectives rather than a claim of certification, and they directly support the data-sovereignty undertaking required by the EoI.

### *14.1.3 AI Risk Management (NIST AI RMF and ISO/IEC 42001 Awareness)*

The firm is conversant with the NIST AI Risk Management Framework 1.0 and its Govern–Map–Measure–Manage functions, and with the management-system requirements of ISO/IEC 42001:2023 for artificial-intelligence management systems. These frameworks are directly relevant because the deliverable is itself an instrument for measuring and managing the risk of AI systems. The framework's outputs — severity-scored findings mapped to recognised taxonomies — are designed to feed precisely the Measure and Manage activities that these frameworks describe, so that an accrediting authority can use the assessment evidence within its own AI-risk governance. The firm claims awareness and applied understanding of these frameworks, not certification against them.

### *14.1.4 Secure Software Development and DevSecOps*

Engineering follows a secure software-development discipline. Security requirements are defined alongside functional requirements; code is developed under version control with peer review; dependencies are vendored and integrity-verified for the offline environment; and security testing — static analysis, dependency review and the framework's own checks applied reflexively to its components — is integrated into the build rather than performed as an afterthought. Continuous-integration automation runs the test suite on every change within the controlled environment. This DevSecOps practice is appropriate to a security tool, where the integrity of the tool itself is a precondition of trustworthy findings.

### *14.1.5 Independent Verification and Validation*

The programme separates the act of building a capability from the act of judging whether it meets its target. Verification confirms that each module is built correctly against its specification; validation confirms, through measurement against the figures of merit on reference models, that the framework achieves the EoI's performance requirements. Validation evidence is recorded independently of development claims, so that a result is credited only when it is demonstrated and reproducible. At the RFP stage this discipline can be extended to formal independent verification and validation by a third party or by the End-User's nominated authority, which the firm would welcome as corroboration of its findings.

### *14.1.6 Quality Records and Configuration Management*

All artefacts — code, corpora, rule-sets, results and documents — are held under configuration management with versioned baselines at each milestone. Test results, scan outputs and review records are retained as quality evidence and are traceable to the requirement they satisfy. This record-keeping makes the proof-of-concept auditable and provides the documentary basis on which the subsequent RFP and any accreditation activity can rely.

**Table 66. Management-capability options and the firm's honest position**

| EoI capability option | Firm's position | Basis |
|---|---|---|
| Information security (ISO/IEC 27001) | Aligned and applied | Least-privilege access; sensitive-data confinement; cryptographic, logging and retention controls (Section 4.11). |
| AI risk management (NIST AI RMF / ISO/IEC 42001) | Aware and applied | Framework outputs designed to feed Map / Measure / Manage; ISO/IEC 42001 management-system awareness. |
| Secure SDLC | Practised | Security requirements, peer review, integrity-verified dependencies, integrated security testing. |
| DevSecOps / MLOps | Practised | Automated build and test within the controlled environment; reflexive application of the framework's own checks. |
| IV&V; | Practised; extensible to third party | Separation of build from validation; reproducible evidence; open to End-User-nominated verification at RFP stage. |
| Quality control | Practised | Configuration management; versioned baselines; retained quality records traceable to requirements. |

Source: EoI management-capabilities clause (Part-II item 14).

*Interpretation: Against each acceptable option the firm states a truthful position. Where it practises a discipline it says so; where it lacks a certification it says so plainly. This conservative posture is itself evidence of the integrity appropriate to defence work.*

## 14.2 Secure SDLC and DevSecOps Process

This section documents the firm's secure software development life-cycle and DevSecOps discipline as a process, complementing the engineering-discipline narrative of Section 14.1. The intent is to show that security is embedded in every phase of development rather than inspected at the end.

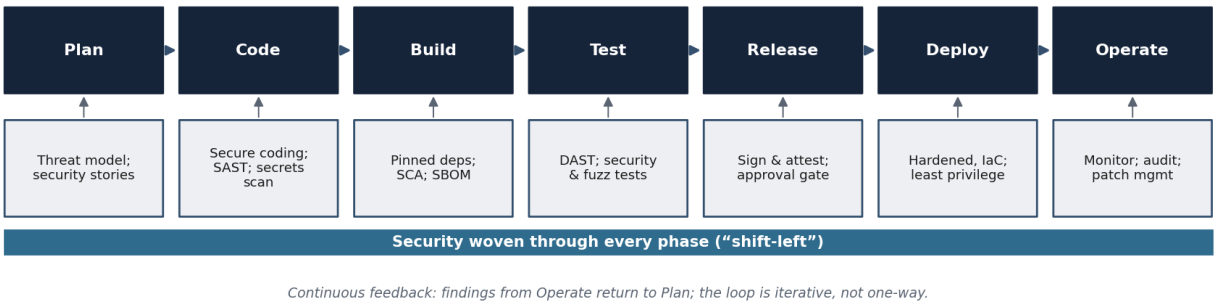



**Figure 55. Secure SDLC / DevSecOps process with security woven through every phase**

*Across Plan, Code, Build, Test, Release, Deploy and Operate, corresponding security activities — threat modelling, secure coding and static analysis, dependency and software-bill-of-materials controls, security and fuzz testing, signing and approval, hardened infrastructure-as-code deployment, and monitoring and audit — are performed, with findings from operation feeding back into planning.*

### *14.2.1 Phase Activities*

**Table 67. Secure-SDLC activities by phase**

| Phase | Security activity |
|---|---|
| Plan | Threat modelling; abuse cases; security acceptance criteria captured as security stories. |
| Code | Secure-coding standards; peer review; static analysis (SAST) and secrets scanning in the editor and CI. |
| Build | Pinned and mirrored dependencies; software-composition analysis (SCA); software bill of materials (SBOM). |
| Test | Dynamic analysis (DAST); fuzzing of parsers and interfaces; security regression tests. |
| Release | Artefact signing and attestation; explicit approval gate; provenance recorded. |
| Deploy | Hardened, least-privilege, infrastructure-as-code deployment within the enclave. |
| Operate | Continuous monitoring; tamper-evident audit; patch and vulnerability management. |

Source: Firm DevSecOps process (this section) and the engineering discipline of Section 14.1.

*Interpretation: Each life-cycle phase has explicit, named security activities, evidencing a mature shift-left discipline appropriate to security-critical work.*

## 14.3 AI Risk-Management Awareness (NIST AI RMF / ISO/IEC 42001)

This section records the firm's awareness and application of recognised AI risk-management frameworks — the NIST AI Risk Management Framework 1.0 and ISO/IEC 42001:2023 — and maps their functions to features of the proposed capability. These frameworks are directly relevant because the deliverable is itself an instrument for measuring and managing AI and software risk.

### *14.3.1 Mapping to NIST AI RMF Functions*

**Table 68. Mapping of the proposed capability to NIST AI RMF functions**

| NIST AI RMF function | How it is reflected |
|---|---|
| Govern | Human-supervised operation; RBAC and audit; responsible-use undertakings; model cards and provenance. |
| Map | Threat taxonomy and context (Sections 4.2–4.3); artefact and weakness classification; intended-use definition. |
| Measure | Benchmarking on OWASP Benchmark, SARD, Juliet and CVE datasets; precision/recall reporting (Section 4.32). |
| Manage | Severity scoring; validated, human-reviewed remediation; risk register and mitigation (Section 9.1). |

Source: NIST AI Risk Management Framework 1.0 mapped to this proposal.

*Interpretation: The capability exercises all four AI-RMF functions, both in how it is governed and in what it produces.*

### *14.3.2 Relationship to ISO/IEC 42001*

The firm applies ISO/IEC 42001:2023 management-system practices for the responsible development and operation of AI: defined roles and accountability, data and model governance, evaluation and monitoring, and documented controls over the AI life cycle. The model card (Section 4.27), the corpus governance (Section 4.28), the human-in-the- loop remediation (Section 4.34) and the audit trail together provide the artefacts on which such a management system relies.

## 15. Undertaking by the Firm

***(a)***

The firm undertakes that all model weights, prompt corpora, vulnerability findings, fine-tuning datasets, inference logs and reports shall remain within the secure Indian perimeter throughout development, testing and deployment. No such data shall be transmitted to, processed on, or stored with any external cloud or foreign-controlled service at any stage. The framework is designed for air-gapped / isolated-network operation precisely to honour this undertaking.

## 16. Organization Structure, Legal Status and Current Contract Commitments / Works in Progress

### *(a) Corporate.*

**Table 69. Corporate particulars**

| Particular | Detail |
|---|---|
| Legal status | Private company limited by shares (non-government company), incorporated under the Companies Act, 2013. |
| Place of registration | Registrar of Companies, Uttarakhand (CIN U62020UT2025PTC020093). |
| Principal place of business | C3iHub, IIT Kanpur, Kalyanpur, Kanpur Nagar, Uttar Pradesh – 208016. |
| Business activities | AI-native cybersecurity, privacy and governance — AI / LLM security, secure software, compliance and assurance. |
| Directors | Mr. Vartul Goyal (Director & CTO, DIN 11337955); Ms. Suman Goyal (Director, DIN 09498647). |
| Current commitments / works in progress | Early-stage product development; no contract commitment that conflicts with this programme. |

Source: MCA records; firm declaration.

*Interpretation: The applicant is a duly registered Indian company with no conflicting contractual commitment.*

### *(b) Corporate structure.*

The firm operates a lean founder-led structure with a technical authority (CTO) and functional leads, augmented by documented advisory / collaboration and an independent IV&V step — consistent with EoI provisions for MSMEs / start-ups.

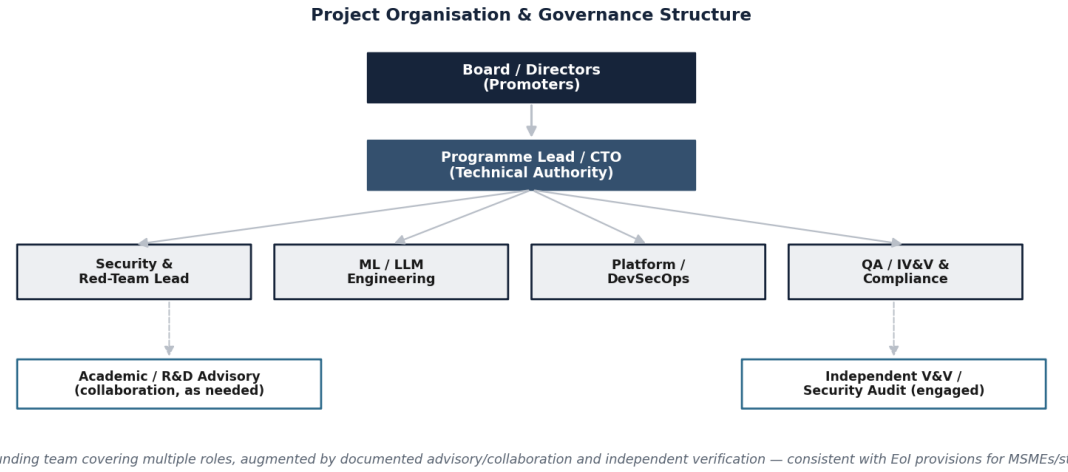



**Figure 56. Project organisation and governance structure**
*A small founding team covers multiple roles, augmented by documented advisory / collaboration and independent verification.*

### *(c) Shareholding pattern and major investors.*

### *i) The shareholding pattern of the firm.*

**Table 70. Shareholding pattern**

| Shareholder | Shares | Holding | Role |
|---|---|---|---|
| Mr. Vinod Kumar K B (HUF) | 4,000 | 40% | Non-director (Karta: Mr. Vinod Kumar Kokkattu Balakrishnan) |
| Ms. Suman Goyal | 2,000 | 20% | Director |
| Mr. Vartul Goyal | 2,000 | 20% | Director & CTO |
| Mr. Md Sajid Ansari | 2,000 | 20% | Non-director |
| **Total** | **10,000** | **100%** | |

Source: Register of members; firm declaration.

*Interpretation: Paid-up equity reconciles to 10,000 shares of Rs.10 each (Rs.1,00,000).*

### *ii) Majority shareholder and top five shareholders.*

The majority shareholder is **Mr. Vinod Kumar K B (HUF)** at 40%. The firm has four shareholders in total (all listed above), which constitute the top-five disclosure.

### *iii) Direct or indirect foreign shareholders (including foreign entities) holding more than 5% stake.*

**Nil.** There is no foreign shareholding, direct or indirect.

### *iv) Any direct or indirect shareholders of a country sharing a land border with India.*

**Nil.** No shareholder, direct or indirect, is from a country sharing a land border with India.

## 17. Financial Capabilities

The firm has a positive net worth. Its authorised share capital is Rs.10,00,000 and its paid-up share capital is Rs.1,00,000; it carries no accumulated losses as at incorporation. As the company was incorporated on 11 October 2025, its first full audited annual financial statements (balance sheet, profit & loss account and audit report) will be available following the close of financial year 2025–26 and will be furnished at the RFP stage. The firm self-certifies that it is **not** under liquidation, court receivership or any similar proceeding. (No project cost or commercial figures are stated at this EoI stage.)

## 18. Litigation / Arbitration History

The firm has no history of court litigation or arbitration proceedings arising from contracts executed in the last five years or currently under execution, and the same applies to any significant sub-contractor.

**Table 71. Litigation and arbitration history**

| Year | Decree / award for or against the applicant, or arbitration ongoing | Client, cause of litigation and matter in dispute | Disputed amount (current value) |
|---|---|---|---|
| — | Nil | Nil | Nil |

Source: Firm declaration.

*Interpretation: There is no litigation or arbitration history to report.*

## Statement of Compliance, Authenticity and Self-Certification

ZeroShield Private Limited makes the following statements in respect of this Expression of Interest response. These statements are made by the authorised signatory of the firm and are binding upon the firm.

1. **Authenticity.** All information, statements, and documents contained in or annexed to this response are true, correct, and complete to the best of our knowledge and belief, and are self-certified by the authorised representative of the firm. We understand that DTDF reserves the right to physically verify the originals of any document submitted, and that any material misstatement may result in disqualification.
2. **No mention of cost.** In strict compliance with clause 4.2.1 of the EoI, no cost, price, quotation, budgetary figure, or financial bid relating to the proposed project is stated anywhere in this response. Statutory share-capital figures, where reproduced, are public records of the Ministry of Corporate Affairs and are not project costs.
3. **Eligibility.** The firm is a private limited company registered under the Companies Act, 2013, holds valid MSME (Udyam) registration, is owned and controlled by resident Indian citizens, has nil foreign investment, and has no direct or indirect shareholder from any country sharing a land border with India.
4. **Standing.** The firm has never been debarred, banned, blacklisted, suspended, or put on hold by the Ministry of Defence or any other authority of the Government of India, and no such order applies to any allied or subsidiary entity.

5. **Solvency.** The firm is not under liquidation, court receivership, insolvency, or any similar proceeding, and has positive net worth.
6. **Litigation.** There is no court litigation or arbitration proceeding, decided or pending, arising out of any contract executed by the firm in the last five years or currently under execution.
7. **Data sovereignty undertaking.** All model weights, prompt corpora, vulnerability findings, and reports shall remain within the secure Indian perimeter throughout development, testing, and deployment.
8. **Responsible-disclosure undertaking.** No discovered vulnerability or zero-day shall be sold, transferred, or disclosed to any third party without the explicit written authorisation of the End-User.
9. **Confidentiality.** The firm is willing to execute a Non-Disclosure Understanding (NDU) with TDF as and when required, and to comply with all applicable security and accreditation requirements.
10. **No liability on DRDO.** The firm acknowledges that DRDO/DTDF bears no liability for any cost incurred by the firm in the preparation or submission of this EoI, and that the EoI process creates no binding commitment on DTDF.

For and on behalf of **ZeroShield Private Limited**

_____

**Vartul Goyal**
Director & Chief Technology Officer
DIN: 11337955 (Authorised Signatory)

Place: Kanpur, Uttar Pradesh | Date: ____ / ____ / 2026

ZEROSHEILD PRIVATE LIMITED
C3iHub, 2nd Floor, New Technopark Building (In front of NCC Office),
IIT Kanpur, Kalyanpur, Kanpur Nagar, Uttar Pradesh – 208016
Ph – +91 – 8088054916
www.zeroshield.ai

# Annexure A — Letter of Willingness and Undertaking

*(On the letterhead of ZeroShield Private Limited)*

Ref: ZS/EoI/DRDO-TDF/VAF-LLM/2026/01/A | Date: _____

To,

The Director, Technology Development Fund (TDF),
Defence Research and Development Organisation (DRDO),
DRDO Bhawan, Rajaji Marg,
New Delhi – 110011.

**Subject:** Expression of Interest for the design and development of a Comprehensive Vulnerability Assessment Framework for Large Language Models — EoI Reference DGTM/01/100010/M/48/VAF/PC1 — Letter of Willingness and Undertaking.

Respected Sir / Madam,

With reference to the above Expression of Interest, we, ZeroShield Private Limited, a company incorporated under the Companies Act, 2013 (CIN U62020UT2025PTC020093) and a registered micro-enterprise (Udyam UDYAM-UK-12-0096527), hereby express our willingness and interest to undertake the design, development and proof-of-concept demonstration of the Comprehensive Vulnerability Assessment Framework for Large Language Models under the Technology Development Fund.

We further undertake and confirm that:

1. We have read and understood the Expression of Interest and its terms and conditions, and we satisfy the eligibility criteria set out therein, being an Indian company owned and controlled by resident Indian citizens.
2. All model weights, corpora, prompts, intermediate data, findings and reports handled during the engagement shall remain within the secure, defence-controlled Indian perimeter at all times, and shall not be transmitted, copied or exposed to any external, foreign or cloud-based service at any stage.
3. Any vulnerability, zero-day or exploit discovered during the engagement shall be treated as sensitive and shall not be sold, published, transferred or otherwise disclosed to any party without the prior written authorisation of the End-User.
4. We have not, in this Expression of Interest, mentioned or implied any cost, price, budget or financial quotation in any form, in accordance with the EoI’s requirement.
5. There is no foreign shareholding in the company, and no promoter, director or beneficial owner is from a country sharing a land border with India.
6. The company has never been blacklisted, debarred or banned by any Government of India organisation or public-sector undertaking, and there is no litigation or arbitration involving the company.

7. The information furnished in this Expression of Interest is true, complete and correct to the best of our knowledge and belief, and we shall furnish such further documentation as may be required at the subsequent RFP stage.

For and on behalf of ZeroShield Private Limited,

_____

**Vartul Goyal**
Director & Chief Technology Officer
DIN: 11337955
Authorised Signatory

# Annexure B — Compliance Traceability Matrix

This matrix maps each information requirement of Part-II of the EoI, and the principal eligibility conditions of Part-III, to the section(s) of this document that respond to it. It is provided to assist the evaluation committee in locating the firm's response to each requirement.

**Table 72. Part-II / Part-III requirement-to-section traceability**

| EoI requirement | Response location | Status |
|---|---|---|
| Item 1 — Name and address of firm | Section 1; Front matter | Complied |
| Item 2 — Contact details | Section 2 | Complied |
| Item 3 — Title of technology / project | Section 3; Cover | Complied |
| Item 4 — Project description, WBS, critical tech, block diagram | Sections 4.4–4.6, 4.13; 4 | Complied |
| Item 5 — Development estimate; suo-moto | Sections 5.1, 5.1.4; 5 | Complied |
| Item 6 — Academic / research partners | Section 6 | Complied |
| Item 7 — Existing academia support | Section 7 | Complied |
| Item 8 — Indigenous content | Section 8.1; 8 | Complied |
| Item 9 — Risk mitigation plan | Section 9.1; 9 | Complied |
| Item 10 — Additional infrastructure | Section 10; Section 4.12 | Complied |
| Item 11 — Experience (A1–A6) | Section 11 | Complied |
| Item 12 — Test infrastructure (A1–A6) | Section 12 | Complied |
| Item 13 — Manpower (A1–A5) | Section 13; Annexure C | Complied |
| Item 14 — Management capabilities | Section 14.1; 14 | Complied |
| Item 15 — Undertakings (15a / 15b) | Section 15; Annexure A; Declaration | Complied |
| Item 16 — Organisation, legal status, shareholding | Section 16 | Complied |
| Item 17 — Financial capabilities | Section 17 | Complied |
| Item 18 — Litigation / arbitration (NIL) | Section 18 | Complied |
| Item 19 — Willingness / undertaking | Section 18; Annexure A | Complied |
| Part-III — Eligibility conformance | Section 18.21 | Complied |
| No mention of cost (clause 4.2.1) | Whole document | Complied |

Source: EoI Part-II and Part-III; this document.

*Interpretation: Every information requirement and eligibility condition of the EoI is addressed and located, providing the committee a single index to the firm's complete response.*

# Annexure C — Key-Personnel Profile and Role Specifications

This annexure provides the profile of the technical authority for the programme and the role specifications for the positions to be staffed as the programme mobilises. In keeping with the firm's commitment to accuracy, role specifications (responsibilities and required competencies) are provided for positions not yet filled, rather than illustrative or placeholder curricula vitae.

## *E.1 Technical Authority — Director and Chief Technology Officer*

**Table 73. Profile of the technical authority (Director and CTO)**

| Attribute | Detail |
|---|---|
| Role | Director and Chief Technology Officer; technical authority and programme lead. |
| Identifier | DIN 11337955 (as disclosed in the company records). |
| Domain focus | AI / LLM security, adversarial machine learning, secure software engineering, and security architecture. |
| Programme responsibilities | Overall technical direction; architecture ownership; security-assurance governance; stakeholder and accreditation interface; IV&V; coordination. |
| Relevant competencies | Design of adversarial-testing and red-teaming methods; air-gapped / offline engineering; standards alignment (OWASP LLM Top 10, MITRE ATLAS, NIST AI RMF); secure-development discipline. |

Source: Company records; the firm's capability statement.

*Interpretation: The programme is led by a director-level technical authority whose focus areas correspond directly to the critical technologies of the framework.*

A detailed curriculum vitae for the technical authority will be furnished at the RFP stage. The profile above is provided at the EoI stage in summary form and contains no fabricated credentials.

## *E.2 Role Specifications for Positions to be Staffed*

The following positions will be staffed against these specifications as the programme mobilises, through direct hiring and documented collaboration. The specifications define the responsibilities and the essential competencies for each role.

**Table 74. Role specifications for positions to be staffed**

| Role | Key responsibilities | Essential competencies |
|---|---|---|
| Security & Red-Team Lead | Own the attack-generation and agentic red-teaming modules; design jailbreak and injection corpora; adjudicate findings. | Offensive security; adversarial ML; prompt-injection and jailbreak techniques; scripting and automation. |
| ML / LLM Engineer(s) | Implement gradient and discrete attack methods; integrate inference serving; build the privacy-audit module. | PyTorch and the Transformers ecosystem; GCG / AutoDAN / PAIR; membership-inference (LiRA / RMIA); model serving. |
| Platform / DevSecOps Engineer | Build and harden the air-gapped environment; offline dependency management; CI and signed-job execution; observability. | Containers and orchestration; sandboxing (micro-VM); secrets management; CIS hardening; offline mirroring. |
| QA / IV&V; & Compliance Engineer | Define and run verification and validation; measure figures of merit; maintain quality records and standards mapping. | Test engineering; benchmarking; OWASP / ATLAS / NIST mapping; configuration management and audit. |
| Indian-Language NLP Specialist (collaborative) | Curate and validate Indian-language and code-mixed attack corpora and harm taxonomies. | Hindi / Tamil / Telugu / Bengali NLP; transliteration and code-mixing; corpus curation and annotation. |

Source: Programme staffing plan (Section 5.1) and module design (Section 4.6).

*Interpretation: Each role maps to a defined slice of the technical scope; the specifications give the End-User a transparent view of how the delivery team will be constituted.*

# Annexure D — References, Standards and Bibliography

The technical approach in this document draws on the following recognised standards, frameworks and research. The list is provided for the evaluator's reference; citations are to publicly recognised works and standards.

### *F.1 Standards and Frameworks*

1. OWASP Foundation, "OWASP Top 10 for Large Language Model Applications" (2025 edition).
2. MITRE, "ATLAS — Adversarial Threat Landscape for Artificial-Intelligence Systems" (adversarial tactics and techniques knowledge base).
3. National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)", NIST, 2023.
4. NIST, "Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations" (NIST AI 100-2).
5. International Organization for Standardization, "ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system".
6. International Organization for Standardization, "ISO/IEC 27001:2022 — Information security, cybersecurity and privacy protection — Information security management systems".
7. OWASP Foundation, "Machine Learning Security Top 10".

### *F.2 Selected Research and Technical Literature*

1. Zou, A., Wang, Z., Kolter, J. Z., and Fredrikson, M., "Universal and Transferable Adversarial Attacks on Aligned Language Models" (Greedy Coordinate Gradient / GCG), 2023.
2. Liu, X. et al., "AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models", 2023.
3. Chao, P. et al., "Jailbreaking Black-Box Large Language Models in Twenty Queries" (PAIR), 2023.
4. Carlini, N. et al., "Membership Inference Attacks From First Principles" (LiRA), IEEE S&P, 2022.
5. Zarifzadeh, S. et al., "Low-Cost High-Power Membership Inference Attacks" (RMIA), 2024.
6. Shokri, R. et al., "Membership Inference Attacks Against Machine Learning Models", IEEE S&P, 2017.
7. Carlini, N. et al., "Extracting Training Data from Large Language Models", USENIX Security, 2021.
8. Hubinger, E. et al., "Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training", 2024.
9. Greshake, K. et al., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", 2023.
10. Research and tooling on insecure deserialisation of model artefacts (e.g. malicious Python pickle objects) and their detection.

*References to research are provided to indicate the technical basis of the proposed methods; the firm will apply, adapt and extend such methods within the indigenous framework and the secure perimeter.*

# Annexure E — Glossary of Key Terms

The following glossary defines key technical terms used in this document, for the convenience of all readers of the evaluation committee.

**Table 75. Glossary of key technical terms**

| Term | Definition |
|---|---|
| Adversarial suffix | A sequence of tokens, often produced by optimisation, appended to a prompt to induce a model to violate its safety behaviour. |
| Agentic red-teaming | Automated, multi-turn security testing in which one or more AI agents plan, execute and judge attacks against a target model. |
| Air-gap | A security measure in which a system has no network connection to external or untrusted networks, so data cannot egress electronically. |
| Canary | A deliberately inserted, identifiable data record used to test whether a model has memorised or will leak specific training data. |
| GCG (Greedy Coordinate Gradient) | A gradient-guided method for generating adversarial prompt suffixes against language models. |
| Jailbreak | An input crafted to bypass a model's safety alignment and elicit prohibited or harmful output. |
| Membership inference | An attack that determines whether a particular record was part of a model's training data, indicating privacy leakage. |
| MITRE ATLAS | A knowledge base of adversarial tactics and techniques against AI systems, analogous to MITRE ATT&CK for AI. |
| OWASP LLM Top 10 | A community-maintained list of the most critical security risks to applications built on large language models. |
| Prompt injection | An attack that inserts adversarial instructions into a model's input — directly or via retrieved content — to subvert its intended behaviour. |
| Sandbox | An isolated execution environment that contains untrusted code or artefacts so that any malicious behaviour cannot affect the host or wider system. |
| safetensors | A safe serialisation format for model weights that avoids the code-execution risks of formats such as Python pickle. |
| Sleeper agent | A model containing hidden, trigger-activated malicious behaviour that persists despite ordinary safety training. |
| WORM (Write-Once-Read-Many) | A storage property ensuring that records, once written, cannot be altered — used for tamper-evident audit and evidence. |
| Zero-day | A previously unknown vulnerability for which no patch or public knowledge yet exists. |

Source: Compiled for this document.

*Interpretation: The glossary supports a common understanding of the specialist terminology used throughout the proposal.*

# Annexure F — Defence-Specific Threat Considerations and Assurance Rationale

Because the framework is intended for the assessment of language models in a defence context, this annexure sets out the defence-specific threat considerations that shape its design, and the rationale by which the framework provides assurance against them. It complements the general threat taxonomy of Section 4.2 with the considerations that are particular to sensitive and operational defence use.

### *H.1 Defence-Specific Threat Scenarios*

The following scenarios are representative of the threats that a defence organisation must consider when adopting a language model, and which a sovereign assessment capability must be able to address.

- **Model and weight exfiltration.** A sensitive or fine-tuned model represents concentrated intellectual property and operational knowledge; any assessment process that exposes the weights to an external environment is itself a threat. The framework removes this threat by operating entirely within the air-gapped perimeter with zero egress.
- **Poisoned or backdoored procured models.** A model obtained from an external or untrusted source may carry hidden triggers or sleeper-agent behaviour. The framework tests for these through static artefact analysis and embedding / behavioural probing (Modules M2 and M5).
- **Supply-chain compromise.** The artefacts and dependencies of a model may be tampered with or carry malicious serialised content. The framework verifies provenance and integrity and detects malicious artefacts before a model is trusted (Module M2).
- **Operational disinformation in Indian languages.** A model deployed in an operational context may be induced to produce harmful or false output, including in Indian languages and code-mixed forms that English-centric tools do not exercise. The framework includes Indian-language and India-specific harm coverage (Modules M3 and M4).
- **Disclosure of sensitive training data.** A model trained on sensitive material may leak that material under adversarial questioning. The framework quantifies this risk through membership inference and extraction testing (Module M5).
- **Excessive agency in connected deployments.** A model with access to tools or actions may be induced to exceed its authority. The framework probes the agency and permission surface (Module M4 and M5).

### *H.2 Defence-Concern-to-Capability Mapping*

**Table 76. Mapping of defence-specific concerns to framework capabilities**

| Defence concern | Framework capability that addresses it |
|---|---|
| Sovereignty of sensitive models and data | Air-gapped operation; one-way import; zero egress (Sections 4.11, 12). |
| Trust in procured / external models | Static artefact analysis and poisoning / backdoor detection (M2, M5). |
| Operational safety of model outputs | Adversarial generation and agentic red-teaming across harm categories (M3, M4). |
| Protection of sensitive training data | Membership inference and extraction auditing (M5). |
| India-specific and multilingual threats | Indian-language corpora and India-relevant harm taxonomy (M3, M4). |
| Accreditation and re-testing | Standards-mapped, reproducible, certification-grade evidence (M6, Section 4.15). |
| Freedom from foreign-vendor dependence | Indigenous design and intellectual property; offline operation (Section 8.1). |

Source: Defence threat considerations (this annexure) and the module design (Section 4.6).

*Interpretation: Each defence-specific concern is addressed by an identifiable capability of the framework, evidencing fitness for the defence use case the EoI describes.*

### *H.3 Assurance Rationale*

The assurance the framework provides rests on three pillars. The first is containment: by operating air-gapped with a one-way import and verified-nil egress, the framework guarantees that the act of assessing a model does not itself create a sovereignty or exfiltration risk. The second is comprehensiveness anchored to recognised references: by covering the OWASP LLM Top 10 and tagging findings to MITRE ATLAS and NIST AI RMF, the framework produces evidence that is both broad and defensible within taxonomies the accrediting authority already recognises. The third is reproducibility and integrity of measurement: by seeding and recording every run in a tamper-evident log and by separating capability-building from measurement, the framework ensures that its results can be relied upon and independently verified. Together these pillars allow a defence accrediting authority to base a deployment decision on the framework's output with confidence.

### *H.4 Boundary of Assurance*

In the interest of candour, the framework's assurance has a defined boundary. It assesses the security posture of a model against the vulnerability classes in scope at the time of assessment; it does not certify that a model is free of all possible vulnerabilities for all time, since the threat landscape evolves and no assessment can be exhaustive. The framework therefore supports periodic re-assessment, and its corpora and techniques are designed to be refreshed under national control as new attack classes emerge (Section 4.21.5). This honest statement of the boundary of assurance is itself part of responsible security practice.

**End of document.** This Expression of Interest has been prepared and submitted by ZeroShield Private Limited in response to DRDO TDF EoI reference DGTM/01/100010/M/48/VAF/PC1. The information furnished herein is true and correct to the best of the firm's knowledge and belief, and contains no mention of cost in any form, in accordance with the terms of the Expression of Interest.